The Charlotte Operating System

An async-first, capability-based microkernel
for critical, high-performance software

2026-08-27

Scope and status

This manual describes both the implemented CharlotteOS prototype and its intended architecture. Unless a section explicitly says otherwise, statements about security, performance, transport independence, hardware support, or failure recovery are design goals rather than production guarantees. The implementation-status appendix records the repository's currently verified scope; source code and tests remain authoritative when the manual and implementation differ. Repository paths in this manual are clickable links to the corresponding file or directory on the public `main` branch; begin with the `docs/README.md` documentation index.

Contents

1 The Problem

1.1 The operating system is the bottleneck

Critical software must be both **correct** and **fast**. Modern systems often pursue both goals by adding layers to operating systems whose basic abstractions were designed for timesharing.

This manual calls the resulting complexity and performance cost the **retrofit tax**: a synchronous, ambient-authority, monolithic system must be made to behave as though it were asynchronous, least-privilege, and decomposed.

1.2 The retrofit tax, itemized

1.2.1 Async over blocking

Many operating systems expose synchronous calls: `read()` may block the calling thread until data arrives. Applications use `epoll`, `kqueue`, or `io_uring` for concurrency without dedicating one thread per operation. These interfaces are effective, but applications may still need adapters when a kernel or library operation blocks. Runtimes then translate blocking work into readiness or completion notifications.

This translation layer brings with it:

- Async-signal-safety problems — the kernel may deliver a completion while userspace holds a lock, requiring elaborate signal-handler discipline or a dedicated thread pool.
- Thread-pool overhead — many async runtimes spawn a pool of OS threads to convert blocking syscalls into futures, undoing the resource benefits of asynchronous execution.
- Two concurrency models — the application uses async/await; the kernel uses threads and blocking. The mismatch forces every boundary crossing to bridge the two models.

1.2.2 Ambient authority

A conventional OS process has access to the filesystem namespace, the network, and other global resources by default. Security is subtractive: you start with everything and try to take things away (chroot, seccomp, namespaces). This is brittle because it requires the developer to enumerate what *should not* be accessible — an unbounded problem. In practice, most programs run with far more authority than they need, and exploits exploit that gap.

1.2.3 Hardware asynchrony disguised as software synchrony

Hardware is inherently asynchronous with respect to the CPU. A disk controller signals completion via an interrupt. A network card delivers packets on its own schedule. A timer fires after a programmed interval. Yet the operating system interface presents these as synchronous calls: `read()` returns when the data is ready, blocking the calling thread in the meantime.

A thread blocked on I/O cannot do other work. High-concurrency servers compensate with thread pools; real-time systems may use carefully tuned polling loops.

1.2.4 Fault propagation across trust boundaries

A bug in a device driver can crash a monolithic kernel. A memory corruption in one application can, in a system without hardware-enforced isolation, corrupt another. Conventional microkernels solve this with address-space isolation, but at the cost of IPC overhead that often drives designers back toward monolithic architectures for performance.

A common compromise is to isolate untrusted code in virtual machines while keeping trusted code in the kernel. Kernel defects can still cross that boundary.

1.3 Why Rust ownership is not enough

Safe Rust prevents many data races, use-after-free errors, and null-pointer dereferences. Unsafe code, hardware interfaces, and logic errors remain. This is a substantial improvement for systems programming, but Rust programs run on operating systems that do not share Rust's ownership discipline.

- A Rust program cannot protect itself from a C driver that corrupts kernel memory. Isolation must be enforced by the hardware (MMU, IOMMU), not by the language.
- A Rust `Mutex<Vec<T>>` crossing a syscall boundary is still a shared-memory lock with all the contention, priority inversion, and deadlock risks that implies. The kernel does not know the lock exists.
- Rust's `async`/`await` compiles to a state machine driven by a userspace executor, but the executor must still interact with the kernel's blocking I/O model. The `Future` trait and the kernel's wait queue speak different languages.

Rust prevents some bug classes within an address space. The OS still defines the rules between address spaces. If authority is ambient until restricted, the language's guarantees remain local.

1.4 What needs to change

We need an operating system whose foundational abstractions align with the demands of critical, performant software:

1. Asynchronous by construction — operations that may wait for external progress should have submission/completion semantics, not blocking-call semantics. No subsystem should force a thread to dedicate itself to one operation.
2. Capability-based, not ambient, authority — a process should be able to perform an operation only if it holds a capability authorizing it. Capabilities are unforgeable and must be explicitly granted.
3. Isolation by default — every protection domain should run in its own address space. Communication crosses a kernel-mediated boundary. A fault in one domain should not propagate to another.
4. Ownership that crosses boundaries — data movement between domains should transfer ownership, not copy bytes. The kernel should move page mappings, so the sender loses access and the receiver gains it.

5. Bounded resource consumption — every queue, table, and buffer should be bounded. Backpressure should propagate explicitly. No unbounded allocation should be a correctness requirement.
6. The OS and the language runtime should agree — the kernel's model of concurrency (asynchronous, completion-driven) should match the language's model of concurrency (futures, wakers). No translation layer should be needed.

CharlotteOS is built around these six principles. The rest of this manual describes the architecture, programming model, implementation status, and evidence for its safety claims.

2 Design Philosophy

CharlotteOS combines ideas from three projects around one thesis.

2.1 The combined thesis

> **Hardware is asynchronous; the operating system and the userspace runtime should be too.**

Three systems approach the same design space from different directions. Rather than collapsing their abstractions into one universal mechanism, the architecture composes them at the right boundaries.

2.1.1 CharlotteOS — the protection, scheduling, and interrupt substrate

CharlotteOS contributes the mechanisms that *must* be privileged:

- Address-space isolation through page tables and the MMU.
- Kernel scheduling of threads across logical processors.
- LP-local state and interrupt routing.
- Page-table manipulation and physical-memory management.
- MMIO and DMA authority delegation.
- Per-domain capability tables.
- Event observation and thread wakeup.
- Syscall dispatch and asynchronous completion delivery.

Its design rule is: **the kernel provides the minimum set of primitives from which everything else can be composed.** If a facility can be built in userspace without compromising isolation, it belongs in userspace.

2.1.2 Sitas — the shard-per-core execution discipline

Sitas (`https://github.com/FrodeRanders/sitas`) contributes an execution discipline — a set of rules about how state is owned and mutated:

> Only the owning shard mutates shard-local state. Cross-shard interaction occurs through bounded typed messages carrying owned values.

Sitas provides:

- One executor per shard — cooperative task scheduling within a shard; preemption between shards.
- Futures as userspace state machines — every operation that may wait is a `Future`, driven by the shard's executor.
- Bounded backpressure — every queue has fixed capacity; senders must handle `WouldBlock`.

- Shard-local ownership — mutable state belongs to exactly one shard; no locks, no atomics for shard-internal data.
- Explicit placement — where a task runs is a deliberate choice, not an accident of scheduling.
- Typed command/reply APIs — what a shard receives is a specific Rust type, not a generic byte buffer.
- Structured shutdown and cancellation — tasks can be cancelled, and cancellation has defined ownership semantics.
- Observability through owned snapshots — diagnostics capture state by value, never by borrowed reference into runtime internals.

The sitas discipline is enforced by Rust's type system within a single protection domain. CharlotteOS extends it across domains through the kernel.

2.1.3 Xous — the isolated userspace service model

Xous, a microkernel OS for Precursor and Betrusted hardware, contributes the protection-domain service model:

- Services are userspace servers — drivers, filesystems, and protocol stacks run in their own protection domains, not in the kernel.
- Clients connect to servers and receive connection identifiers — a connection *is* authority; possessing one means you may communicate with that server.
- IPC is a kernel primitive — communication is not a convention layered on shared memory; it is mediated and validated by the kernel.
- Scalar messages are distinct from memory-bearing messages — small control-plane operations (opcodes, handles, status codes) travel in fixed-size messages; bulk data travels as memory-object attachments.
- Memory IPC distinguishes ownership transfer from borrowing — move transfers ownership permanently. Borrow grants temporary access, revoked on reply or cancellation.
- A userspace name service maps human-readable names to opaque server identities — the kernel knows nothing about strings.

CharlotteOS adopts Xous's userspace-server model and combines it with the sitas execution discipline and an async-native kernel. The detailed co-design is maintained in `docs/architecture/sitas-xous.md`.

2.2 How they fit together

The three systems do not collapse into one. They compose at well-defined boundaries:

Layer	Provided by
Shard-local async execution	sitas
Cross-domain protection and IPC	CharlotteOS kernel (Xous-inspired)
Completion delivery for kernel/device ops	CharlotteOS kernel
Service discovery and policy	Userspace (name service, supervisor)

The layers are:

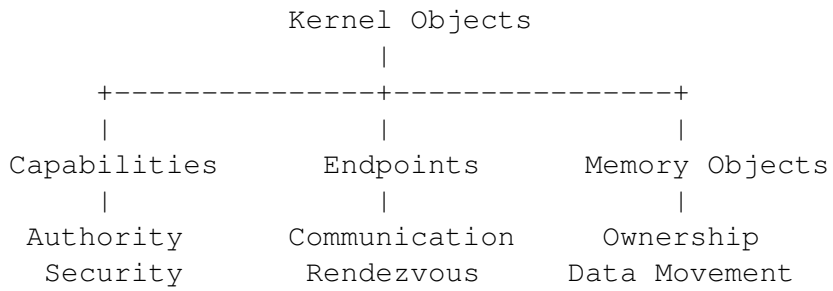
```
CharlotteOS kernel
    protection domains
    threads and LP scheduling
    capabilities and endpoint connections
    interrupt routing
    memory objects and page mappings
    operation submission and completion queues

Userspace service process
    one or more externally visible IPC endpoints
    protocol-specific request/reply handling
    one sitas executor per assigned shard
    shard-local state
    typed in-process commands
    driver or service implementation

Client process
    typed protocol library
    endpoint connection capabilities
    futures backed by IPC replies or completion records
    owned buffers whose transfer mode is explicit
```

2.3 The three kernel primitives

Three orthogonal kernel abstractions form the foundation:



Every other kernel facility — completion queues, reply tokens, device grants, DMA mappings — is a **composition** of these three primitives.

2.3.1 Capabilities: “May I?”

A capability answers one question: **“May I perform this operation?”** It represents authority, not work. Examples include endpoint capabilities, connection capabilities, memory-object capabilities, interrupt capabilities, DMA-domain capabilities, and timer capabilities.

2.3.2 Endpoints: “With whom?”

An endpoint answers: **“Who am I communicating with?”** It provides bounded rendezvous between protection domains. Endpoints know nothing about packet buffers, futures, DMA, or ownership. They carry requests and replies.

2.3.3 Memory Objects: “Where is the data, and who owns it?”

A memory object answers: **“Where is the data, and who currently owns it?”** It represents page-backed storage whose ownership can move independently of communication. Its responsibilities include page ownership, mappings, transfer modes, DMA state, and lifetime.

2.4 An explicit design rule

> **When adding a new subsystem, first ask whether it can be expressed using capabilities, endpoints, and memory objects. New kernel primitives should be introduced only when they cannot be naturally composed from the existing ones.**

This rule limits the kernel interface and keeps the ABI and trust model reviewable.

2.5 What “asynchronous throughout” means

“Asynchronous throughout” means:

1. Operations that may wait for external progress have submission/completion semantics.
2. No subsystem boundary forces the caller to dedicate a thread to waiting for one operation.
3. A userspace shard can wait on one aggregated kernel source (the CQ).
4. Completions are retained until observed.
5. Buffers and capabilities have explicit ownership during an operation.
6. Cancellation and teardown have defined state transitions.
7. Bounded queues preserve backpressure across layers.
8. Service implementations can suspend one task while continuing other work on the shard.

This is not the same as saying:

- Every syscall is asynchronous.
- Every call returns a newly allocated completion capability.
- No thread ever blocks.
- Every wake sends an IPI.
- All waitable objects have identical semantics.
- Userspace code is interrupted by arbitrary asynchronous callbacks.

2.5.1 Blocking is still useful

When no sitas task is runnable, the shard's execution context should block in one kernel wait operation:

```
ready tasks exist
    -> poll tasks

no ready tasks
    -> wait on CQ / endpoint readiness / deadline

event arrives
    -> execution context becomes runnable
    -> drain events
    -> wake relevant futures
```

> **The shard parks directly on native completion and message state; no helper-thread pool is required merely to convert blocking kernel interfaces into futures.**

2.5.2 Immediate operations remain immediate

Operations that cannot meaningfully block — capability duplication, metadata queries, nonblocking CQ drain, closing a quiescent object — complete synchronously. The ABI does not force asynchronous ceremony onto inherently synchronous work.

2.5.3 No asynchronous userspace upcalls

Completions do not inject callbacks into userspace at arbitrary instructions. The path is:

```
hardware interrupt
    -> kernel records result or notification
    -> blocked execution context becomes runnable
    -> normal return to userspace
    -> sitas executor drains CQ/endpoints
    -> executor wakes relevant Rust tasks
```

The kernel may preempt threads, but it does not inject arbitrary task callbacks into userspace. Task scheduling therefore remains cooperative and reviewable.

2.6 Why this matters for critical software

Every design decision in CharlotteOS traces back to one or more of the six principles from Chapter 1:

- Capabilities replace ambient authority — a compromised service cannot access resources it was never granted. The set of things a process can do is enumerable and auditable by construction.

- Endpoints make communication authenticated — a client that holds a connection capability has proven (at grant time) that it is authorized to communicate. The kernel enforces this on every message.
- Memory objects constrain stale cross-domain access — ownership moves through the kernel and is enforced by page tables. Correctness still depends on MMU, TLB, and DMA handling.
- Composition limits the privileged code footprint — most system functionality lives in userspace, where faults can be contained to a service domain.
- The async model reduces adaptation — the kernel and runtime share submission, completion, and wake semantics.
- Bounded ingress makes pressure visible — endpoint and hardware queues apply explicit backpressure. Chapter 10 records the remaining unbounded internal backlog.

3 Vocabulary

This chapter defines the manual's CharlotteOS-specific terms.

3.1 Protection domain

A **protection domain** is an address space and authority boundary. It owns:

- A capability table.
- Memory mappings.
- One or more execution contexts (threads).
- Resource accounting.
- Failure and teardown state.

A *process* is the concrete implementation of a protection domain, but architectural discussion prefers the semantic term.

3.2 Execution context

An **execution context** is a kernel-scheduled thread. It may have:

- LP affinity.
- Priority or scheduling class.
- A current protection domain.
- A wait state.
- An associated userspace executor.

3.3 Logical processor (LP)

A **logical processor** is CharlotteOS's representation of a schedulable hardware execution context — typically one CPU core (or one hyperthread).

An LP is **not** a shard (in the sitas sense). A typical mapping is:

```
one sitas shard
    <-> one LP-affine userspace thread
    <-> normally one logical CPU
```

The architecture retains the distinction so that CPU hotplug, oversubscription, migration, asymmetric CPUs, and multiple sharded processes remain possible.

3.4 Sitas shard

A **shard** is a userspace ownership and execution domain:

- One executor runs its tasks.
- At most one task at a time mutates shard-owned state.
- Cross-shard access occurs through typed messages carrying owned values.
- Placement is explicit but is not the shard's identity.

3.5 Capability

A **capability** is an unforgeable reference to a kernel object, carrying rights. It answers: "May I perform this operation?"

Capabilities are scoped to a protection domain's capability table. A process cannot fabricate a capability; it can only receive one through the bootstrap contract or through IPC.

3.6 Endpoint

An **endpoint** is a bounded server-side IPC receive object. It has:

- An owning protection domain.
- A queue capacity.
- A protocol/interface identity.
- Receive rights.
- Lifecycle and closure state.

3.7 Connection capability

A **connection capability** is a client-side authority to send or call an endpoint. Possession of a service name is not authority. A name service resolves a name and conditionally returns a connection capability.

3.8 Message

A **message** is a kernel-validated IPC envelope consisting of:

- Interface or protocol identifier.
- Opcode.
- Flags.
- Inline scalar words.
- Zero or more memory-object attachments.
- Zero or more delegated capability attachments.

- Optional reply authority.

The kernel understands the envelope structure, not arbitrary Rust types.

3.9 Reply token

A **reply token** is one-shot authority to complete a blocking or deferred call. It is:

- Created by the kernel.
- Bound to the caller's pending call.
- Consumable exactly once.
- Invalidated by cancellation, caller death, or endpoint teardown.
- Optionally delegable when explicitly allowed.

3.10 Resource capability

A **resource capability** authorizes operations on a kernel-managed object such as a memory object, endpoint, connection, interrupt, MMIO region, DMA domain, device, completion queue, or timer.

3.11 Operation and completion queue

An **operation** is submitted work whose progress may depend on the kernel, hardware, or a privileged service.

A **completion queue (CQ)** is the aggregated, per-shard or per-process destination for operation results. It is waitable (with `CQ_WAIT`) and has non-lossy terminal completion semantics.

3.12 Notification

A **notification** means: "State may have changed; inspect the corresponding object or queue." It may be coalesced. It is not itself a result.

3.13 Rust Waker

A Rust `Waker` means: "Poll this userspace future again." It is a userspace scheduling hint. It is **not** a kernel completion record and should not cross the ABI directly.

3.14 Distinctions that matter

Several concepts sound similar but must be kept separate. Confusing them leads to architectural mistakes.

Concept	Is	Is not
Capability	Unforgeable authority to act	An event, a file descriptor, a handle
Endpoint	Server-side bounded receive queue	A socket, a pipe, a channel
Completion	A result of an async operation	A Waker, a notification
Waker	A hint to repoll a future	A kernel completion record
Shard	A userspace ownership domain	An LP, a core, a thread
LP	A schedulable hardware context	A shard
Notification	“Inspect state”	A result, a data payload
IPC call	Cross-domain service invocation	A kernel operation submission

3.15 Quick reference: acronyms

ASID Address Space Identifier. 0 = kernel; >0 = user address space.

CQ Completion Queue. Shared-memory ring buffer for async operation results.

EL0 Exception Level 0 on AArch64. CharlotteOS also retains `EL0` as an architecture-neutral source and test label for userspace execution, including ring 3 on x86-64.

EL1 Exception Level 1 (AArch64). Kernel execution.

HHDM Higher Half Direct Mapping. Linear mapping of all physical memory into kernel virtual address space (provided by the Limine bootloader).

IPI Inter-Processor Interrupt. Signal from one LP to another.

LP Logical Processor. One schedulable hardware execution context.

MMU Memory Management Unit. Hardware that enforces page-table permissions.

IOMMU / SMMU I/O Memory Management Unit. Enforces DMA access permissions from devices, analogous to the MMU for CPU access.

4 Capabilities — Authority Without Ambiguity

A capability is the answer to one question: **"May I perform this operation?"** It is authority, not work. CharlotteOS capabilities are unforgeable, delegable, and attenuable.

4.1 What a capability is

A capability is a per-address-space handle that names a kernel object and carries a rights mask. The kernel maintains a **capability table** for each protection domain. An entry in this table is the only way a userspace process can reference a kernel object.

Capabilities have four key properties:

- Unforgeable — a process cannot create a capability through computation. Capabilities enter a domain only through the bootstrap contract or through IPC delegation.
- Scoped — a capability is valid only in the domain that holds it. Capability index 3 in domain A refers to a different object (or nothing) than capability index 3 in domain B.
- Righted — a capability carries a rights mask specifying which operations are permitted. An endpoint capability might carry only `SEND | CALL` rights; it does not grant `RECEIVE`.
- Revocable — when a kernel object is destroyed, all capabilities referencing it become invalid. Subsequent use returns an error, not undefined behavior.

4.2 Object types and their rights

CharlotteOS defines a fixed set of kernel object types, each with its own rights model. A capability carries both a type tag and a rights mask.

Type	Example rights	What it authorizes
Endpoint	RECEIVE, MINT_CONNECTION	Server-side IPC receive queue
Connection	SEND, CALL, DELEGATE	Client-side authority to invoke a service
ReplyToken	COMPLETE, DELEGATE	One-shot completion of a pending call
CompletionQueue	SUBMIT, WAIT, DRAIN	Submission to / waiting on a CQ
MemoryObject	MAP, UNMAP, MOVE, LEND, COPY	Page-backed memory ownership and access
Interrupt	ACK, MASK, BIND_TO_CQ	Hardware interrupt handling
MmioRegion	MAP_DEVICE	Device register access (MMIO)
DmaDomain	PIN, UNPIN	DMA buffer management

Device	RESET, CONFIGURE	Device lifecycle management
Timer	ARM, CANCEL, BIND_TO_CQ	Timer operations

4.3 Delegation and attenuation

Capabilities can be **delegated** to another protection domain through IPC. When delegating, the sender may **attenuate** the rights: the receiver gets a subset of the rights the sender held. A service manager that holds a connection with `SEND | CALL | DELEGATE` rights can delegate a client-facing connection with only `SEND | CALL`, keeping `DELEGATE` for itself.

Delegation happens in the kernel, mediated by the IPC path. The sender specifies the target domain, the capability to delegate, and the reduced rights mask. The kernel validates that the sender actually holds the capability with at least the rights being transferred.

4.4 Generation-safe lookup

Capability tables use generation counters to prevent use-after-free through stale indices. When a capability is revoked, the table slot is freed and its generation is incremented. Any attempt to use an old index with a stale generation returns `InvalidCapability` rather than accessing a different object that happened to be allocated at the same index.

4.5 Bootstrap — how capabilities enter a domain

A newly created protection domain starts with an empty capability table. It receives its initial capabilities through the **config-page contract**: a mapped page that the supervisor populates with typed entries before the domain begins execution. The bootstrap contract delivers at minimum:

- A connection to the name service (for service lookup).
- For driver domains: device capabilities (MmioRegion, Interrupt).
- For service domains: an endpoint on which to listen.
- A default CQ handle for submission and waiting.

After bootstrap, additional capabilities arrive through IPC: the name service delegates service connections; the supervisor delegates device grants; other services delegate whatever the protocol authorizes.

4.6 Why capabilities matter for critical software

1. Least privilege is architectural, not advisory — a process starts with nothing. Every capability it holds was explicitly granted by a trusted component (the supervisor, the name service, or another service under a protocol). There is no ambient authority to strip away; the default is zero.

2. Kernel-mediated authority is enumerable — a process's capability table records the kernel objects it may use. This makes authority easier to audit, but does not by itself account for device side effects, protocol state, or implementation defects.
3. Delegation is mediated — a process cannot grant authority it does not hold; it cannot grant rights beyond what it possesses; and every delegation crosses the kernel, which enforces both constraints.
4. Teardown revokes the destroyed domain's capabilities and closes its endpoint relationships. Existing tests cover stale connections and pending-call reconciliation; this is not a claim of system-wide atomic revocation for every future object type or external device effect.
5. Capability tables are bounded — a process cannot create unlimited capabilities to exhaust kernel memory. Submission backpressure (`WouldBlock`) applies when a table is full, forcing the application to drain completions before submitting more work.

Capabilities give endpoints and memory objects explicit authority instead of making them ambient resources addressed only by an index or address.

5 Endpoints — Communication as a First-Class Primitive

An endpoint answers: **"Who am I communicating with?"** It is CharlotteOS's primary primitive for calls between protection domains.

5.1 Server and connection model

An **endpoint** is a bounded server-side IPC receive object. A service creates one and receives messages on it. A **connection capability** is a client-side authority to send to that endpoint. Connections are minted from the endpoint (typically by the name service) and delegated to clients.

```
Service creates endpoint:   endpoint_create(interface="ns", capacity=16)
Name service mints conn:    connection_mint(endpoint, SEND | CALL)
Client receives conn:       name_service.lookup("ns") -> ConnectionCap
```

Ordinary clients do not mint arbitrary connections. The name service or a policy service normally delegates them after an access check.

5.2 Message classes

The ABI distinguishes two message classes:

5.2.1 Scalar messages

A scalar message carries an opcode, one 64-bit argument word, and optionally a reply token. It fits in registers. It is suitable for:

- Opcodes and request types.
- Capability handles.
- Status values and queue indices.
- Compact control-plane operations where the payload is a single word.

Scalar messages require no memory-object allocation, page-table change, or TLB invalidation.

5.2.2 Memory-bearing messages

A message can carry one or more **memory-object attachments** with explicit transfer modes (see Chapter 6). These are suitable for:

- Structured requests and responses.

- Strings, names, and configuration structures.
- Packet payloads and block I/O buffers.
- Larger replies that exceed register width.

The kernel validates each attachment: the sender must hold the referenced memory object with sufficient rights for the declared transfer mode. The receiver gets a new capability to the memory (or a temporary mapping, for borrows).

5.3 Message envelope

The kernel validates a fixed-width message envelope. It does not understand Rust enums, `serde` formats, or protocol-specific types:

```
struct MessageHeader {
    interface: u128,       // protocol identity
    version: u32,
    opcode: u32,
    flags: u32,
    inline_len: u16,
    segment_count: u16,
    capability_count: u16,
    reserved: u16,
    user_tag: u64,
}
```

Constraints on the ABI:

- Fixed-width fields — no `usize`, no pointer-sized values.
- Explicit alignment — no compiler-dependent layout.
- Checked offsets and lengths — the kernel rejects out-of-bounds.
- Explicit protocol versioning — forward compatibility is negotiated.
- Maximum attachment counts — bounded kernel work per message.
- No arbitrary userspace pointers as durable message identity.

5.4 Call and reply

The most common IPC pattern is **call/reply**: a client sends a request and expects a response.

5.4.1 Synchronous-looking futures

At the Rust level, a client writes:

```
let response = network_service.connect(address).await?;
```

Underneath, the runtime:

1. Sends a call message through the connection capability.
2. The kernel creates a one-shot **reply token** bound to this call.
3. The caller task becomes pending (the `Future` returns `Poll::Pending`).
4. The caller shard continues running other tasks — no thread is dedicated to this request.
5. The server receives the message on its endpoint.
6. The server replies (immediately or later).
7. Reply arrival wakes the client future, which returns `Poll::Ready(response)`.

5.4.2 Deferred replies

A server may retain a reply token and complete it later — while continuing to serve other requests on the same shard. This supports device-driven operations:

```
async fn handle_read(pending: PendingRequest) {
    // Submit a DMA read, retaining the reply token.
    // Continue serving other clients while the read is in flight.
    let buffer = device.read(address, len).await?;
    pending.reply_with_buffer(buffer);
}
```

The shard's executor polls other tasks while the read completes.

5.4.3 Reply token semantics

A reply token is:

- Created by the kernel when a call message is enqueued.
- Bound to the pending call — it completes exactly that caller's future.
- Consumable exactly once — a second reply attempt returns an error.
- Invalidated by cancellation — if the caller times out or aborts, the reply token becomes inert.
- Invalidated by endpoint teardown — if the server dies, all outstanding reply tokens are invalidated, and all waiting callers receive `ServiceRestarted`.
- Optionally delegable — a server may forward a reply token to another service, allowing a chain of services to contribute to a single reply.

5.5 Connection delegation

A connection capability can be delegated to another domain with attenuated rights. This is the mechanism by which the userspace name service returns service-level connections:

```
// Name service:
let full_connection = name_service.lookup("driver.nic.v1")?;
```

```
// full_connection has SEND | CALL | DELEGATE rights

// Delegate to a client with reduced rights:
connection_delegate(full_connection, client_domain, SEND | CALL)?;
// client_domain now holds a connection with SEND | CALL only
```

The kernel mediates. The name service never sees raw endpoint identities; it holds connection capabilities, which it re-delegates. Each delegation step can only reduce rights.

5.6 Service identity and discovery

The kernel uses opaque endpoint identities. A userspace **name service** maps human-readable names to re-delegable connection capabilities:

- Registration — a service registers as `"driver.nic.v1"` with a specific endpoint. The name service stores the endpoint's connection.
- Generation tracking — each registration bumps a generation counter. Stale client connections bound to the previous generation fail with `ServiceRestarted`.
- Access-key gating — a registration may specify a non-zero access key. Lookups without the matching key are denied — policy enforced by a userspace service, not the kernel.
- Lookup — a client asks for `"driver.nic.v1"` and receives a connection capability (or an error). It never receives an address, a port number, or a raw handle.

The name service runs in userspace. The kernel provides the endpoint and connection primitives; userspace provides the naming policy.

5.7 Two kernel data paths, not one

A central distinction is that endpoint IPC is for **cross-domain service calls**. Completion queues are for **kernel and device operations**. They are not the same mechanism.

Endpoint IPC path	Completion queue path
Application → network service	Timer expiry
Network service → NIC driver	DMA completion
Application → filesystem service	IRQ-derived device completion
Service → name service	Waiting for thread/process exit

A userspace driver may compose both: receive an endpoint message (data path), then submit a hardware operation and await a CQ entry (control path). The endpoint message and the hardware completion are related *in the service's logic*, but they are different kernel abstractions with different semantics.

5.8 Why endpoints matter for critical software

1. Communication carries authority — a client that holds a connection capability was authorized (at grant time, by the name service or policy service) to communicate with the target. Every message is authorized by possession of a kernel-mediated capability rather than by an address or UID alone.
2. Scalar and memory messages are distinct — the kernel does not copy arbitrary data between address spaces by default. Small control messages stay small. Bulk data moves as memory objects (Chapter 6) with explicit ownership semantics.
3. Deferred replies enable efficient concurrency — a server is never forced to block waiting for one slow client. It retains reply tokens for in-flight work and continues serving other requests. The shard's executor multiplexes naturally.
4. Restart errors are defined — stale connections return typed errors such as `EndpointClosed` and `ServiceRestarted`. A replacement cannot inherit the old instance's reply tokens.
5. Name service is policy, not mechanism — the kernel provides endpoints and connections. Userspace decides who can register what name, who can look up what service, and what access keys are required. The kernel remains small; policy is replaceable without a kernel rebuild.

6 Memory Objects — Ownership That Crosses Boundaries

A memory object answers: **"Where is the data, and who owns it?"** It lets page-backed data cross protection-domain boundaries with explicit ownership and, for move and borrow modes, without copying the payload.

6.1 The problem with copying

In a conventional OS, moving data between processes typically means copying: the kernel reads from the sender's address space and writes to the receiver's. For large payloads (packet buffers, file blocks, IPC messages), this copy dominates the cost of the operation.

Unrestricted shared memory avoids the copy but permits two processes to modify the same pages. Correctness then depends on synchronization rather than page-table exclusivity.

CharlotteOS adopts a third approach: **ownership transfer through page-table manipulation.** When a memory object moves from one domain to another, the kernel changes the page-table mappings. The sender loses access; the receiver gains it. No bytes are copied. The MMU enforces the ownership.

6.2 First-class memory objects

A memory object is a dedicated kernel object representing page-backed storage with explicit ownership and capability semantics. The kernel manages:

- Physical frames — the backing storage.
- Mappings — which domains have virtual addresses pointing to the pages, and with what permissions.
- Ownership — which domain currently holds the master capability.
- Transfer state — whether the object is idle, in transit, borrowed, or undergoing DMA.
- Lifetime — when the object is freed, all mappings are revoked and all capabilities to it are invalidated.

Userspace never exchanges raw pointers across protection domains. Instead, IPC messages attach **memory-object capabilities** with declared transfer modes.

6.3 Four transfer modes

CharlotteOS defines four transfer modes, enforced by the MMU:

6.3.1 Copy

The receiver gets a byte-for-byte duplicate. The sender retains the original, unmodified.

- **Use when:** the data is small, the sender needs to keep using it, or the receiver is not trusted with the original.
- **Kernel work:** allocate new physical frames, copy the contents, map them into the receiver's address space.
- **Semantics:** simplest; both domains can read their respective copies independently.

The kernel performs the potentially large byte copy without holding the interrupt-masking memory-object or frame-allocator locks. Before releasing the registry lock it acquires a shared-read copy pin on the source object. That pin keeps the frames alive and rejects every tracked operation that could introduce a writer—a writable CPU mapping, in-kernel write, writable or exclusive DMA pin, write lend, move, or ownership rollback—until the snapshot is complete. Read-only mappings and DMA pins may coexist. If the source domain exits during the copy, destruction is deferred until both the copy-pin and DMA-pin counts reach zero; physical frames are freed only after the registry lock is released.

6.3.2 Move

Ownership transfers to the receiver. The sender loses all mappings and all capability rights. The receiver becomes the new owner, responsible for subsequent transfer or destruction.

- **Use when:** the sender is finished with the data and wants to hand it off without copying.
- **Kernel work:** validate sender ownership, prepare receiver mappings, remove sender mappings, perform TLB invalidation, commit ownership, publish the IPC message.
- **Semantics:** physical pages never move. Mappings change. After the move, any attempt by the sender to access the (now unmapped) virtual addresses faults.

6.3.3 BorrowRead

The receiver gets a temporary read-only mapping. The sender retains ownership. The borrow is revoked when the receiver replies (or the call is cancelled, or the server dies).

- **Use when:** the receiver needs to inspect data but must not modify it.
- **Kernel work:** map the pages into the receiver's address space with read-only permissions. Track the borrow so it can be revoked.
- **Semantics:** writable aliases in the receiver are forbidden by the page-table permissions. The receiver cannot write even through unsafe code. Revocation at reply time unmaps the pages.

6.3.4 BorrowWrite

The receiver gets a temporary exclusive writable mapping. The sender's access is revoked for the duration of the borrow. When the receiver replies, sender access is restored.

- **Use when:** the receiver needs to write into a buffer the sender will later read (e.g., a read-into-user-buffer pattern). This is *mutable lending*.
- **Kernel work:** map the pages writable into the receiver, temporarily revoke (unmap or read-protect) sender access, track the borrow for reply-time revocation.
- **Semantics:** the hardware enforces mutual exclusion. The sender cannot access the pages while they are lent out. Unsafe Rust can violate this — the MMU will fault.

6.4 Why hardware enforcement matters

The transfer modes are enforced by page tables, not by Rust's type system:

- A C program running in a protection domain cannot violate the constraints any more than a Rust program can.
- A buffer overrun cannot read pages that are not mapped.
- An unsafe block cannot write to a `BorrowRead` buffer.
- A use-after-drop cannot resurrect a moved memory object.

Rust's ownership complements the kernel enforcement — it catches errors within a domain at compile time. But the kernel is the ultimate guarantor for cross-domain safety, so non-Rust or buggy-Rust processes are still contained by page tables and capability checks.

6.5 Control plane versus data plane

Endpoint IPC belongs to the **control plane**: it configures connections, carries opcodes, sets up device queues.

Memory objects belong to the **data plane**: they carry the actual bytes (packets, blocks, buffers, strings).

For networking specifically:

- Control plane — endpoint IPC configures NIC queues, sets MTU, registers buffer pools.
- Data plane — descriptor rings coordinate producer/consumer state; packet payloads move as memory objects.

This separation avoids turning every packet into a copied IPC payload while preserving strict ownership.

6.6 DMA as a third ownership dimension

When a device performs DMA, a memory object enters a third ownership state:

- CPU ownership — the CPU reads and writes the pages normally.
- DMA ownership — the device is reading or writing the pages; the CPU must not access them.
- Rust/API ownership — which domain holds the capability.

The AArch64 SMMUv3 and x86-64 VT-d/AMD-Vi paths track an explicit DMA pin per mapped memory object. The kernel validates access direction, installs only the object's frames in a device-private translation context, and releases the pin only after a synchronous IOTLB invalidation. Drivers receive IOVAs rather than physical addresses. CPU mappings are not automatically revoked at the raw syscall layer, so coherent descriptor rings remain an explicit driver protocol. Ordinary Rust applications should instead use `catten_rt::owned`: calling `unmap` on `MappedMemory` consumes the CPU mapping, and only the resulting `OwnedMemory` can create a `DmaTransfer`. The memory cannot be mapped or exposed as a Rust slice again until the transfer's `finish` method has synchronously removed the IOMMU mapping.

6.7 Why memory objects matter for critical software

1. Page transfer without payload copying — for aligned, page-backed objects, ownership can move by changing mappings rather than copying bytes. Whether this is faster depends on payload size and mapping/TLB costs.
2. Cross-domain stale access is constrained by mappings — when a memory object is moved, the sender's mappings are removed. Any later access through those mappings faults. When a borrow is revoked, the borrower's mappings are removed. This does not rule out bugs in unsafe code, page-table maintenance, or DMA.
3. Borrows have defined lifetimes — a borrow is always tied to an IPC call. When the call completes (reply, cancellation, timeout, or server death), the borrow ends. There is no "I forgot to return it."
4. DMA ownership is tracked on the reference platforms — AArch64 SMMUv3 and x86-64 VT-d/AMD-Vi pin mapped memory and restrict each requester to its DMA domain. The ownership-aware Rust API prevents ordinary CPU mappings from overlapping a transfer; raw coherent-ring implementations remain responsible for volatile/atomic access and the device protocol.
5. Hardware enforcement, not language enforcement — the page tables are the ultimate arbiter. No amount of unsafe code, C FFI, or compiler bug should access CPU pages that are not mapped, assuming correct MMU setup and TLB maintenance. The implemented SMMUv3, VT-d, and AMD-Vi paths provide the corresponding DMA boundary for explicitly mapped objects.

7 Completion Queues and Non-Lossy Results

A completion queue (CQ) carries asynchronous kernel-operation results to userspace. Its central safety rule is that a full userspace ring must not silently discard a terminal result.

7.1 When to use a completion queue

Use submission/completion when the operation belongs to the **kernel or a hardware-facing mechanism**, not to another userspace service:

- Timer expiry.
- DMA completion (disk read finished, packet transmitted).
- IRQ-derived device completion (data available on UART).
- Asynchronous page operations.
- Waiting for thread or process exit.
- Kernel-managed asynchronous object work.

Use endpoint IPC (Chapter 5) when one protection domain invokes another. Confusing these two paths leads to architectural mistakes: a service call is not a kernel operation, and a reply token is not a completion record.

7.2 The CQ ring — a shared-memory completion buffer

A completion queue is a **shared-memory ring buffer** between the kernel (producer) and userspace (consumer). A single 4 KiB page contains a header and a circular array of fixed-size entries.

Each entry is 32 bytes:

```
struct CompletionEntry {
    operation: u64,    // operation identifier (e.g., timer id)
    cookie: u64,       // opaque value set by the submitter
    status: u32,       // completion status code
    flags: u32,        // per-entry flags
    result: i64,       // operation result (e.g., bytes read)
}
```

The ring page is mapped read-write in userspace because the consumer advances its tail; the kernel accesses the same frame through its privileged mapping. The ABI assigns each side its own fields:

- Userspace drains entries without syscalls — advancing the consumer tail is a store to a shared page; reading entries is a load. No kernel crossing needed for the common fast path.
- The kernel publishes entries with a release fence — the consumer sees coherent data without additional synchronization.
- Overflow is detected, not hidden — an overflow counter in the header lets userspace detect when entries were produced faster than consumed.

7.3 The non-lossy invariant

> **A terminal operation result must never be silently lost because the userspace CQ ring is full.**

This is an architectural invariant, so when the ring is full, the kernel has several options:

- Retain overflow in a per-domain kernel backlog — entries that cannot fit in the ring wait in a kernel-side buffer. When userspace drains the ring, the backlog drains into it.
- Stop accepting new submissions — the kernel may return `WouldBlock` on `submit`, applying back-pressure at the submission point rather than at the completion point.
- Propagate `WouldBlock` — the CQ's wait operation can indicate that the ring is full and must be drained.

The overflow counter is pressure telemetry: it tells the consumer that it is falling behind.

In a critical system, a lost completion for a disk write or a timer expiry is a correctness bug with potentially severe consequences. CharlotteOS treats each completion as a datum that must be delivered at least once.

7.4 Per-shard CQ partitioning

Each address space owns a set of completion queues, keyed by `CqId`. Queue 0 is the default. For a service with multiple sitas shards, **per-shard CQ rings** enable targeted wakes:

- Shard A's executor waits on CQ 0.
- Shard B's executor waits on CQ 1.
- A completion destined for Shard A is posted to CQ 0.
- Only Shard A's executor is woken; Shard B is undisturbed.

This prevents cross-shard completion stealing (a pathological behavior where one busy shard drains entries intended for another) and keeps wakeup latency proportional to the number of entries, not the number of shards.

7.5 Operation lifecycle

Every asynchronous operation transitions through a defined state machine:

```
Created
    -> Submitted
    -> Accepted | Rejected
    -> InFlight
    -> Completed | Failed | Cancelled
    -> ResultObserved
    -> ResourcesReclaimed
```

Each transition is explicit and observable:

Created The operation descriptor exists in userspace but has not been submitted to the kernel.
Submitted `submit()` has been called. The kernel has the operation record but has not yet validated it.
Accepted / Rejected The kernel has validated the operation. It may be rejected immediately (invalid parameters, insufficient rights, queue full) with a synchronous error.
InFlight The operation is being processed. The kernel or device owns any associated buffers.
Completed / Failed / Cancelled A terminal state. The result is posted to the CQ. Buffer ownership returns to userspace.
ResultObserved The consumer has read the CQ entry.
ResourcesReclaimed The operation's capability-table slot has been freed (via `close`).

7.6 Waiting on a CQ — `CQ_WAIT`

When a shard's executor has no ready tasks, it calls `CQ_WAIT` on its assigned queue. The kernel blocks the calling thread until one of three things happens:

1. A completion entry is posted to the CQ.
2. An explicit `CQ_WAKE` arrives (from another shard or the kernel).
3. A deadline elapses (timeout variant: `CQ_WAIT_TIMEOUT`).

The wait is race-free. The sequence is:

1. Inspect the CQ ring — if completions are pending, return immediately.
2. Register a waiter against the CQ's generation counter.
3. Re-inspect the CQ ring — if completions arrived during registration, unregister and return.
4. Block the thread — it will be woken when the generation counter changes (a new entry is posted or a `CQ_WAKE` bumps it).

This is the **single wait point** for the entire shard. Three event sources converge on it:

- Kernel completions (CQ entries from `submit` operations).
- Endpoint readiness (a coalesced wake when messages arrive on a bound endpoint).
- Device interrupts (a coalesced wake from a bound interrupt object).

The shard does not need separate wait operations for timers, IPC messages, and device events. One aggregated wait covers all of them.

7.7 LP affinity for CQ waits

A thread that blocks in `CQ_WAIT` is assigned an affinity LP. When woken, the scheduler returns it to the same LP rather than scanning for the least-loaded one. This has three benefits:

1. Timer events stay on the same LP's queue, eliminating cross-LP migration races (observer-not-found when a completion is posted on a different LP than the one where the waiter was registered).
2. Cache warmth is preserved — the waking thread finds its data in the LP's local caches.
3. Shard-to-LP affinity is maintained — the Sitas shard that blocked on CQ 0 on LP 2 wakes up on LP 2, where its shard-local state lives.

7.8 Operation IDs versus per-operation capabilities

An earlier experimental design returned a `CompletionCap` for every submission. The final ABI uses **operation IDs** for the common path:

- `submit()` returns an `OperationId` (a lightweight identifier).
- The result arrives in the CQ with the same operation ID.
- No capability-table slot is consumed for common operations.
- A first-class operation capability is created only when independent waiting, cancellation, delegation, inspection, or policy requires it.

This avoids capability-table pressure for high-rate operations (timer events, network packet completions) while preserving the capability model for operations that need first-class authority.

7.9 Cancellation and buffer ownership

When a userspace task drops a completion handle or explicitly cancels an in-flight operation:

1. The kernel marks the operation as `Cancelling` but **retains any transferred buffer.** The buffer may still be the target of DMA or kernel access.
2. A terminal `Cancelled` completion is eventually posted to the CQ, returning buffer ownership to userspace.
3. On domain teardown (process exit), outstanding buffers may be leaked rather than freed while the kernel or a device may still touch them. Reclamation goes through explicit operation and device teardown.

Under this contract, a buffer is retained while the kernel or device may still reach it. The safe Rust wrapper consumes the owned value on submission and returns it on terminal completion.

7.10 Why completion queues matter for critical software

1. Non-lossy completions are a correctness requirement — a disk-write confirmation, a timer expiry, or a sensor reading is a fact and losing it is a bug. The backlog retains terminal results when the ring is full.
2. The single aggregated wait eliminates a class of race conditions — a shard that must wait on "timer or IPC message or device interrupt" in a conventional OS must manage three separate wait objects, with potential lost-wake races between checking one and arming another. One CQ means one atomic check-and-arm sequence.
3. Per-shard CQ rings enable predictable latency — no shard wakes another shard's executor to deliver a completion. Wakes are targeted, bounded, and proportional to the actual work.
4. The CQ ring is zero-syscall in the common case — the fast path — drain entries, process results, loop — requires no kernel transitions. Only when the ring is empty and the shard must block does a syscall occur.
5. Cancellation is defined, not best-effort — every cancellation path ends in a terminal completion. There is no "the buffer might or might not be freed" ambiguity. The ownership contract is explicit and testable.

8 Shards and Logical Processors

CharlotteOS separates the hardware execution context (logical processor, or LP) from the userspace ownership domain (shard).

8.1 The logical processor (LP) — the hardware unit of scheduling

An LP is CharlotteOS's representation of a schedulable hardware execution context. In most configurations, one LP equals one CPU core (or one hyperthread). The kernel shards its own state per-LP, meaning each LP has:

- Its own run queue of ready threads.
- Its own timer queue and quantum timer.
- Its own scheduler instance (e.g., `RoundRobin`).
- Its own IPI command queues for receiving work from other LPs.
- Per-LP data structures accessible through `PerLp<T>`.

An LP is a **place**: threads are pinned to an LP at creation and, by default, stay there. Migration is opt-in and restricted.

8.2 The shard — the userspace unit of ownership

A shard is a userspace ownership and execution domain. The rules of the shard are the Sitas discipline:

1. Only the owning shard mutates shard-local state — there is no lock on shard-local data because there is no concurrent access. The executor runs one task at a time on the shard.
2. Cross-shard interaction goes through bounded typed messages — a value that moves from shard A to shard B is sent through a typed channel (`ShardSender<M>`). The sender loses the value; the receiver gains it.
3. Placement is explicit — code that needs to be shard-local is accessed through `ShardLocal<T>`, which verifies at runtime that the calling thread belongs to the owning LP.
4. Observability returns owned snapshots — diagnostics and introspection capture state by value. No borrowed reference into shard internals escapes the shard.

8.3 The usual mapping: one shard ↔ one LP

In the typical configuration:

```
one sitas shard
    <-> one LP-affine userspace thread
    <-> normally one logical CPU
```

The userspace thread and its executor are pinned to one LP. ShardLocal<T> state is valid only while that LP's owning thread runs.

The architecture retains the distinction (shard $\neq$ LP) so that more complex configurations remain possible:

- Multiple sharded processes — one LP might host shards from multiple protection domains, time-sliced by the kernel scheduler.
- CPU hotplug — an LP can be taken offline; its shards can be migrated (when migration-safe) or torn down.
- Oversubscription — a test can run N shards on $M < N$ LPs to exercise migration and scheduling paths.

8.4 PerLp<T> — interrupt-safe shared data

For kernel data that must be accessible from multiple LPs (including from interrupt context), CharlotteOS uses PerLp<T>: a boxed slice of per-LP cells, each wrapped in a lock. Access is:

```
per_lp_data.with(|lp_data| { lp_data.do_something(); });
```

The with closure acquires the lock for the current LP, calls the closure, and releases the lock. This is interrupt-safe (the lock masks interrupts during the critical section on architectures where that is necessary).

Use PerLp<T> for data that is:

- Accessed from interrupt handlers (timers, device IRQs).
- Modified by an IPI from another LP.
- Part of the scheduler, memory allocator, or other kernel subsystem that must be LP-safe.

8.5 ShardLocal<T> — lock-free single-owner data

For kernel data that is accessed only from one LP and never from interrupt context, CharlotteOS provides ShardLocal<T>: a lock-free per-LP container using UnsafeCell<T> with runtime owner-check and borrow-flag guard. Access is:

```
shard_local.with(|val| { *val = 42; });
```

The with closure:

1. Checks that the calling thread belongs to the owning LP (panics if not).
2. Checks that the borrow flag is clear (panics on re-entrant access).
3. Sets the borrow flag.
4. Calls the closure with &mut T.
5. Clears the borrow flag.

The closure must not:

- Store the `&mut T` reference anywhere (it must not escape).
- Send it to another thread or LP.
- Cross an `.await` point.
- Panic or unwind (the borrow flag would remain set, deadlocking future access).

8.6 Why the distinction matters for critical software

1. Single-owner mutation — the intended shard path does not access shard-local state concurrently. Runtime owner and borrow checks complement Rust's type rules.
2. Placement is deliberate — sending work to a specific LP is an explicit operation (`try_run_on_lp`, `ShardSender::try_send`) and it is not at the mercy of a global work-stealing scheduler. This makes it possible to reason about locality, cache behavior, and latency.
3. The two container types force a design choice — when you reach for shared data, you must decide: is this accessed from interrupt context? Use `PerLp<T>` with locking. Is it thread-only? Use `ShardLocal<T>` without locking. The choice is explicit.
4. Cross-shard communication is typed and bounded — A `ShardSender<M>` carries a specific Rust type (not `Box<dyn Any>`, not a byte buffer). The receiver knows what it will get. The channel has fixed capacity; senders handle backpressure. There is no unbounded message queue that can exhaust memory.

9 The Sitas Executor

The sitas executor drives futures on one shard and parks on the shard's completion queue when no task is ready.

9.1 One executor per shard

Each sitas shard has:

- One executor — a cooperative scheduler that polls ready futures within a budget.
- One ready queue — tasks whose futures returned `Poll::Pending` and whose wakers have since been invoked.
- One timer wheel — futures awaiting deadlines.
- One CQ partition — the completion queue on which this shard waits (`CQ_WAIT`).
- Shard-owned service state — mutable data that only this executor touches.

The executor runs on one LP-affine thread. When that thread is scheduled onto its LP by the kernel, the executor runs. When the thread blocks (no ready tasks), the LP is free to run another domain's thread.

9.2 The polling loop

The executor's main loop is:

```
loop {
    // 1. Poll ready tasks within a budget.
    while budget_remaining > 0 && ready_tasks.not_empty() {
        let task = ready_tasks.pop();
        match task.future.poll(task.waker) {
            Poll::Ready(output) => { /* task done */ }
            Poll::Pending => { /* will be re-awoken by waker */ }
        }
        budget_remaining -= 1;
    }

    // 2. If tasks remain but budget exhausted, re-queue and yield.
    if ready_tasks.not_empty() {
        ready_tasks.requeue_for_next_tick();
    }

    // 3. No ready tasks? Block in the unified wait.
    if ready_tasks.is_empty() && timer_wheel.is_empty() {
        reactor.wait_for_events(); // -> CQ_WAIT
    }
}
```

The budget prevents one task (or a tight loop of always-ready tasks) from starving other tasks on the shard. If the budget is exhausted with work remaining, the executor re-queues the remaining tasks and yields (either to other threads on the same LP via the kernel scheduler, or to idle).

9.3 The unified wait — three event sources, one wait point

When the executor has no ready tasks and no pending timers, it calls `CQ_WAIT` on the shard's completion queue. This is the **only** blocking point in the shard's execution.

Three event sources converge on this one wait point:

9.3.1 Kernel completions (CQ entries)

When a submitted operation completes (timer expired, DMA finished, device interrupt processed), the kernel posts a `CompletionEntry` to the shard's CQ ring and wakes the blocked thread. The executor drains the ring:

```
fn drain_cq(&mut self) {
    while let Some(entry) = self.cq_ring.read() {
        let waker = self.waker_registry.take(entry.user_data);
        if let Some(waker) = waker {
            waker.wake(); // task becomes ready
        }
    }
}
```

9.3.2 Endpoint readiness (coalesced wake)

An endpoint can be bound to a CQ. When a message arrives on the endpoint, the kernel posts a coalesced wake to the bound CQ. The executor, on waking, calls the endpoint's receive path to drain the pending messages.

Coalescing means: if 10 messages arrive before the shard wakes, the kernel delivers one wake, not 10. The executor drains all 10 messages in one batch.

9.3.3 Device interrupts (coalesced wake)

A device interrupt object can be bound to a CQ. When the interrupt fires, the kernel posts a coalesced wake. The executor, on waking, calls the device's interrupt handler (or the driver's polling function).

9.4 Task wakeup — from CQ entry to future poll

When the executor drains a CQ entry, it looks up the registered waker for that entry's `user_data` cookie. The cookie was set by the task when it submitted the operation. The waker marks the task ready:

```
// Inside a task's Future::poll:
fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<u64> {
    match self.cap.poll() {
        Ok(Some(completed)) => Poll::Ready(completed.result),
        Ok(None) => {
            // Register waker so we are notified when this completes.
            self.cap.register_waker(cx.waker().clone());
            Poll::Pending
        }
        Err(e) => Poll::Ready(/* error */),
    }
}
```

The waker is a `ShardWaker` — when invoked, it sets the task's ready flag and signals the reactor (if the reactor is currently blocked) to re-enter the polling loop. The reactor then polls the task, which finds its completion ready.

9.5 Cross-shard messaging without the kernel

Within one protection domain, tasks on different shards communicate through sitas typed mailboxes — not through kernel endpoint IPC:

```
// Shard A: send work to Shard B.
let sender: ShardSender<Command> = shard_b.mailbox();
sender.try_send(Command::Process { data: owned_buffer })?;

// Shard B: receive work.
let receiver: ShardReceiver<Command> = shard_b.receiver();
loop {
    let cmd = receiver.recv().await?;
    match cmd {
        Command::Process { data } => { /* handle */ }
    }
}
```

The `ShardSender/ShardReceiver` pair is a bounded, typed, MPSC channel within userspace. Sending does not cross the kernel boundary. The receiving task registers its waker on the channel; when a message arrives, the waker fires and the task re-enters the executor's polling loop.

This is **internal** cross-shard communication (same protection domain, different shards). External communication (different protection domains) uses endpoint IPC (Chapter 5).

9.6 The ShardParker — blocking on native state

When the executor has no ready tasks, it must block its OS thread. The mechanism is the `ShardParker` trait:

```
trait ShardParker {
    fn park(&self);     // Block the current thread.
    fn unpark(&self);  // Wake the blocked thread (from another shard).
}
```

On CharlotteOS, the implementation is:

```
fn park(&self) { syscall::cq_wait(self.cq_id); }
fn unpark(&self) { syscall::cq_wake(self.cq_id, self.target_lp); }
```

The parker maps directly to `CQ_WAIT` and `CQ_WAKE`. There is no intermediate thread pool, condition variable, or busy-loop. The thread parks on native completion and message state.

This direct mapping is the basis of the "no retrofit tax" claim. The runtime does not need a thread pool to translate blocking calls into futures because `CQ_WAIT` covers the shard's asynchronous sources.

9.7 Why the sitas executor matters for critical software

1. Cooperative scheduling means no data races on shard-local state — within a shard, only one task runs at a time. The executor polls tasks sequentially. There is no preemption within the shard. This means shard-local mutable state never needs locks.
2. The single wait point eliminates a class of race conditions — the executor does not wait on "timer or CQ or endpoint" through three separate blocking calls with three separate check-and-arm sequences. One CQ, one wait, one drain loop. The kernel guarantees the wake is not lost between checking and arming.
3. No thread-pool overhead — the executor uses exactly one kernel thread per shard. When that thread blocks, it blocks on the CQ — the same object that will deliver its wake. There is no need for a pool of threads to convert blocking syscalls into futures.
4. Budgeting prevents starvation — the polling budget ensures that a shard with many ready tasks makes progress on all of them, not just the one that finishes fastest. This prevents priority-inversion-like behavior within a shard.
5. Internal messaging avoids unnecessary kernel crossings — shards within the same process communicate through typed channels in userspace. Only cross-process IPC crosses the kernel boundary. This keeps the fast path fast and the trusted computing base small.

9.8 A worked example: the mailbox index build

The mailbox index build is an end-to-end sitas demo at EL0. It is the `no_std` counterpart of the host-side `sharded_index_mailbox` example: scanners route generated records to logical assembler work units through typed mailboxes; assemblers sort the records; and a coordinator merges and verifies the runs.

The demo is implemented by `sitas_core::mailbox_index` and runs inside `catten-user`, the EL0 sitas image that the kernel's `el0_sitas` self-test loads and verifies.

9.8.1 The workload

The demo uses 1024 synthetically generated records over two shard threads and three logical assembler work units. A record's key is a pure function of its index (a SplitMix64 avalanche over `seed` XOR `index`), so any thread can re-derive any key without storing the data set. An index entry is a `(key, offset)` pair.

- Scanners — two producer threads, one per shard. Each generates its contiguous partition of records, routes every key to an assembler with a key hash, batches the entries, and sends the batches as owned messages.
- Assemblers — three logical work units. Two are placed on shards 0 and 1; the third is deliberately placed on shard 0 so that logical naming and physical placement are visibly separate. Each assembler receives its entries, sorts them into a run, and hands the completed run to the coordinator as an owned value.
- Coordinator — the main thread. It spawns the shard threads, waits for all three runs, k-way merges them, verifies the final index, and writes the verified record count to the status page.

9.8.2 Kernel and userspace responsibilities

The demo separates kernel and userspace responsibilities.

The kernel supplies:

- An address space with mapped user pages (code, heap, CQ ring, status page) and nothing else: no file, network, or mailbox syscalls are used anywhere in the demo.
- `spawn_thread` via SVC, which pins each shard thread to its LP.
- The completion queue: a ring mapped read/write into user memory, with `CQ_WAIT`/`CQ_WAKE` as the only blocking primitive.

Userspace (the sitas `no_std` lane) supplies:

- One `ShardExecutor` per shard, polling its futures and blocking in the unified wait (Section 9.3).
- The typed mailboxes (`ShardSender`/`ShardReceiver`) that carry work between shards without ever crossing the kernel boundary (Section 9.5).
- All of the logic: routing, batching, backpressure, sorting, the merge, and the verification.

The kernel never sees a message, key, or run. It sees threads being spawned and blocking on the completion queue. The division is explicit: the kernel provides authority isolation, thread placement, and wakeups; userspace owns the workload state and message discipline.

9.8.3 The mailbox path

The typed channel carries `IndexMessage`:

```
enum IndexMessage {
    Entries { from: ShardId, batch: Vec<IndexEntry> },
    ProducerDone { from: ShardId },
}
```

Senders are cloned into every scanner; each receiver moves into its assembler shard, and assemblers are started before scanners so receivers exist before producers begin. The `no_std` channel exposes a non-blocking `try_send`; a producer whose mailbox is full parks briefly on the runtime's `ShardParker` and retries, rather than spinning. The assembler task awaits `receiver.recv()`; the await parks the shard in its reactor wait, and the sending side's channel waker re-queues exactly that task and releases the reactor — the same channel-waker-to-reactor chain as the sharded KV service (Section 9.5).

9.8.4 Coordination and joining

The demo coordinates like the sharded KV requester: the coordinator parks and polls the result mailbox until all three runs arrive, then verifies. A parked coordinator consumes no CPU, and a stolen or coalesced wake costs at most one park interval.

Joining shard threads is also real on CharlotteOS now. A shard thread terminates itself with `thread_exit` once its closure completes (a user thread has no "returned" trap, so the trampoline must ask the kernel to reap it), and the kernel's `ObserveThreadExit` syscall registers a completion capability as an exit-observer of that thread: when the kernel reaps the thread, the capability fires through the ordinary completion ABI. The backend's `CharlotteJoinHandle` wraps this — `join` blocks in the kernel's completion wait and then reads the closure's return value from a per-spawn result slot, and `is_finished` polls the capability. The `catten-user` join probe exercises it: it spawns two shard threads that return 40 and 41, joins both, and writes the sum (81) to the status page. The coordinator of the mailbox demo itself does not join because work completion (all runs arrived) already implies the shards are done — but a caller that needs "the thread really exited" has the kernel-backed primitive.

9.8.5 Verification

Verification checks the product, not the implementation path:

1. Every run is internally sorted.
2. A k-way merge over the runs is globally sorted and contains all 1024 records.
3. Every entry's key equals the deterministic key of its offset, and the offsets form a complete bijection of the record range (checked with an XOR invariant).
4. Exactly one run arrives per work unit.

On success the demo writes 1024 (the number of records) to status-page offset 1 and the join probe writes 81 to offset 2, next to the `basic_kv` result at offset 0. The kernel's deferred verifier polls all three words and reports:

```
[sitas] SUCCESS: basic_kv total_len=3,
mailbox index verified 1024 records,
join probe sum 81
```

A failed demo writes the failure sentinel `0xBADC0DE` instead, which the verifier turns into a failed self-test.

9.8.6 Why this demo matters

The mailbox index build shows the architecture working as designed: mutable state is owned, work is transferred as owned values, shards communicate without locks or kernel crossings, and the kernel stays

a thin substrate. Because the demo is written against the `no_std` `ShardRuntime`, the identical code path runs on a Unix host through `sitas-unix` with a plain `cargo test` and on CharlotteOS through the kernel syscall backend: the runtime is not rewritten or emulated for either side. That is the Rust-affine version of the old division of labour: the kernel contributes the primitives that only it can provide, and the language contributes the discipline that userspace is trusted to enforce.

10 Scheduling and Backpressure

The kernel schedules threads through a block–yield–wake cycle. Bounded ingress propagates pressure, although not every internal queue has a hard memory bound.

10.1 The kernel scheduler

CharlotteOS uses a pluggable scheduler architecture. The system scheduler (`SYSTEM_SCHEDULER`) delegates to per-LP `LpScheduler` instances. The reference implementation is `RoundRobin`:

- A FIFO run queue per LP.
- A per-LP quantum timer (10 ms by default) that drives preemption.
- Operations: `spawn_thread`, `block_thread`, `submit_ready_thread`, `yield_lp`, `abort`.

The scheduler is pluggable: a different scheduling policy (EDF, CFS, fixed priority) can be provided by implementing the `LpScheduler` trait. The rest of the kernel interacts with the scheduler only through the abstract interface.

10.2 The block-yield-wake cycle

This cycle underlies asynchronous operations. Every async primitive in CharlotteOS — `sleep()`, `wait()`, `CQ_WAIT` — is implemented through this cycle.

10.2.1 Block

A thread blocks by calling `block_thread(tid, &observable)`:

1. The kernel marks the thread `Blocked` in the scheduler.
2. The thread's `Waker` is registered as an observer on the observable event (a completion, a timer, an endpoint, or a CQ).
3. The thread is removed from the LP's run queue.
4. `yield_lp()` saves the thread's register state and stack pointer to its `ThreadContext`.
5. The scheduler selects the next ready thread (if any) and restores its context. The blocked thread is suspended at the point of the yield.

For a timer sleep, steps 1–2 include creation of the wake source itself. Publishing `Blocked` and inserting the timer into the local queue must therefore be one local non-preemptible transaction. `sleep()` masks local interrupts across both operations; otherwise a scheduler-quantum interrupt can switch out the newly blocked thread before any event exists to wake it. The previous interrupt state is restored before `yield_lp()`, and no lock is held across the yield.

10.2.2 Wake

An event fires (timer expired, completion posted, message arrived):

1. The event's observable fires, calling `Waker::notify(tid)`.
2. `notify` calls `submit_ready_thread(tid)`, which marks the thread `Ready` and adds it to its affinity LP's run queue.
3. If the target LP is idle, a reschedule IPI is sent to wake it. If the target LP is already running, no IPI is needed — the thread will be picked up at the next scheduling point (quantum expiry or voluntary yield).

10.2.3 Resume

At the next scheduling opportunity on the target LP:

1. The scheduler selects the woken thread from the run queue.
2. The thread's saved `ThreadContext` (registers, stack pointer) is restored.
3. Execution resumes immediately after the `yield_lp()` call that originally blocked the thread.
4. From the thread's perspective, `block_thread` returned normally and the event has fired.

10.3 Wake coalescing

Wakes are **idempotent**: calling `submit_ready_thread` on an already-`Ready` or `Running` thread is a no-op. This enables coalescing:

- If 10 messages arrive on an endpoint before the shard wakes, the kernel posts one wake, not 10.
- If 3 timer events fire before the LP processes the timer queue, the kernel delivers them in one batch during the next `process_events`.
- The unified shard wait (Chapter 9) aggregates multiple wake sources onto a single thread. Re-waking that thread is harmless.

The rule is: **enqueue before signaling.** The work is placed in a queue (CQ ring, endpoint queue, timer queue) first; then a wake is delivered. The woken thread finds the work when it drains the queue.

10.4 The quantum timer

Each LP's `RoundRobin` scheduler arms a periodic quantum timer (10 ms default). When it fires:

1. The timer IRQ handler sets a context-switch-pending flag.
2. At the next `yield_lp()` (or at the IRQ handler's tail), the scheduler checks the flag.
3. If set and another thread is runnable, the current thread's context is saved and the next thread's is restored.

An `armed` flag ensures exactly one quantum event is in flight at any time, preventing unbounded timer-queue growth from manual yields.

10.5 The deferred reaper

A thread cannot free its own kernel stack (it is executing on it). When a thread exits:

1. `abort()` moves the dying thread from the thread table to a `DEAD_THREADS` list. The thread is extracted from the table without being dropped.
2. The scheduler performs a context switch to the next thread.
3. **After** the switch (on the new thread's stack), `reap_dead_threads()` drops the dead threads, freeing their stacks and other resources.

Deferred reclamation is required because threads own their stacks.

10.6 Load rebalancing

By default, threads are pinned to their affinity LP. Migration is opt-in:

- A thread must be marked `migration_safe`.
- Migration is rejected while the thread holds timer events, endpoint waiters, CQ registrations, or other LP-affine ownership constraints.
- Rebalancing uses a **sustained-imbalance window**: per-LP load is sampled over a runtime-adjustable interval rather than reacting to a single instantaneous queue-depth observation.

This conservative approach preserves cache warmth and eliminates migration races (observer-not-found when a completion is posted on a different LP than the one where the waiter was registered). Stable affinity is the correctness default; migration is an opt-in optimization.

10.6.1 Generation-safe retirement

A numeric thread identifier is a recyclable table slot, not a permanent identity. Every queued scheduler handle, waker, deferred verifier callback, and asynchronous retirement request therefore carries the thread generation. `abort_thread_generation` and the LP scheduler's removal operation validate both values before changing state. A delayed cleanup request for an exited thread consequently cannot abort a newer thread that reused the same numeric identifier. The scheduler-lifecycle boot test exercises stale wake rejection and remote owner-LP retirement.

10.7 Synchronization and lock ordering

Shared kernel state is synchronized by a small, deliberate set of primitives. Their full semantics and the cross-subsystem ordering rules are maintained in `docs/reference/locking.md`, which is authoritative over this overview.

10.7.1 Lock families

- **Interrupt-masking spin `Mutex`** guards the frame allocator, memory-object registry, address-space table, kernel address space, domain-authority table, address-space lifecycle, scratch-window cursor,

and the global allocator. It attempts acquisition with maskable IRQs disabled and keeps them disabled while owned. A failed attempt restores the caller's interrupt state before spinning, then retries with IRQs masked. The ownership masking exists to prevent a preemption deadlock: these locks are taken from both preemptible kernel threads and synchronous EL0 exception paths, so a timer-preempted owner could otherwise be starved forever.

- **Interrupt-masking spin `RwLock`** is the workhorse. It protects the master thread table, the deferred-dead table, the system scheduler, the IPC registry, the completion registry, the userspace mailbox table, and each per-LP slot. Failed reader and writer attempts likewise restore `INT_STATE` while contending. A `waiting_writers` counter gives writers preference only during the masked atomic acquisition attempt; retaining that preference while waiting could deadlock a same-LP interrupt reader.
- **The external `spin` crate** is used only by the stack allocator, which wraps its arena mutex in explicit `mask_interrupts!`/`unmask_interrupts!` calls. Its guard-page map is touched only from thread context, never from IRQ context.
- **Lock-free structures** cover the rest: `ShardLocal` and `PerLp` for per-LP data, `ConcurrentQueue` for the IRQ-to-thread deferred-wake handoff, and plain atomics for the `on_cpu` ownership handshake and generation counters.

A scheduler-blocking lock family exists under `cpu/scheduler/sync` that parks a contended caller via `block_thread` instead of spinning. It currently has no callers and does not participate in any ordering guarantee.

10.7.2 Interrupt-masking discipline

Both spin locks mask IRQs while owned, but leave IRQs in the caller's original state while waiting for another LP. This is required for synchronous TLB-shootdown rendezvous: a contender must remain able to acknowledge the lock owner's IPI. Only the `RwLock` is nesting-aware. It routes through a per-LP `INT_STATE` record that counts save depth and re-enables interrupts exactly once on the outermost restore. This lets a thread take several different `RwLock`s, or a read after a read, without prematurely re-enabling IRQs. The `Mutex` records its pre-acquire interrupt bit on the lock object itself. Both guard types are non-sendable; acquiring different mutexes still nests correctly, but re-acquiring the same lock on one LP deadlocks.

10.7.3 Lock ordering

The scheduler's chain is fixed:

```
SYSTEM_SCHEDULER.read() -> lp_scheduler.lock() -> MASTER_THREAD_TABLE.write()
```

Cross-subsystem rules hold today:

- **Never take address-space or frame-allocator locks while the heap lock is held.** The heap's growth reserve is pre-mapped at boot so allocation can extend within mapped memory without those locks.
- **Never hold the memory-object registry across the address-space table lock.** Teardown takes table-then-frame-allocator while allocation takes allocator-then-registry; registry-then-table would close an AB-BC-CA cycle.
- **Dropping the memory-object registry for a bulk copy requires a shared-read copy pin.** While such a pin exists, writable CPU mappings, in-kernel writes, writable or exclusive DMA pins, write

lends, moves, and ownership rollback are rejected. Owner teardown is deferred until both the DMA-pin and copy-pin counts reach zero. The final release removes the object under the registry lock and frees its frames only after dropping that lock.

- **Never block or yield while holding any lock.** A spin lock also masks IRQs, so parking the thread would abandon the lock and the LP could not schedule its successor. All guards are dropped before `switch_ctx`.
- **IRQ context takes no locks and never blocks.** Interrupt delivery uses atomics and MMIO only; a deferred wake crosses into thread context through a lock-free queue.

10.8 Backpressure and queue bounds

Backpressure is a design principle. Endpoint and hardware rings are bounded, but not every internal queue currently has a hard memory bound. In particular, the non-lossy CQ overflow backlog may grow until its consumer drains it.

Layer	Bounded object	Backpressure signal
Application request	Service endpoint queue	`QueueFull`
Service shard mailbox	`ShardSender<M>`	`Err(m)` returned
Driver endpoint/ring	NIC descriptor ring	Descriptor exhaustion
Kernel submission	Capability table	`WouldBlock`
Hardware	DMA descriptor ring	Ring full

Each layer has an explicit backpressure policy:

```
enum SendPolicy {
    Try,               // Fail immediately if full.
    Wait,              // Wait for capacity.
    Deadline(Instant), // Wait until a deadline.
    // Merge a pending "data available" notification.
    DropIfCoalescible,
}
```

Rules:

- Terminal completions are never dropped — the CQ backlog guarantees this (Chapter 7).
- Coalescible notifications may be merged — two “data available” signals before the consumer wakes become one.
- Control messages normally fail or wait explicitly — a service registration request that cannot be queued fails with `QueueFull`.
- Data-plane producers should stop before unbounded allocation — a NIC driver should stop accepting TX packets when its descriptor ring is full.
- Emergency paths require reserved capacity — the bounded IPI queue reserves slots for mandatory kernel messages (TLB shootdowns, scheduler commands), ensuring they cannot be starved by user work.

10.9 Why scheduling and backpressure matter for critical software

1. The block-yield-wake cycle is exercised across subsystems — the same mechanism handles sleep, IPC wait, CQ wait, and timer wait. A bug fixed in one path fixes all paths. The cycle is exercised by the async demo and by self-tests that call `wait()`; this is test evidence, not a formal proof.
2. Idempotent wakes eliminate lost-wake races — re-waking an already-ready thread is a no-op. This means the kernel can be aggressive about delivering wakes without worrying about double-delivery.
3. Bounded ingress limits some exhaustion paths — endpoint and descriptor-ring capacities reject excess work with backpressure. Resource quotas and a hard bound for all completion backlog memory are not yet implemented, so this is not a general denial-of-service guarantee.
4. Stable LP affinity prevents migration races — completion delivery is LP-local. A waker registered on LP 2 is signaled on LP 2. There is no window where a completion is posted on LP 1 while the thread is migrating to LP 2 and the observer is not found.
5. The scheduling interface is pluggable — `RoundRobin` is the implemented policy. Other policies would require their own implementation and validation behind the same interface.

10.10 Scheduler observability

The prototype records integer running statistics for each thread at scheduler context switches. A snapshot includes the thread identity and generation, address-space owner, scheduler state, LP affinity, dispatch count, and the count, minimum, maximum, total, and sum of squares of completed on-CPU slices. The active slice start tick is reported separately. The counter frequency in the snapshot header permits conversion from ticks to time.

The accumulator uses integer arithmetic and reports saturation. Mean and sample variance are exported as exact rational components; standard deviation and coefficient of variation are calculated in userspace. This avoids using floating-point/SIMD state in the kernel. Accumulators are mergeable so future hot-path metrics can be gathered per LP rather than through a global lock.

The `THREAD_STATISTICS` syscall is address-space-scoped by default: callers see only their own threads and receive the snapshot as a memory-object capability. At boot the supervisor starts exactly one `observe` EL0 service and delegates a typed system-observer capability through its launch vector. Presenting that capability authorizes a machine-wide snapshot; a fabricated or wrong-type handle fails closed. The service publishes snapshots through endpoint IPC and registers itself with the node name service. Machine-wide observation is therefore explicit authority rather than an ambient privilege.

An HTTP `GET /metrics` adapter is a reasonable userspace consumer of this IPC protocol, but it is not implemented yet. The present TCP/IP service does not supply the complete server-side bind/listen/accept path needed to host it. See `docs/reference/observability.md` for the wire format, security boundary, and suggested next metrics.

11 Service Architecture

Protection domains contain service faults and authority. Naming and protocol policy remain in userspace; privileged code manages lifecycle and resources.

11.1 Protection domains as failure boundaries

A protection domain (Chapter 3) is the unit of both authority and failure. When a service crashes:

- Its capability table is destroyed, invalidating its handles — client connections to the closed endpoint fail through the IPC lifecycle.
- Its memory mappings are removed — outstanding borrows are revoked; the kernel unmaps the pages from the (dead) borrower and restores the lender's access.
- Its threads are aborted — any thread blocked on a CQ or endpoint is woken with a terminal error.
- Its device grants are recovered — the supervisor regains ownership of any MMIO region and interrupt objects the domain held.

The MMU and capability checks are intended to contain the fault to that domain; kernel, hardware, and DMA defects remain outside this guarantee.

11.2 The supervisor — domain lifecycle manager

The **supervisor** is a kernel-side component that creates and destroys protection domains. It does not understand service protocols, but it does understand ELF images, memory mappings, and capability grants.

Its operations are:

Load Parse an ELF image, validate its segments, create a new address space, map `PT_LOAD` segments at their linked virtual addresses, set up a stack with a guard page.
Grant Populate the domain's config-page with the bootstrap capabilities it needs:

- A connection to the name service (for service lookup).
- For driver domains: an `MmioRegion` capability, an `Interrupt` capability.
- For service domains: an `Endpoint` capability on which to listen.
- A default CQ handle.

Spawn Create the initial thread with the domain's entry point and its affinity LP.
Monitor Register to be notified when the domain exits.
Teardown on exit, reclaim all capabilities, revoke mappings, reconcile outstanding operations, and make the address-space ID available for reuse.

Grants are minted kernel-side by the supervisor and never through a syscall. A driver does not ask the kernel for MMIO; the supervisor grants it at creation time based on the device topology it enumerated during boot.

11.3 The name service — discovery without the kernel

The kernel does not know about service names. A userspace **name service** maps human-readable names to connection capabilities:

Registration A service calls `register("driver.nic.v1", endpoint, access_key)` on the name service. The name service stores the endpoint's identity and opens a connection to it.
Generation tracking Each registration bumps a generation counter. If the service restarts and re-registers with the same name, the new generation replaces the old one. Existing connections bound to the old generation fail with `ServiceRestarted`.
Lookup A client calls `lookup("driver.nic.v1", access_key)`. The name service checks the access key (if the service registered one), then delegates a connection capability to the client — typically with `SEND | CALL` rights only.
Access-key gating A registration with a non-zero access key requires lookers to provide the matching key. This is userspace policy: the name service enforces it; the kernel does not participate.

The name service runs as a `no_std` EL0 service process. Boot starts exactly one node-local instance and the supervisor retains its registry endpoint. Each ordinary service or application receives an attenuated connection to that endpoint through its bootstrap page, so all processes on the node observe the same registry. Registering another name does not create another name-service process. A separate registry is appropriate only for an explicitly isolated namespace, such as a test or a future tenant domain.

The supervisor's endpoint handle remains kernel-private. Possessing a bootstrap connection permits the operations granted on that connection; it does not permit an application to mint arbitrary registry connections or inspect another process's capability table.

11.4 Service restart and recovery

When a service crashes and the supervisor restarts it:

1. Device reset — if the service was a driver, the supervisor resets the device before re-granting it to a new driver instance. The new driver sees a clean hardware state.
2. Generation increment — the name service bumps the service's generation. All old connections are invalidated.
3. Stale connections — clients connected to the old instance receive `ServiceRestarted` on their next call attempt. They re-lookup the service and receive a connection to the new instance.
4. Outstanding operation reconciliation — the supervisor reconciles any operations the old domain had submitted but not completed: cancellations are posted, borrowed memory is returned, DMA buffers are recovered.
5. Fresh bootstrap — the new domain receives a fresh config-page with freshly minted capabilities. It does not inherit the previous instance's capability table, reply tokens, or DMA ownership.

A restarted service receives fresh authority. Tests cover stale connections and operation reconciliation; the status appendix records the verified scope.

11.5 The overall system topology

At steady state, a CharlotteOS system consists of:

- The kernel — capabilities, endpoints, memory objects, CQs, scheduling, interrupt routing.
- The supervisor (kernel-side) — domain lifecycle, ELF loading, device enumeration.
- The node name service (one userspace instance) — the shared registry for service registration, lookup, access-key gating, and generation tracking.
- Service processes — one or more per logical function: NIC driver, TCP/IP stack, UTC time, filesystem, Raft consensus, application servers.
- Client processes — application code that looks up services and invokes them through capabilities.

Every component beyond the kernel, supervisor, and name service is a replaceable userspace process. A different NIC driver, a different filesystem, a different network stack — these are configuration choices, not kernel modifications.

11.6 Why service architecture matters for critical software

1. Failure containment is architectural — a service fault closes its endpoints and affects clients through defined errors rather than direct memory corruption. Availability still depends on the failed service and its dependants.
2. Recovery follows a defined sequence — the restart sequence (device reset, generation bump, connection invalidation, operation reconciliation) is the same for every driver. A new driver implemented to the same endpoint protocol replaces the old one without any special-case recovery code.
3. Policy lives in userspace — the kernel enforces mechanism (capabilities, endpoints, memory objects). Userspace enforces policy (who can register what name, who can look up what service, how many restarts are allowed, what log level to use). The kernel stays small; policy is auditable and replaceable.
4. The bootstrap contract is explicit and minimal — every domain starts with exactly the capabilities it needs and no more. The config-page contract defines what a domain receives; there is no ambient authority to discover or exploit.

12 Userspace Drivers

CharlotteOS runs device drivers as userspace processes with delegated device resources and endpoint interfaces.

12.1 The problem with kernel drivers

In a monolithic kernel, a driver has access to everything: all physical memory, all interrupt vectors, and all kernel data structures. A driver bug — such as a buffer overflow, use-after-free, or race — can corrupt the kernel and crash the system.

The microkernel solution is to move drivers to userspace. But simply moving them is not enough: a userspace driver that can still map arbitrary physical memory or receive arbitrary interrupts is still dangerous. The driver must be **granted** only what it needs, and the grants must be mediated by the kernel.

12.2 The DriverGrant

When a driver domain is created, the supervisor delivers a `DriverGrant` through the config-page contract:

```
struct DriverGrant {
    device: DeviceCap,        // identity and lifecycle of the device
    mmio: Vec<MmioRegionCap>, // register windows, page-granular
    interrupts: Vec<InterruptCap>, // IRQ sources for this device
    // DMA translation, if an IOMMU is present.
    dma_domain: Option<DmaDomainCap>,
    // Endpoint on which to serve clients.
    bootstrap_endpoint: EndpointCap,
}
```

The driver receives **exactly** the MMIO register windows and interrupt vectors it needs. It cannot:

- Map arbitrary physical memory.
- Receive interrupts not assigned to its device.
- Access other devices' MMIO regions.
- Perform DMA without explicit pinning into its DMA domain.

12.3 MMIO delegation

An `MmioRegion` capability represents a page-granular device register window. The driver maps it into its address space as Device-nGnRnE memory (user-accessible, execute-never, strongly ordered on AArch64; uncacheable on x86-64).

The mapping is a kernel-mediated operation: the driver calls `mmio_map(region_cap, addr)`. The kernel validates the capability, creates the page-table entries with the correct memory attributes, and returns. The driver then performs **direct EL0 MMIO** — load and store instructions to the mapped virtual addresses, with no kernel involvement on the fast path.

The common MMIO access therefore avoids a syscall or context switch.

12.4 Interrupt delivery

An `Interrupt` capability represents a routed interrupt source. The driver binds it to its completion queue:

```
interrupt_bind(int_cap, cq_id);
```

When the interrupt fires:

1. The IRQ dispatcher reads the per-interrupt route table atomically.
2. It masks the source at the interrupt controller (GICv3 on AArch64, x2APIC on x86-64).
3. It atomically bumps a coalescing counter.
4. It pushes a wake onto a deferred queue. The actual `completion::wake` (which takes locks) is performed from thread context by `yield_lp` and the LP idle loop.
5. The driver's shard wakes.
6. The driver reads the device's status register (direct EL0 MMIO) to determine the interrupt cause.
7. The driver processes the interrupt and unmasks the source.

The interrupt path performs an atomic counter update and deferred-wake enqueue; thread context performs the remaining work.

12.5 The reference UART driver

The PL011 UART driver demonstrates the full lifecycle:

- Direct MMIO writes to the transmit FIFO — no syscalls.
- Interrupt-driven deferred reads — the driver retains a reply token for a pending read. When the RX interrupt fires, the driver reads the receive register and completes the deferred reply.
- Cooperative shutdown — on graceful exit, the driver unmaps its MMIO region and closes its interrupt.
- Crash recovery — if the driver terminates without releasing its device caps (panic, infinite loop, killed by supervisor), the teardown path:
  1. Reclaims the MMIO region (unmaps it, returns to the kernel).
  2. Unroutes the interrupt.
  3. Reconciles outstanding operations: any pending reply tokens are invalidated; any borrowed memory is returned to lenders.
  4. Resets the device hardware.

  A restarted instance with fresh grants resumes under a bumped generation.

12.6 DMA and the IOMMU

For DMA-capable devices (network cards, storage controllers), a userspace driver needs more: the ability to tell the device "read from physical address X" or "write to physical address Y." Without an IOMMU, this is a safety gap: a malicious or buggy driver could program the device to DMA into kernel memory or another domain's pages.

The implementation has two platform levels:

- Without IOMMU — the driver is trusted with DMA. It still receives only the MMIO and interrupt it needs, and it still runs in a separate address space, so a CPU-side bug cannot read kernel memory. But a DMA programming bug can violate memory isolation.
- With IOMMU/SMMU — the DMA domain capability binds the device to a specific IOMMU translation table. The driver can only DMA into pages explicitly pinned into its DMA domain. The kernel validates every pin/unpin through the capability. The IOMMU hardware enforces the rest — even a malicious driver cannot DMA outside its domain.

On the implemented AArch64 ACPI path, `dma_map(domain, memory, direction)` returns an IOVA and `dma_unmap(domain, iova)` synchronously invalidates the IOTLB before releasing the memory pin. QEMU's coherent SMMUv3 is boot-tested with the userspace NVMe driver. Each PCI requester receives a private stage-1 context containing only explicitly mapped memory objects and its MSI doorbell page. SMMU event-queue faults are handled by the kernel.

On x86-64, ACPI DMAR selects Intel VT-d and IVRS selects AMD-Vi. Both backends create per-requester DMA domains, validate domain and requester limits, map and unmap IOVAs, perform synchronous IOTLB invalidation, and roll back partial attachment failures before frames can be reused. QEMU boots validate NVMe, AHCI, virtio-blk, and virtio-net through these domains. VT-d supports `INCLUDE_PCI_ALL` units and directly attached endpoint scopes; multi-hop bridge scopes remain rejected. AMD-Vi fault events are polled until post-topology MSI delivery is implemented. Non-coherent SMMUv3, ATS/PRI, and multi-IOMMU topologies remain future work.

12.7 Driver service layering

Complex I/O stacks decompose into separate failure and authority domains:

```
Application
    -> socket endpoint
Network service
    -> packet endpoint
NIC driver
    -> MMIO / DMA / IRQ
Hardware
```

The NIC driver knows about hardware registers and descriptor rings, the network service knows about protocols and sockets, and the application knows about its business logic. A crash in any layer is contained: the NIC driver crashes, the network service sees `ServiceRestarted`, and reconnects to a fresh instance.

12.8 Why userspace drivers matter for critical software

1. CPU-side driver faults are contained — a buffer overflow in a NIC driver is confined by its address space, assuming correct kernel, MMU, and DMA isolation.
2. The privilege model is grant-by-grant — a driver receives exactly the resources it needs. There is no "driver has access to everything because it's in the kernel" default.
3. Direct MMIO keeps the fast path fast — reading a device register uses a direct load rather than a syscall or context switch.
4. Restart follows a defined path — the supervisor handles driver restart (device reset, grant re-issuance, generation bump). Driver authors do not write recovery code.
5. Hardware-isolated DMA is implemented on the reference AArch64 SMMUv3 platform and on x86-64 with Intel VT-d or AMD-Vi. Advanced facilities and more complex IOMMU topologies remain future work.

13 Networking

CharlotteOS treats networking as an extension of its message-passing and capability model rather than making TCP/IP the native interface for all communication.

The guiding principle is:

> **Remote IPC is local IPC with a different transport.**

13.1 Why not sockets?

The Berkeley socket API (`socket()`, `connect()`, `send()`, `recv()`) has served the Internet for decades. But it is not the right foundation for a capability-based microkernel:

- Sockets identify hosts, not services — `10.2.3.17:443` tells you where to connect, not what you are talking to. The mapping from address to service is out-of-band (DNS).
- Sockets are byte streams, not messages — message boundaries are lost. Every protocol invents its own framing on top of TCP's undifferentiated stream.
- Sockets carry ambient authority — any process can open a socket to any address. There is no capability check; authority comes from being able to reach the network, not from being granted a connection.
- Sockets couple applications to transports — code written against TCP cannot transparently use shared memory for local communication or RDMA for high-performance communication.

CharlotteOS instead uses layers with separate responsibilities.

The stack below is the target architecture. The current repository contains a userspace virtio-net driver, an Intel 82574L/E1000E driver, a frame demultiplexer ("frouter"), a reliable-message service with fragmentation, a distributed name service that replicates a `name → {node, generation}` catalog across two guests, a smoltcp TCP/IP service, and a remote-invocation path through the catalog. The NIC/TCP path is runtime-validated on macOS QEMU TCG with both virtio-net and E1000E (`-relmsg-test`, `-disco-test`, `-dns-test`, `-tcpip-test`), and remote scalar remote-call prototype is implemented (`dns::OP_CALL`); the general remote capability delegation is not.

13.2 The native protocol stack

```
Applications
        |
Capability Invocation
        |
Distributed Objects
        |
RPC
        |
Reliable Message Layer
        |
```

```
Ethernet Frame Transport
         |
VirtIO / NIC Driver
```

13.2.1 Ethernet — generic frame transport

The NIC driver exports raw Ethernet frames. It knows about MAC addresses, MTU, and frame CRCs. It knows **nothing** about IP, TCP, RPC, or services. Its endpoint protocol has two operations:

- `OP_SEND(frame_buffer)` — transmit a frame via a moved memory object.
- `OP_RECV` — deferred receive: the caller retains a reply token; the driver replies with a received frame buffer.

Ethernet is a generic frame transport that can carry IP, ARP, LLDP, custom protocols, or CharlotteOS native messages. The driver provides the frames and higher layers interpret them.

13.2.2 Reliable message layer

This layer provides sequenced, acknowledged, reliable delivery of **messages** (not streams) over the raw frame transport. It handles:

- Sequencing (each message has a monotonic sequence number).
- Acknowledgements (the receiver confirms delivery).
- Retransmission (unacknowledged messages are resent).
- Fragmentation (messages larger than MTU are split and reassembled).
- Batching (multiple messages in one frame, one ACK for multiple messages).
- Congestion control (bounded in-flight windows).

The abstraction exported is a **reliable message**, not a byte stream. This is the building block for RPC, consensus protocols, and distributed objects.

Fragmentation is implemented: a message larger than one frame's payload (≈1460 bytes in wire protocol v2) is split across frames that share the message's sequence number, each fragment carrying its byte offset and a `FLAG_FRAG`/ `FLAG_MORE` pair in the header's reserved bits. The receiver buffers fragments by offset and reassembles the message once the last fragment arrives. The application-level ceiling is `relmsg::MAX_MSG` (65535 bytes).

Wire protocol v2 carries a 64-bit sender-session identifier. Data and acknowledgement frames assert it with `FLAG_SYN`. Observing a new, non-retired session resets the per-peer sequence spaces (including an outstanding transmission), so one reliable-message service can restart while its peer remains running. A bounded retired-session window rejects delayed frames from prior instances. The receive queue and fragment count/bytes are also bounded; a full delivery queue withholds sequence advancement and its acknowledgement, applying retransmission-based backpressure.

13.2.3 RPC layer

The RPC layer provides request/reply semantics over the reliable message layer:

- Synchronous-looking futures — `service.call(opcode, args).await?`
- Asynchronous invocation — the `Future` is backed by a pending RPC call; the shard continues other work while waiting.
- Object references — the target architecture can represent delegated remote authority; general remote capability delegation is not implemented.
- Serialisation — typed protocol crates handle encoding/decoding between Rust types and the wire format.

The implemented DNS v3 remote-call path is narrower than this target model. It currently carries scalar opcode/argument calls. A request is identified by caller node, caller DNS-session generation, and call ID, and a reply is accepted only from the expected peer. The implementation bounds outstanding calls at 64, returns `ERR_UNCERTAIN` after five seconds (execution may already have occurred), and retains 128 completed results to suppress duplicate execution within that window. The replicated catalog assigns a monotonically advancing service generation; requests and replies carry it, and the target rejects stale generations before execution. Remote object-capability delegation remains future work.

Registration uses two replicated phases. A `prepare` command allocates the next generation but remains invisible. After the owner successfully publishes its node-local endpoint, a generation-fenced `activate` command makes the entry visible. Failed local publication therefore leaves an inactive tombstone rather than a remotely resolvable ghost. Automatic replicated unregistration when the hosted service dies remains future work, but explicit unregistration is implemented. Its replicated tombstone must match the owning node and distributed generation. Subsequent removal from the node-local name service must also match the local generation captured before the proposal, so a delayed operation cannot remove a replacement instance.

Lookup and call-routing decisions do not use follower-local catalog state. A follower sends a correlated query to the known leader, and the leader answers only when Graft's quorum-contact read barrier succeeds. Query failure returns a safely retryable `ERR_NOT_LEADER`; a timeout after remote execution may have begun remains `ERR_UNCERTAIN`. Raft time is supplied by a persistent detached-timer completion, preventing sustained endpoint traffic from freezing the lease clock. Catalog and status dumps are diagnostic local observations, not linearizable reads.

13.2.4 Distributed objects

At the top of the stack, applications interact with **capabilities** — the same abstraction they use for local IPC:

```
let payroll: Capability<PayrollService> = lookup("Payroll")?;
payroll.call("calculate_salary", employee).await?;
```

The intended API hides whether a service uses local endpoint IPC or remote RPC. The implemented remote path is limited to the scalar DNS v3 call contract described above.

13.3 TCP/IP as a compatibility service

TCP/IP is supported as a userspace compatibility service, not the native programming model. The frouter owns the NIC's deferred `OP_RECV` and routes IPv4 (`0x0800`) and ARP (`0x0806`) frames to the `tcpip` service's `OP_FRAME` ingress; a smoltcp `phy::Device` adapter pulls those frames and transmits via `OP_SEND`:

```
// smoltcp adapter (receive is fed by the frouter's OP_FRAME ingress):
fn receive(&mut self, now: Instant) -> Option<(IpPacket, ...)> {
    self.rx.pop_front() // frames queued by the tcpip service
}

fn transmit(&mut self, now: Instant) -> Option<...> {
    // Calls OP_SEND with a memory object.
}
```

The TCP/IP stack itself uses smoltcp 0.13. It is wrapped in a userspace service that registers as `"tcpip"` and serves clients through a socket-API protocol (`OP_SOCKET`, `OP_CONNECT`, `OP_BIND`/`OP_LISTEN`, `OP_ACCEPT`, `OP_SEND`, `OP_RECV`, `OP_CLOSE`). Legacy software that requires TCP/IP connects to this service.

The architecture runners attach a NIC by default. Hardware discovery starts the driver and frame router, followed by discovery, cluster services, and the DHCP-configured TCP/IP service. The `-no-network` runner option is the explicit isolated-boot configuration. Options ending in `-test` add verification workloads and result reporting; they do not enable the runtime services.

13.4 UTC time as an internal service

The userspace `time` service is a client of the TCP/IP socket API. It waits for local boot readiness and a nonzero DHCP address, then samples an NTPv4 server over connected UDP. It calibrates the monotonic counter, estimates drift and uncertainty across samples, prevents time from moving backwards within a service lifetime, and persists calibration in the object store for holdover.

Applications do not speak NTP. They resolve `"time"` through the local name service and call its endpoint protocol for a full snapshot, scalar Unix seconds, or a fixed-width UTC ISO 8601 value. Synchronization state and uncertainty are explicit so callers can reject unsynchronized or weak holdover time. This service starts during an ordinary networked boot; no test option is required. The detailed wire contract and security limitations are recorded in `docs/reference/time-service.md`.

13.5 Capability-scoped external data services

CharlotteOS has narrow S3 and Kafka compatibility services on top of the userspace TCP/IP stack. They preserve the capability model at the application boundary even though they communicate with conventional external systems:

```
application
    |
    | delegated endpoint capability
    v
S3 or Kafka service
    |
    | owned TCP socket (+ verified TLS when configured)
    v
managed object store or Kafka broker
```

An application receives neither a generic network capability nor the remote credentials and protocol state. Instead, each service instance is provisioned with one bounded profile. The connection to that instance is the authority to use only the profile's endpoint, namespace, and operation rights. The application-side helpers build on `catten_rt::owned`: moved memory pages carry payloads, and each remote stream, delivery, consumer, or transaction has an explicit consuming teardown plus a best-effort `Drop` fallback.

These services are not launched unconditionally. A trusted provisioning path must supply their profiles, including secrets and trust anchors. The `-s3-test` and `-kafka-test` runner options create disposable Docker fixtures, generate short-lived test certificates, launch test profiles, and add guest verifiers. They validate integration; production provisioning does not conceptually depend on those switches.

13.5.1 S3 client service

One `s3` service instance is fixed at launch to an IPv4 endpoint and DNS name, port, region, bucket, key prefix, credential identity, optional Dell EMC ECS namespace, and a GET/PUT/DELETE rights mask. Applications use keys relative to the granted prefix. The service rejects absolute paths and `./..` path segments, constructs path-style S3 requests, and signs them with AWS Signature Version 4. The secret key remains inside the service address space.

The implemented data plane provides streaming GET and PUT, HEAD, DELETE, ranged GET, and create-only PUT. Object bodies cross the application boundary as moved, bounded memory chunks; response metadata includes length, ETag, version id, and request id. The HTTP/1.1 adapter accepts fixed-length and chunked responses. Bucket listing, multipart upload, presigned URLs, automatic region discovery, and a general AWS API are not implemented. A LIST right is reserved in the profile format but currently has no operation.

TLS profiles require an explicit DER trust anchor, a matching server name, system entropy, and synchronized UTC from the time service. Certificate verification or entropy failure aborts the connection; there is no downgrade to plaintext. The S3 request date also comes from synchronized UTC, so a TLS or signed request fails closed while the clock is unsynchronized.

The AArch64 integration test starts RustFS behind verified TLS and performs a PUT/HEAD/GET/DELETE round trip. The same client surface is intended for centrally managed S3-compatible stores. The optional `x-emc-namespace` header specifically supports Dell EMC ECS namespace profiles, but ECS interoperability has not yet been runtime-qualified in this repository.

The transport-independent Signature V4 and HTTP code lives in `crates/charlotte-s3`, the application protocol in `crates/charlotte-protocol-s3`, the owned client in `crates/catten-services/src/s3_client.rs`, and the service adapter in `crates/catten-services/src/bin/s3.rs`.

13.5.2 Kafka client service

One `kafka` service instance grants access to one broker endpoint, a fixed consume topic/partition, an ordered allow-list of produce routes, a consumer group, transactional id, and rights mask. It owns the broker socket, producer ids, producer epochs, and per-route sequence numbers. Applications can produce idempotently, open a directly assigned consumer at the group's committed offset (or the earliest retained offset), poll with `read_committed` isolation, commit or release a delivery, and close the consumer.

The complete version-2 broker profile is serialized into one immutable object, covered by a SHA-256 digest, and transferred to `kafka` with a kernel-enforced `MAP_READ`-only launch capability. The service validates

the exact size, version, digest, field bounds, route count, configurable ceiling, and duplicate routes before opening a broker connection. The hard ceiling is 64 produce routes, while deployment policy may select a lower limit. Metadata for all distinct declared topics is requested in one broker exchange.

Only the broker-facing service receives this profile. Producer, consumer, and transactional-step logic receives a connection capability to the provisioned service instance, never its broker address, trust material, or future SASL or mTLS credentials. Credential rotation can therefore replace the connector profile without rebuilding application or procedure artifacts.

The transaction owner supports the consume–transform–produce path. An application can produce records, move one consumed delivery's next group offset into the same Kafka transaction, and then commit or abort. Committing consumes the transaction resource; dropping an unfinished transaction attempts an abort. Dropping an uncommitted delivery releases it for redelivery rather than silently advancing the group offset. These linear owners make the local lifecycle explicit, but they do not turn arbitrary application side effects outside Kafka into an exactly-once transaction.

The wire crate negotiates `ApiVersions` and implements a deliberately small set of stable Kafka request versions. The current service requires a single broker that is simultaneously the partition leader, group coordinator, and transaction coordinator. It does not implement multi-broker routing or failover, consumer-group join/rebalance, SASL, record compression, topic administration, or arbitrary partition assignment. Records and the number of simultaneous consumers, deliveries, and transactions are bounded.

TLS uses the same explicit trust-anchor, synchronized-time, and entropy path as S3. The opt-in AArch64 test uses a single-node Apache Kafka fixture over TLS and verifies idempotent production, read-committed consumption, atomic production plus group-offset commit, transaction abort, and filtering of the aborted record.

The bounded Kafka wire implementation is in `crates/charlotte-kafka`, the application protocol in `crates/charlotte-protocol-kafka`, the linear client owners in `crates/catten-services/src/kafka_client.rs`, and the broker adapter in `crates/catten-services/src/bin/kafka.rs`.

13.6 Transport independence

The reliable-message interface is designed to be transport-agnostic. Candidate transports include:

- Ethernet — for inter-machine communication.
- Shared memory — for co-located processes on the same machine, where the kernel can set up shared memory regions instead of going through the NIC.
- Loopback — for same-machine same-stack communication.
- RDMA — for high-performance cluster interconnects.
- PCIe NTB — for backplane communication in multi-socket systems.

The intended RPC layer and everything above it should remain unchanged across transports. This is the same principle that makes local IPC transparent to the application when it moves from same-process to same-machine to different-machine.

13.7 Zero-copy networking

The intended network stack avoids copying whenever practical:

- Receive path — the NIC driver receives a buffer from hardware. That buffer (a memory object) is passed up the stack by transferring ownership — first to the message layer (for reassembly), then to the RPC layer (for deserialization), then to the application. No intermediate copies.
- Transmit path — the application creates a buffer. It is passed down by moving ownership. The NIC driver DMA's it directly from the application's pages.
- Buffer recycling — after the application consumes a received buffer, it returns the (now empty) memory object to the driver's RX pool. The driver reuses it for DMA and the cycle repeats. IOMMU-backed pinning is implemented with AArch64 SMMUv3 and x86-64 VT-d/AMD-Vi.

This is the memory-object transfer model (Chapter 6) applied to networking.

13.8 Why the networking architecture matters for critical software

1. Remote invocation follows the local call shape — the target API keeps transport choices out of application protocols. This permits:
   - Services can be relocated without application changes.
   - Testing can substitute a local mock for a remote service.
   - A common interface can reduce environment-specific application code, although the network boundary still introduces failures absent from local IPC.
2. Authority is not an address — an application invokes a service capability rather than relying on `host:port` reachability. Network access alone does not mint that capability.
3. Copy avoidance is a design goal — page ownership transfer can avoid intermediate payload copies. CPU mappings are MMU-enforced; DMA isolation is implemented with SMMUv3, VT-d, and AMD-Vi. No performance comparison with sockets has yet been established.
4. Message boundaries are preserved — the native abstraction is a message with identity, ownership, metadata, and boundaries. There is no need for an additional stream-framing layer. This can simplify protocol implementations, but does not eliminate parsing or framing defects.
5. TCP/IP is available but not mandatory — legacy software connects to the TCP/IP service. Native software uses the capability-based stack. The transition is incremental: a service can expose both a native endpoint and a TCP/IP socket through the compatibility service.

14 Consensus and Replication

CharlotteOS exposes distributed consensus as a capability service: a replicated state machine reached through the same endpoint IPC used by local services.

The test suite covers single-voter term recovery from NVMe and cross-machine catalog replication and invocation through `dns`. Each operational node runs exactly one durable Raft member inside `dns`; membership, names, deployments, keys, and cluster events share that member's ordered log. The multi-guest admission test observes it through discovery and exercises `JOIN`, joint consensus, and `FINALIZE`. The separate `raft.elf` process is network-disabled and retained only by the NVMe term-recovery fixture. Power-loss behavior within a live quorum is not yet validated end to end.

The core follows the shared Graft model for normalized peer identity, voters and learners, joint-consensus quorums, replicated `JOIN`/`JOINT`/`FINALIZE` commands, leader no-op entries, current-term commit rules, linearizable-read contact quorums, and membership-bearing snapshots. These paths have focused regression tests. The `dns` service exposes both client commands and membership administration end to end through one concrete state machine and log.

14.1 Raft as a capability service

The Raft consensus protocol provides a replicated, fault-tolerant state machine across a cluster of nodes. In CharlotteOS, the operational Raft node is owned by the per-node DNS EL0 process, which:

- Registers the DNS endpoint and a "`raft-<id>`" administration/status endpoint backed by the same in-process node.
- Learns transient peer MAC routes through discovery; voter authority changes only through committed membership commands.
- Carries VoteRequest, AppendEntries, InstallSnapshot, and admission messages through the reliable-message service.
- Uses completion-queue waits for time — a bounded `CQ_WAIT` drives election and heartbeat checks without a detached timer completion or a busy polling loop.
- Applies membership commands and DNS catalog commands to the same durable log and state-machine snapshot.

The Raft core (`catten-graft`) is a generic, transport-agnostic library. It implements the `RaftTransport` trait, which abstracts peer communication behind a few methods:

```
trait RaftTransport {
    fn send_vote_request(&self, request: VoteRequest);
    fn send_append_entries(&self, request: AppendEntries);
    fn send_install_snapshot(&self, request: InstallSnapshot);
    fn poll_completions(&self) -> Vec<RpcCompletion>;
}
```

The operational CharlotteOS implementation uses the reliable message layer between DNS replicas; the consensus core does not change.

14.2 Intended properties relative to socket-based Raft

14.2.1 Authority, not addresses

Once the cluster is running, peers are identified by name service entries, not `IP:port`. A node that has joined the cluster receives connection caps for its peers through the name service:

```
let peer_1: ConnectionCap = name_service.lookup("raft-node-1")?;
let peer_2: ConnectionCap = name_service.lookup("raft-node-2")?;
```

No IP configuration. No DNS. The authority to communicate with a peer is the capability; without it, you cannot even address the peer.

14.2.2 Transport transparency

The transport is behind the `RaftTransport` trait. A connection capability routes through local IPC (same machine), Ethernet frames (same network), or a future RDMA transport (high-performance cluster) — the consensus core never changes. This means:

- Development and testing can run a multi-node Raft cluster on one machine with local IPC.
- Deployment can switch to cross-machine networking by changing how peers are resolved, not by changing the Raft code.
- The cluster can be heterogeneous: some peers communicate via shared memory, others via Ethernet.

14.2.3 Zero-copy log replication

Locally, log payloads can transfer via Move memory objects: the kernel flips page-table ownership instead of copying bytes. A follower that receives a Move attachment gets the exact same physical pages the leader held — no intermediate buffer, no copy.

The current local transport uses one moved page per RPC and batches the largest AppendEntries prefix whose protobuf encoding fits. This is a 4 KiB transport attachment limit, not a persistent-object or Raft-log limit. A network transport can use different framing without changing the protobuf contract.

14.2.4 Capability-based security

A client invokes the Raft service only if it possesses a connection cap. The service manager attenuates rights: a typical client receives only `CALL` for `OP_CLIENT_COMMAND`, not for `OP_ADD_SERVER` or `OP_REMOVE_SERVER`. There is no ambient authority, no IP-based ACL, and no "any process on the cluster network can propose commands."

14.2.5 Failure isolation

Each DNS/Raft replica runs in a separate protection domain. A crash in one node does not affect the others. The isolated storage fixture demonstrates recovery of term/vote state across explicit process teardown and

restart; restarting a DNS replica and recovering its catalog inside a live quorum remains future integration work. The generic service lifecycle separately demonstrates that stale connections fail and clients can re-lookup a replacement generation.

14.3 Persistence and launch configuration

The operational DNS node requires persistent storage. Stable object identifiers derived from cluster and node identity hold term/vote state, log entries, and snapshot data; mutations are followed by an NVMe flush.

The cluster mnemonic and election timeout remain in the launch manifest. A fresh identity bootstraps as a durable singleton. Discovery supplies MAC routes, the lexicographically larger singleton applies to the smaller anchor, and active membership changes only through the Raft log and snapshot envelope. The distinct storage fixture verifies one generic node across process restart; the two-guest DNS test verifies the operational cluster.

14.4 Cluster bootstrap — the seed problem

A new node must contact at least one existing cluster member. CharlotteOS expresses the seed as a service name or capability rather than an IP address.

14.4.1 Ethernet discovery and deterministic admission (current)

Nodes broadcast their durable identity on the local Ethernet segment. DNS records every discovered identity-to-MAC route in its relmsg transport. Among fresh singletons, only the lexicographically larger identity applies to the smaller anchor. Before sending the request it durably enters a non-voting joining posture; the anchor then commits `JOIN`, catches the node up, commits the joint configuration, and finalizes it.

14.4.2 Pre-shared cluster capability (small kernel change)

A cluster administrator provisions a “cluster capability” — a connection to a known cluster member — and delivers it to new nodes via the config page. No name lookup required; the node already holds a first-class connection.

14.4.3 Ethernet broadcast discovery (zero-configuration)

On a single Ethernet segment, a joining node broadcasts a frame with a well-known EtherType carrying a “Raft peer discovery” payload. Existing members respond with their service names. The joining node looks them up through the name service to obtain attested connection caps. This works without any seed configuration.

14.4.4 Distributed name service (implemented)

The `dns` service realizes this end state: its `CatalogMachine` is a `StateMachine` implementation on the Raft substrate, and a new node needs only a single seed to discover all peers through the consistent name registry. There remains one process-facing name-service instance per machine; those instances apply the same committed registry state. The `ns` node-local registry and the `dns` cluster catalog remain distinct services.

14.5 The generic StateMachine trait

Raft replicates an opaque command log. The meaning of those commands is provided by a pluggable `StateMachine` trait:

```
trait StateMachine {
    fn apply(&mut self, command: Vec<u8>) -> Result<Vec<u8>, ApplyError>;
    fn snapshot(&self) -> Vec<u8>;
    fn restore(&mut self, snapshot: Vec<u8>);
}
```

Different services are different state machines on the same Raft substrate:

- Distributed name service — commands are "register(name, owner, route, generation)" and "unregister(name, owner, generation)". Unregistration is rejected unless both owner and generation still match the active entry. The owning DNS process watches the local endpoint through a waitable kernel completion. Endpoint death proposes this same fenced tombstone; a follower forwards an authenticated, idempotent request to its known leader and retries across leadership changes. A stale death notification therefore cannot remove a replacement generation. Scalar registration adopts an existing node-local service connection through a non-blocking local lookup. This lets DNS route to and observe the endpoint without granting ordinary lookup clients minting rights; names without a local endpoint remain catalog-only registrations. The state maps names to owning-node identities, routable service identities, generations, and policy metadata. Replicated, it makes service discovery available cluster-wide; this is the implemented `dns` service.
- Distributed lock service — commands are "acquire(lock_id)" and "release(lock_id)". The state is a set of held locks.
- Cluster configuration store — commands are arbitrary key-value writes. The state is a key-value map.

None of these state machines know about sockets, IP addresses, or TCP. They implement `apply()` and `snapshot()`. The Raft substrate handles replication, leader election, and fault tolerance.

Raft RPCs and DNS control messages use one serialized relmsg queue per peer. Because QEMU may acknowledge that queue slowly, consecutive queued AppendEntries requests and their serially ordered responses are coalesced, and heartbeats are generated only at a bounded fraction of the election timeout. Non-Raft control messages retain their relative FIFO order but pass queued, supersedable append traffic. A two-second stop-and-wait retry lease is shorter than the DNS remote-call deadline, preventing one lost heartbeat from holding the peer's only transmission slot long enough to starve service retraction, placement, or invocation traffic.

The DNS process also never waits synchronously for an arbitrary local service while running this reactor. Name lookup and local invocation are retained as bounded pending operations and polled on later turns.

Consequently a failed service cannot stop Raft heartbeats or relmsg receive draining. A timeout before execution is reported as a retryable lookup failure; after invocation has been submitted it is reported as an uncertain outcome. Duplicate remote calls are suppressed both while pending and in the acknowledged-result window.

14.6 Relationship to the name service

The node-local `ns` registry and replicated `dns` catalog are separate services. DNS applies committed catalog state and resolves remote routes; NS holds machine-local endpoint capabilities:

- Cluster registrations are submitted through the Raft leader and applied to each DNS replica; local endpoint publication remains an NS operation.
- Service generations are replicated consistently. A restart on node 3 cannot create a split-brain where node 1 sees generation 5 and node 2 sees generation 3.
- Replicated policy metadata and tombstones propagate through the log; local delegation still depends on machine-local authority.

Kernel capability identifiers and endpoint objects are machine-local and are not Raft state. For a local registration, lookup delegates a connection to the local endpoint. For a remote registration, the local name-service instance must return a local proxy connection whose transport reaches the owning node. Raft replicates the metadata needed to choose that route, not the capability object itself.

14.7 Election timers as first-class completions

In a traditional Raft implementation, election timeouts are implemented with `sleep()` or a polling loop:

```
// Traditional: busy wait or sleep
while !timeout_elapsed() {
    sleep_ms(10); // or just spin
}
```

In CharlotteOS, they use the completion system:

```
// CharlotteOS:
let timer_id = submit_timer(timeout_duration)?;
let result = wait_on_cq(timer_id).await?;
// Timer fired; start election.
```

The timer is a kernel-managed operation. The shard parks on its CQ. When the timer fires, the executor wakes, drains the CQ entry, and resumes the election task. No polling loop, no thread dedicated to sleeping.

14.8 Why consensus as a service matters for critical software

1. Consistency primitives can be exposed as services — a replicated service implements `StateMachine` and uses the shared Raft, transport, timer, and storage interfaces.

2. Capabilities, not IP addresses, identify peers — the cluster configuration is a set of capabilities, not a list of `host:port` pairs. A network reconfiguration (new NIC, new subnet, move to RDMA) need not change the replicated peer identities.
3. Timers are first-class and reliable — an election timeout that is lost because the polling loop missed its window is a correctness bug (it can cause spurious leader changes or extended unavailability). Kernel-managed timers use the non-lossy CQ delivery path exercised by the local-election test.
4. Failure isolation can extend to the consensus layer — a Raft node is a userspace process. It can crash, restart, and recover without affecting the kernel or other Raft nodes. The supervisor handles the lifecycle; the Raft protocol handles the consensus.
5. A distributed name service is implemented — the `dns` service runs the `CatalogMachine` state machine on the Raft substrate; all other services that need cluster-wide consistency can use the same `StateMachine` pattern and shared consensus implementation.

15 Persistent Storage

CharlotteOS treats storage as a layered stack of userspace services, each speaking a well-defined protocol through endpoint IPC. The block driver, object store, and filesystem services run in userspace. The kernel still participates through capability, memory-mapping, physical-address, interrupt, and PCI/MSI-X setup paths.

15.1 The storage stack

```
Filesystem ("fs")           ← charlotte-protocol-fs
    |
Object store ("obj")        ← charlotte-protocol-objstore
    |
Block driver ("blk0")       ← charlotte-protocol-block
    |
NVMe controller             ← MMIO + DMA + MSI-X (granted at boot)
```

15.2 NVMe block driver

The NVMe driver is a userspace EL0 prototype implementing the subset needed for QEMU bring-up: controller and namespace identification, queue creation, and basic read, write, and flush operations. It is not a claim of NVMe 1.4 compliance. It receives an `MmioRegion` capability covering the controller's BAR0 register window, an `Interrupt` capability for I/O completion signalling, and a `DmaDomain` capability bound to the controller's PCI requester ID. Queue and data memory are mapped with `dma_map`; the driver receives contiguous IOVAs rather than physical addresses. The kernel pins the backing frames and removes each translation with a synchronous SMMU invalidation before unpinning.

15.2.1 Initialisation

The driver follows the standard NVMe initialisation sequence:

1. Controller reset — disable CC.EN, wait for CSTS.RDY = 0.
2. Admin queue setup — allocate physically contiguous memory for the Admin Submission Queue (ASQ, 32 entries) and Admin Completion Queue (ACQ, 32 entries). Write AQA, ASQ, and ACQ registers. Enable the controller and wait for CSTS.RDY = 1.
3. Identify controller — submit an Identify command (CNS=1) through the admin SQ. Read capabilities, version, and supported features.
4. Set Features (Number of Queues) — declare how many I/O queue pairs the driver intends to use. QEMU's NVMe controller requires this before accepting I/O queue creation commands.
5. Identify namespace — submit an Identify command (CNS=0, NSID=1) to read the namespace size (NSZE) and the active LBA format (FLBAS), which determines the block size.
6. Create I/O Completion Queue — allocate memory, submit a Create I/O CQ admin command. The CDW11 register must set bit 0 (PC = Physically Contiguous) to 1 — this is required by the NVMe specification and enforced by QEMU's device model.

7. Create I/O Submission Queue — allocate memory, submit a Create I/O SQ admin command, associating it with the I/O CQ created in the previous step.

After initialisation the driver registers `"blk0"` with the name service and enters its main loop, blocking on `cq_wait` for endpoint messages and I/O completions.

15.2.2 I/O path

OP_READ receives a BorrowWrite memory attachment from the caller. The driver maps it into the controller's DMA domain, builds a PRP chain from the returned IOVA range, and submits an NVM Read. On completion it unmaps the DMA translation, replies, and lets the kernel revoke the borrow and return the filled buffer to the caller.

OP_WRITE receives a Move memory attachment, maps it for DMA, builds the PRP chain, and submits an NVM Write. On completion the driver removes the mapping and replies; the moved buffer is consumed.

OP_FLUSH submits an NVM Flush command to ensure all previously written data reaches durable media before replying.

15.2.3 Lessons from bringup

The NVMe driver exposed several non-obvious requirements:

- CDW11 bit 0 (PC) must be 1 — QEMU's NVMe device model requires physically contiguous queues. Without this bit, the Create I/O CQ command returns status 0x4002 regardless of all other parameters.
- SQE layout is exacting — the Command Identifier occupies bits 31:16 of DW0, the opcode occupies bits 7:0. The NSID shares DW0's high bits with DW0 in the first u64. PRP1 is at byte offset 24, CDW10-11 at byte offset 40. Getting any of these wrong produces silent failures.
- Doorbell stride must be derived from CAP.DSTRD — on QEMU this is 0 (stride = 4 bytes), but the architecture supports strides up to $4 \ll 15$ bytes.
- MMIO mapping must cover the doorbell region — the NVMe controller registers occupy offsets 0x0000–0x003F within BAR0, but doorbell registers start at offset 0x1000. A single-page MMIO grant is insufficient — at least two pages are needed.
- On TCG, `cq_wait` is required for VM exits — a pure spin-loop prevents QEMU from processing I/O because the guest never yields. The driver must block on `cq_wait` to let the host process NVMe commands.

15.3 Block device protocol

The block protocol (`charlotte-protocol-block`) is a minimal `no_std` crate defining the endpoint IPC interface between block device consumers and drivers:

Opcode	Name	Description
1	OP_INFO	Query block size and total block count. Scalar call/reply.

2	OP_READ	Read blocks into a BorrowWrite memory object.
3	OP_WRITE	Write blocks from a Move memory object.
4	OP_FLUSH	Flush write cache to durable media.
5	OP_TRIM	Discard a block range (optional).

The protocol is device-independent: consumers speak `OP_READ`/`OP_WRITE` without knowing the underlying hardware. Three userspace implementations are present. NVMe is exercised on AArch64 and x86-64; x86-64 additionally runs AHCI and virtio-blk implementations through the same protocol. The QEMU matrix validates identify/capacity discovery, bounded read and write requests, flush completion, controller timeouts, and the persistent object-store/Raft restart path. On x86-64 all three transports are tested behind VT-d or AMD-Vi DMA domains. Ramdisk and network block drivers would still require separate implementations.

15.4 Object store

The persistent object-store prototype (`charlotte-protocol-objstore`) provides dynamically sized blob storage on top of a block device. Ordinary objects receive monotonically increasing 64-bit IDs; service-owned objects can use stable IDs through `OP_CREATE_AT`. Stable IDs let a replacement Raft process reopen state derived from its cluster and node identity.

15.4.1 On-disk layout

```
LBA 0--1:  checksummed superblock slots
next:      device-sized allocation bitmap
next:      two device-scaled directory banks:
             id | flags | generation | header_lba |
             header_blocks | crc32
data:      object headers and up to 16 data extents per object
```

15.4.2 Current durability boundary

Each object header is versioned and carries explicit header length and block count, object generation, exact and allocated data lengths, data offset, extent information, and a mandatory FNV-1a content hash. The header can grow without moving the data offset contract. The current format records up to 16 extents, so fragmented free space does not require one large contiguous run.

Replacement is copy-on-write. New data, header, and allocation state are flushed before the directory locator changes, and the old allocation is freed afterward. Mount reconstructs the bitmap from validated reachable headers, reclaiming pre-commit allocations. Whole-object transfers use multi-page memory objects and are divided into block requests of at most 512 KiB, so the NVMe request limit is not an object or file limit. The current service API's 32-bit length limits an individual object to 4 GiB and available disk space. Directory records are checksummed and mirrored. Each update goes to the generation-selected bank, and mount chooses the newest valid copy per record; versioned tombstones prevent deleted records from reappearing. This recovers from one torn directory-sector write. Full device-level sudden-power-loss guarantees still

depend on truthful flush/FUA behaviour. FNV-1a detects accidental corruption, but is not cryptographic authentication.

15.4.3 Operations

- **CREATE** — allocate a directory slot, return a new object ID.
- **CREATE_AT** — create a caller-selected stable object ID, or report that it already exists.
- **WRITE** — replace the object's content. The caller moves a suitably sized memory object; the store writes it in bounded chunks.
- **SET_SIZE** — set the exact byte length consumed by the next whole-object write.
- **READ** — return the object's data as a moved memory object.
- **DELETE** — mark the directory entry as free, return the extent blocks to the free bitmap.
- **FLUSH** — flush the underlying block device.

15.4.4 The initial service-artifact store

The host build produces an architecture-qualified service bundle and blesses every ELF according to `crates/catten-services/artifact-policy.tsv`. AArch64 embeds only the set needed to reach storage (name service, NVMe, object store, UART, and the system observer); `scripts/make-nvme-image.py` writes the remaining services into an initial version-3 object-store image. The QEMU x86-64 runner can use the same host-preseeded model.

The x86-64 kernel also carries an immutable signed installation bundle for the blank-disk appliance path. Once NVMe and the object store are available, it checks the stable object ID for every bundled logical name. A valid stored artifact is retained, including a newer deployed version; a missing artifact or one that fails identity and signature verification is replaced from the bundle. The installer flushes the store before dependent services start, so the operation is idempotent across reboot. Services are subsequently resolved by logical name, read through the object-store protocol, identity-checked, and cached for the boot. The ordinary loader independently enforces ELF and signature policy before mapping.

The signature record is CLS2, an identity-bearing ELF note. Besides signing the complete bytes, it binds the logical service name, artifact class, release and rollback counter, policy flags, signing-key id, and an optional provenance digest. Consequently a validly signed `dns` ELF cannot occupy the `raft` slot. A deployment generation also carries the complete ELF SHA-256, preventing an object replaced under the same logical name from changing already-committed code.

For preseeded QEMU images, the runner fingerprints the whole blessed bundle. It rebuilds an instance's initial NVMe image when that fingerprint changes, while `-reuse-storage` deliberately retains an older store and emits a staleness warning. `-fresh-storage` always recreates it. The image producer refuses an object-id collision rather than silently replacing an earlier service. The x86-64 options `-blank-storage` and `-build-only` instead support guest-side installation and appliance packaging without constructing a preseeded store.

15.5 Managed object storage through S3

The local `objstore` and the S3 client service solve different problems. The local store is part of node bootstrap and supplies service artifacts, Raft persistence, and the native filesystem without depending on a

working network or external credentials. The `s3` service is an application-facing compatibility adapter for centrally managed object stores. It runs above TCP/IP and, when configured, verified TLS; it is described in Section 13.5.1.

An S3 service profile narrows authority to one endpoint, bucket, key prefix, credential identity, optional Dell EMC ECS namespace, and operation mask. Applications receive only the profile's endpoint capability and transfer object data as owned memory chunks. This makes external object storage usable without handing each application general socket access or raw S3 credentials.

RustFS is the current integration fixture. Dell EMC ECS is an intended managed store and the adapter can emit its namespace header, but it has not yet been qualified against an ECS installation. The cluster deployment agent also does not currently fetch signed service artifacts from S3: deployment bytes remain pre-staged in each node's native object store. Using S3 as an artifact source would require an explicit fetch, digest verification, caching, availability, and credential-rotation policy rather than treating the external bucket as the boot store.

15.6 Raft persistence

The Raft service can use in-memory stores, optionally try persistent stores, or require the object store. Required mode waits for `"obj"` through the name service; it does not use a boot-time polling range. Four stable objects per cluster/node namespace contain term and vote state, snapshot metadata, snapshot data, and serialized log entries. Mutations write the object and flush the underlying NVMe device before returning.

The ordinary two-node local election test deliberately uses memory stores, so the generic consensus service remains testable on a multicore machine without NVMe. A separate storage test starts a single voter with required persistence, records its elected term, tears down that process, restarts the same identity, and requires the new election term to be greater. A repeated term 1 would fail, so the test distinguishes recovery from a fresh in-memory start.

Launch configuration—node identity, peers, election timeout, cluster name, and storage policy—remains in the supervisor-provided manifest. The persistent objects hold Raft protocol state, log entries, and snapshots. Cross-machine catalog replication is provided separately by the `dns` state machine described in Chapter 14 and Chapter 17; this generic Raft persistence test remains a single-voter restart test.

15.7 Native filesystem

The filesystem (`charlotte-protocol-fs`) builds hierarchical naming on top of the object store. Directories and files are stored as objects; the root directory is a fixed object ID (100).

15.7.1 Directory encoding

Each directory object stores entries as a packed sequence:

```
[name_len: u32 LE][name: UTF-8]
[file_id: u64 LE][flags: u32 LE][size: u64 LE]
```

A `name_len = 0` marker terminates the list. Flags distinguish directories (bit 0 set) from files (bit 0 clear).

15.7.2 Path resolution

Paths are walked client-side by chaining `OP_LOOKUP` calls. The filesystem service does not understand path strings — it only resolves single-component lookups within a given directory. This keeps the filesystem protocol minimal: six operations (LOOKUP, CREATE, READ, WRITE, DELETE, LIST) plus FLUSH.

15.7.3 Host inspection

A Python tool (`scripts/fs-inspect.py`) reads the raw disk image and walks the object store and filesystem on-disk structures. It understands both v3 directory banks and multi-extent headers, selects the newest valid directory record, checks CRCs and allocation overlap, compares reachable blocks with the bitmap, and verifies object contents against their FNV-1a hashes. It provides tree, dump, raw, cat, object-list, metadata, information, and format commands, enabling post-mortem debugging without booting CharlotteOS:

```
scripts/fs-inspect.py <image> info
scripts/fs-inspect.py <image> objects
scripts/fs-inspect.py <image> metadata
scripts/fs-inspect.py <image> metadata OBJECT_ID
scripts/fs-inspect.py <image> metadata /path/to/file
scripts/fs-inspect.py <image> tree
scripts/fs-inspect.py <image> cat /raft/state
```

Metadata output includes numeric and symbolic object/filesystem flags, generation, header location, logical and allocated lengths, integrity hash, and every extent. Unknown flag bits remain visible. The `format` command creates a blank v3 store and destructively replaces the image contents.

15.8 Why this matters for critical software

1. Storage policy lives in userspace — a bug in the NVMe driver, the object store, or the filesystem corrupts that service's address space, not the kernel. The kernel grants MMIO and interrupts; everything else is userspace.
2. Device-independent protocol — the block protocol is intended to let the object store and filesystem work over other block drivers. Those alternative drivers have not yet been implemented.
3. Crash-safety mechanism under development — process-restart persistence is boot-validated. Copy-on-write object replacement and mirrored directory records recover from one torn directory-record write. Broader power-loss guarantees still require validation of device atomicity and truthful flush/FUA ordering.
4. Capability-based access — a client that holds a connection to `"blk0"` can read and write blocks. A client that does not hold that connection cannot. There is no ambient filesystem namespace to escape.
5. Host-inspectable — the included Python inspector reads the raw disk image and reports its documented object and filesystem structures.

16 Live Service Upgrade

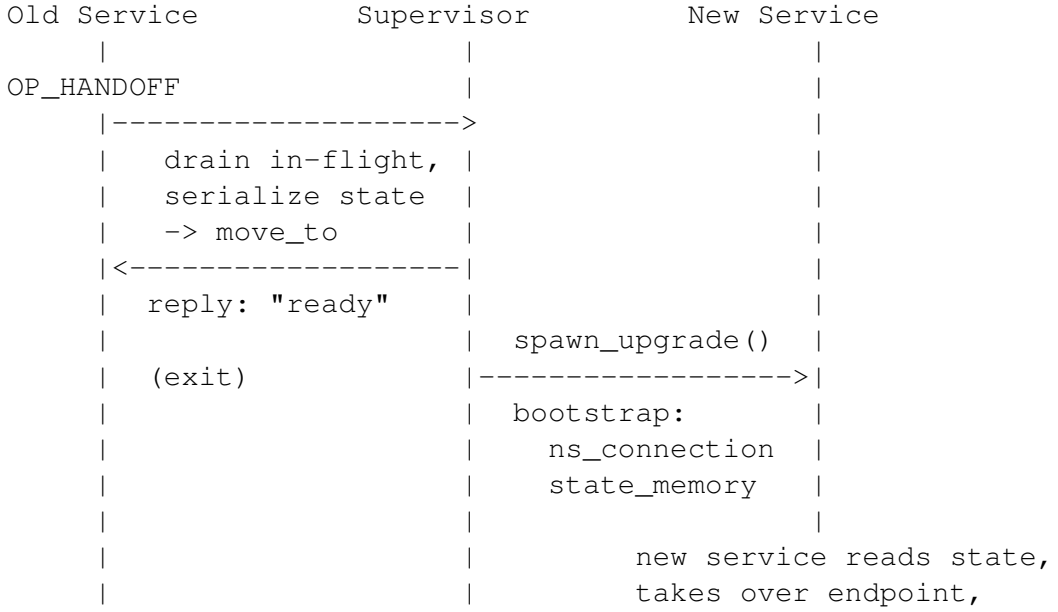
CharlotteOS has a tested prototype for replacing a reference service while transferring its state. It is **restart-with-state**, not zero-downtime: in-flight requests may fail during handoff and clients must re-lookup and retry.

16.1 The four primitives

The prototype builds on four primitives implemented in the kernel and name service:

1. Memory-object ownership transfer — the kernel's `memory::object::move_to` moves a page-backed memory object from one address space to another. The sender loses access; the receiver gains it. This is the vehicle for service state.
2. Generation tracking — the userspace name service records an instance generation that increments on restart. A stale connection returns `EndpointClosed`; a new lookup returns the replacement generation and connection.
3. EndpointClose and Cancelled — when a service domain exits, the kernel closes its endpoint. Queued requests that were never delivered complete as `EndpointClosed`. Delivered calls whose server exited before replying complete as `Cancelled`. Clients see a deterministic failure, not a hung request.
4. Bootstrap capability delivery — the supervisor writes initial capabilities to the config page before a domain starts. For a live upgrade, the bootstrap contract is extended: in addition to the name service connection, the new domain receives the old service's state as a memory object. The current reference replacement creates and registers a fresh endpoint.

16.2 The handoff protocol



```
          |                    |         registers (new generation)
          |                    |                    |
                    clients see EndpointClosed
                    -> re-lookup -> fresh connection
                    -> resume from where they left off
```

The old service receives an `OP_HANDOFF` request (typically from the supervisor or a service manager). It drains any in-flight work, serialises its mutable state into memory objects, moves those objects to the supervisor's address space, and replies "ready" before exiting.

The supervisor then spawns the new service instance with an extended bootstrap contract containing the name service connection and handoff state. The new service deserialises the state, creates a fresh endpoint, and re-registers under the same name. The name service bumps the generation. Teardown of the stopped old domain invalidates its stale client connections.

16.3 Executable source

Embedded service images are bootstrap and recovery inputs. They are not the only executable source. The service manager first tries the object store and reads the well-known executable object for the target service. The returned multi-page memory-object capability and exact byte length are passed to `spawn_upgrade`. If storage is absent or the object is not installed, the manager selects the embedded recovery image.

The kernel snapshots the capability-backed bytes into kernel-owned memory before parsing them, then checks the ELF class, architecture, program-header bounds, segment bounds, page congruence, W^X ("write xor execute")[1], integer overflow, and that the entry point lies in an executable load segment. This prevents a mutable userspace image from changing between validation and mapping.

Executable artifacts carry an Ed25519 CLS2 note that binds the complete ELF to its logical service name, artifact class, release metadata, rollback counter, policy flags, and signing-key identifier. The loader snapshots the bytes and verifies the note before mapping. The FNV-1a object checksum remains corruption detection, not executable authority. Rollback policy, key rotation, and measured boot remain future policy work.

16.4 What the supervisor prototype provides

```
pub struct UpgradeGrant {
    pub state_caps: Vec<MemoryObjectCap>,
}

pub fn spawn_upgrade(
    image: &[u8],
    name_service: &NameServiceHandle,
    old_domain: ServiceDomain,
    grant: UpgradeGrant,
) -> ServiceDomain;
```

[1] a memory page may be writable or executable, but never both simultaneously

`spawn_upgrade` is a privileged kernel operation invoked by the userspace service manager. It validates that the manager holds a callable connection to the target, loads the new service ELF, moves the state memory object to the new domain, writes the bootstrap config page, and starts the new domain. The replacement registers a fresh endpoint and the name service atomically publishes the new connection and generation before retiring the old connection or releasing deferred lookups. The reference path supports both a persistent signed echo image and its embedded fallback, plus one state object. The kernel handoff helper can transfer multiple state objects and rolls a partial transfer back if a later move or endpoint delegation fails. The current reference manager still exercises one object. Rollback policy and manager-owned teardown remain future work.

16.5 What the service must implement

The old service must handle `OP_HANDOFF`:

```
OP_HANDOFF => {
    // Stop accepting new work.
    // Serialize all mutable state into memory objects.
    // Move those objects to the supervisor's address space
    //   (the supervisor passes its ASID in the handoff call).
    // Reply "ready", then exit.
}
```

The new service reads the extended bootstrap contract:

```
fn main(ctx: Context) -> ! {
    let ns_connection = ctx.bootstrap_cap();
    let state_count = ctx.handoff_count();
    let state = ctx.handoff_state_cap();

    // Deserialize state, create and register a fresh endpoint
    // under the same name (NS bumps generation), start serving.
}
```

16.6 What clients see

A client holding a stale connection from the previous generation sees a clean failure and retries:

```
// Before upgrade:
let conn = ns.lookup("payroll")?;       // generation 3
payroll.call(OP_CALCULATE, employee_id); // in-flight

// During upgrade: call completes as EndpointClosed.
// Client retries:
let conn = ns.lookup("payroll")?;       // generation 4
payroll.call(OP_CALCULATE, employee_id); // reaches new instance
```

The new instance receives the old instance's state. The client observes a failed request, performs a new lookup, and retries. Protocols must decide when a retry is safe.

16.7 Restart-with-state vs. zero-downtime

This design is **restart-with-state**: in-flight requests to the old instance fail during the handoff window, and clients must retry. It is potentially suitable for services whose protocols define retry and state recovery. No general latency bound has been measured, and clients must explicitly handle transient failures through name-service re-lookup.

True zero-downtime would require one of:

- Queue migration — the old service's endpoint queue would need to be transferred to the new service atomically, so no request is ever rejected. This is a future kernel feature.
- Sidecar model — the new service starts alongside the old one, both sharing the same endpoint, and the old service stops accepting new work. The kernel does not currently support multiple owners for one endpoint.

16.8 Relationship to crash recovery

The live upgrade path is the graceful variant of the crash-recovery path already exercised by the UART driver test (Chapter 12). In both cases, the supervisor reclaims device authority, invalidates stale connections, and spawns a new instance with a bumped generation. The difference is that live upgrade transfers state explicitly through memory objects before the old instance exits, rather than starting the new instance from scratch.

16.9 Why this matters for critical software

1. The reference path transfers service state — the self-test demonstrates handoff for the reference service. Filesystems, network stacks, and consensus nodes would each need protocol-specific serialization, compatibility, and recovery work.
2. The upgrade sequence is explicit — the old service drains in-flight work, serialises state, and exits. The new service starts and registers. Clients retry. A hard time bound is not currently enforced or benchmarked.
3. Failure recovery has an ownership model — if the new service fails to start, a future supervisor could restart the old service or a fallback. The state memory objects are owned by the supervisor during the transition, so they are not lost.
4. The building blocks are covered by self-tests — memory-object transfer, generation tracking, deterministic teardown, and bootstrap delivery are exercised by the existing self-test suite. This evidence does not establish crash consistency, arbitrary-service compatibility, or production readiness.

17 Server-Class Cluster Vision

CharlotteOS is exploring an operating-system boundary at which the cluster, rather than an individual machine, is the primary administrative and computational unit. An operator should not install a program on a chosen server. The operator should assign a signed workload to a cluster, declare the resources, authority, multiplicity, and placement constraints it needs, and let the cluster decide where the workload runs. Nodes should be replaceable providers of processors, memory, devices, and failure domains, not long-lived machines whose local software configuration is operationally interesting.

This is an architectural vision, but we are taking steps in this direction. What we can demonstrate now is a two-node QEMU cluster with replicated naming and deployment state, signed ELF artifacts, per-node NVMe object stores, deployment agents, remote invocation, and a demonstrated stateless assignment handoff. Discovery-bridged admission has also been demonstrated for a generic Raft group, but the production DNS voter set is still launched statically. Automatic placement, shared artifact storage, replica-aware routing, stateful cross-node migration, and a production administration plane remain future work.

The purpose of this chapter is therefore twofold. It first states the cluster-level model CharlotteOS is trying to reach. It then uses Linux containers and Kubernetes/OpenShift as a counterpoint for identifying which parts of a modern cluster platform are intrinsic consequences of distributed computation and which parts arise from adapting general-purpose, machine-centred operating systems to isolated, reproducible, movable workloads. The comparison is a research question, not a claim that the prototype already replaces a production container platform.

17.1 The cluster is the computer

17.1.1 Nodes are resources, not deployment targets

The central proposition is simple:

> A Charlotte node is not an administrative unit. The Charlotte cluster is the administrative unit.

A node boots, discovers or is directed to a cluster, establishes its identity, joins shared state, advertises resources, and receives work. Its useful description is not a list of packages and locally installed applications, but properties such as processor architecture, memory, accelerators, attached devices, trust domain, and failure domain. If a node fails, the operator should replace the resource and let the cluster reconstruct the intended placement. If a workload moves, clients should continue to address its cluster name rather than learn the identity of its new host.

This makes individual nodes deliberately uninteresting. They still have identities where consensus membership, failure detection, routing, and ownership fencing require them, but those identities are mechanisms, not the operator's application-deployment model. The distinction is important: interchangeable nodes need not be physically identical, and failure domains cannot be ignored. Differences are expressed as schedulable properties and constraints rather than as hand-maintained machine personalities.

The same idea already appears one level down in CharlotteOS. The shard-per-core model of Chapter 9 treats cores as executors over explicitly owned state, while the scheduler in Chapter 10 moves work between them. The cluster vision scales this relationship outward: a cluster node is analogous to a core, cluster-visible

storage is analogous to owned persistent state, and the cluster becomes the unit within which execution is placed.

17.1.2 Workloads are signed execution objects

In the target model, cluster software is not a miniature general-purpose machine encoded as a filesystem. It is a signed execution object with a logical identity and a deliberately small execution environment. Its signed metadata can bind, among other things, its name, class, release, rollback counter, permission to run concurrent instances, and a digest linking to retained provenance.

Signature validity answers only whether the cluster accepts the artifact's identity and origin. It does not grant the workload arbitrary authority. Once loaded, a workload executes in its own address space and receives only the capabilities needed for its purpose. Authority is delegated positively: a component may receive a service endpoint, a storage object, a device, or an administration capability. It does not begin with a complete Unix machine view that must subsequently be hidden from it.

Signing and capability confinement solve different problems and both are required. A correctly signed but over-authorised program can still cause damage; a tightly confined but unauthenticated program is not acceptable cluster software. CharlotteOS combines an immutable, name-bound artifact identity at the loading boundary with explicit authority at the execution boundary.

This clean boundary also changes what installing software means. Nodes need not resolve packages, execute installation scripts, or accumulate machine-local dependency state. Third-party code can still enter during the build, but the cluster receives a self-contained, blessed artifact and its associated evidence. Reproducible builds, vulnerability policy, SBOMs, source attestation, and signing-key custody remain essential supply-chain responsibilities; they are not replaced by the operating system.

17.1.3 Assignment is declarative and cluster-wide

The desired operation is not to install version 71 of a claims engine on servers A, B, and C. It is to make that version active in the cluster with a desired number of instances and declared resource, authority, affinity, anti-affinity, and failure-domain constraints.

That declaration becomes replicated desired state. A cluster controller compares it with observed state and chooses assignments. An assigned node obtains the immutable artifact, verifies its identity and signature, creates an isolated execution domain, and supplies its delegated capabilities. Rollback is another desired-state change to an earlier accepted artifact, not a sequence of machine-repair commands.

Placement is not a fixed one-component/one-node map. A component explicitly blessed as safe for parallel execution may have several instances. Closely interacting components may be co-located to reduce communication cost, while replica anti-affinity and minimum failure-domain rules keep latency optimisation from defeating availability. Node capacity, architecture, devices, policy domains, and actual load all constrain the decision.

Routing must follow the same abstraction. Clients address a logical service through the cluster name service; they do not bind application configuration to a host address. The runtime may choose local delivery when caller and callee are co-located and network delivery when they are not. If placement changes, the name's generation changes, stale endpoints are invalidated, and clients resolve again.

Migration is consequently a cluster operation, not an administrator logging into two machines. Stateless reassignment starts an equivalent instance at the new location and retires the old one under generation

fencing. Stateful migration additionally requires an explicit transfer of owned mutable state and a single, fenced change of ownership. CharlotteOS has demonstrated only the stateless cross-node case; the stateful protocol remains a target.

17.2 Kubernetes and OpenShift as a counterpoint

17.2.1 What the container stack constructs

A Linux container is not a new kernel object equivalent to a virtual machine. It is a group of ordinary Linux processes made into a deployable unit by combining several facilities: namespaces determine which resources and identifiers are visible, cgroups account for and limit resource use, Linux capabilities divide some root privileges, seccomp restricts the system call surface, an LSM such as SELinux constrains object access, and a prepared filesystem and network configuration provide the process's apparent environment. An OCI image and runtime turn those facilities into a repeatable execution contract. Kubernetes adds scheduling, desired-state reconciliation, service naming, failure recovery, rolling change, and many other cluster functions. OpenShift adds an integrated security, policy, build, registry, identity, networking, and operations environment around that model.

This machinery exists for good reasons. Linux is a general-purpose, machine-centred operating system with a vast compatibility surface. Conventional applications expect filesystems, shared libraries, package contents, users, process APIs, network interfaces, and often a distribution userspace. Container platforms must package enough of that environment to make a program reproducible, then construct boundaries that let mutually distrustful workloads share a kernel. OCI images need not contain a complete distribution, but they commonly carry a substantial userspace precisely because existing software was written as if it ran inside a machine.

Kubernetes can therefore be understood partly as a distributed operating-system layer over machines whose native abstractions stop at the node. It translates from the operator's model of workloads in a cluster to Linux processes on machines. That observation is not a criticism of Kubernetes. Its great practical advantage is that it orchestrates the software and operating systems that already exist.

CharlotteOS asks a different question: what becomes possible when the controlled distributed execution environment is the operating system, rather than something assembled above a set of general-purpose hosts?

17.2.2 Two sources of complexity

The comparison is useful only if it separates two kinds of complexity.

17.2.2.1 Distributed-systems complexity is intrinsic

CharlotteOS does not abolish scheduling, consensus, placement, replication, health management, service discovery, upgrades, resource accounting, storage, networking, observability, failure detection, load balancing, or recovery from partial failure. Network partitions still occur. Replicas still need consistency rules. Persistent state still needs ownership, durability, repair, and backup. Placement still needs capacity and policy constraints. Rolling change still needs compatibility and rollback. Secrets still need secure creation and delegation. An operating system that absorbs orchestration must implement these responsibilities; moving them downward does not make them disappear.

17.2.2.2 Compatibility and boundary-construction complexity may change

Some machinery exists because a cluster platform must turn a general-purpose Unix/Linux environment into a constrained, reproducible, movable unit. A clean-break system with signed, self-contained artifacts, separate address spaces, explicit capabilities, cluster-native names, and no per-node application installation may eliminate or radically simplify:

- OCI filesystem images as the normal application-delivery unit;
- CRI/OCI container-runtime and low-level container-launch layers;
- per-workload namespace construction and much container-specific seccomp, Linux-capability, and LSM policy scaffolding;
- the attempt to remove ambient machine authority from a process that began in a conventional Unix environment;
- per-node application package management, configuration drift, and machine-centred deployment procedures; and
- host-oriented workload mobility assembled from process, filesystem, and network emulation.

These are hypotheses about the target architecture, not blanket claims that each mechanism is unnecessary in every CharlotteOS implementation. Resource accounting, for example, survives even if cgroups do not; memory protection survives even if Linux namespaces do not; policy enforcement survives even if it is expressed by capability possession rather than SELinux labels. The responsibility remains while its mechanism and location change.

17.2.3 The control plane may survive the execution stack

The likely result is not that CharlotteOS eliminates Kubernetes. It is that CharlotteOS explores absorbing cluster orchestration into the operating system while removing much of the machinery required to manufacture a controlled execution environment above Linux.

Kubernetes's strongest control-plane ideas may survive remarkably well: declarative desired state, reconciliation, scheduling from constraints, generation-aware updates, health-driven replacement, replicated control state, and separation between specification and observation. A CharlotteOS cluster executive may therefore look recognisably Kubernetes-like at the level of intent and convergence.

The execution path beneath that controller could, however, be much shorter. Instead of a desired workload becoming a Pod, an OCI image, a container runtime request, a low-level runtime invocation, a namespace/cgroup/LSM configuration, and finally Linux processes, a cluster assignment can become a verified Charlotte execution object loaded directly into a capability-constrained address space. Discovering that reconciliation and scheduling survive while Pods, OCI, CRI, and much container security scaffolding do not would identify which platform ideas are consequences of distributed computation and which are consequences of the substrate.

The price of the clean break is equally important: existing Linux software does not simply run. CharlotteOS exchanges compatibility for a smaller and more explicit execution contract. Kubernetes asks how to orchestrate today's applications. CharlotteOS asks what applications and operating systems could look like if cluster execution, artifact trust, and explicit authority were native from the beginning.

17.3 Target architecture

17.3.1 The minimal node

The production target is an appliance-like node with no shell, no per-node application package manager, and no manually curated repository of installed software. Its stable local environment is just enough to boot, establish trust and identity, reach storage and the network, join cluster state, observe resources, and load assigned artifacts. Application differences belong in signed workload objects and replicated cluster policy rather than in mutable node configuration.

A target node lifecycle is:

1. boot a known CharlotteOS image and establish the node's execution and hardware trust boundary;
2. discover a cluster or contact a configured seed;
3. authenticate, learn cluster identity and policy, and join the relevant replicated state;
4. advertise resources and failure-domain properties;
5. receive assignments, fetch immutable artifacts, validate them, and instantiate only their delegated authority; and
6. on failure or retirement, surrender ownership under fencing while the cluster reconciles desired state elsewhere.

Secure Boot, measured boot, or remote attestation can strengthen the hardware-to-cluster trust chain, but they are complementary future concerns; artifact signing by itself does not attest the node.

17.3.2 The cluster executive

The target cluster executive holds desired software state and observes actual execution state. It knows accepted artifact identities, placement policies, node resources, assignments, active service generations, and enough health and communication data to converge the two. Its responsibilities include:

- admitting nodes and maintaining consensus membership;
- accepting authorised, generation-fenced desired-state changes;
- ensuring that referenced artifacts are available and valid;
- choosing placements from resource, trust, affinity, anti-affinity, and failure-domain constraints;
- realising replica counts and routing among active owners;
- observing failure and load and initiating replacement or movement;
- coordinating upgrades, rollback, and mutable-state ownership; and
- exposing cluster-wide status, accounting, logs, metrics, and administrative audit.

This is substantial orchestration work. The simplification sought by CharlotteOS is not a control plane with no hard problems, but a control plane that acts directly on native execution objects rather than indirectly on machine-shaped compatibility environments.

17.3.3 Placement, routing, and migration as native operations

The first placement controller can begin with declared rules. Desired replicas, maximum instances per node, minimum distinct nodes, every eligible node, affinity groups, anti-affinity groups, required resources, and

policy domains form a constraint problem over advertised node capacity. Admission must reject combinations the artifact did not authorise: in particular, concurrent replicas are invalid unless signed metadata blesses parallel instances.

Declared affinity is only an initial estimate. The runtime should aggregate actual message counts, byte flow, queue pressure, and executor load across the cluster. Sustained evidence can then influence placement, just as the per-node scheduler's sustained-window rebalancing replaces a static assignment with a load-observed one. Communication-aware placement remains subordinate to correctness constraints: co-location must not collapse required failure diversity, violate trust boundaries, or create ambiguous state ownership.

Routing is the indirection that makes placement change tolerable. A logical name resolves to an active generation and one or more owners. A local owner may be reached through a local endpoint and a remote owner through cluster transport without changing the caller's authority. When an owner is replaced, generation fencing prevents the retiring instance from deleting or overwriting its successor's registration.

Stateful migration must be explicit. The old instance exports only the mutable state it owns; the target accepts that state under a transfer protocol; ownership changes exactly once; the new endpoint becomes active; and stale clients are forced to re-resolve. Transparent movement must never mean copying mutable state without a consistency and fencing model.

17.4 Implemented foundation

The architectural target above is supported by the following implemented and verified mechanisms. Their existence makes the cluster vision concrete, but does not by itself constitute a complete cluster executive.

- **Cross-machine consensus.** The Graft core replicates the `dns` catalog state machine across two QEMU guests on a stream Ethernet segment and serves remote invocation through `dns::OP_CALL` (Chapter 14). The same replicated catalog carries the set-once cluster signing key and deployment assignments. Placement rules are not yet stored in that state machine.
- **Reliable cluster messaging.** Messages up to 64 KiB are fragmented across Ethernet frames and reassembled at the receiver (Chapter 13). Remote name-service calls, deployment coordination, and the demonstrated assignment handoff use this transport.
- **Persistent per-node object stores.** The NVMe-backed object store uses generation-selected directory records and copy-on-write replacement (Chapter 15) for service artifacts and persistent Raft state. The current image producer places the same signed bundle on each node; network fetch, repair, and cluster-wide artifact replication are not implemented.
- **Capability-scoped external data planes.** Separately provisioned S3 and Kafka services run above the userspace TCP/IP and verified-TLS path (Section 13.5). An S3 capability confines an application to one managed endpoint, bucket, prefix, credential identity, and rights mask. A Kafka capability confines it to one broker, fixed consume partition, bounded produce-route allow-list, group, transactional identity, and operation mask while offering idempotent production, read-committed consumption, and transactional offset commits. These adapters make external infrastructure usable from Charlotte; they are not yet cluster-managed storage or messaging substrates.
- **Local stateful replacement.** Live service upgrade transfers a service's state as a memory object, tracks generations, invalidates stale connections, and re-registers the replacement through the name service (Chapter 16). This has not yet been extended to mutable state crossing the network.
- **Service isolation and routing.** Services execute in separate address spaces with delegated capabilities (Chapter 11). The name service is replicated across nodes and provides the routing indirection for a service somewhere in the cluster.

- **Certified work migration.** The scheduler migrates threads between LPs with generation-validated handoff and sustained-window load rebalancing (Chapter 10). This is a per-node precedent for, rather than an implementation of, cluster rebalancing.
- **Observation substrate.** The sitas executor exposes task counts, poll counts, queue depth, and shard snapshots (Chapter 9). Shard-to-shard message flow can be measured. Cluster-wide collection and use of those measurements by a placement controller remain future work.
- **Signed binaries at the loading boundary.** The EL0 loader validates ELF structure, rejects writable-executable segments, maps images with page-level permissions, and requires an Ed25519 CLS2 note bound to the requested service name (Chapter 15 and Chapter 16). Store lookup, administration ingress, and deployment recheck that identity.

17.5 Current bootstrap and node model

The implemented kernel embeds five signed bootstrap services: `ns`, `nvme`, `objstore`, `uart`, and `observe`. They establish naming, reach persistent storage, and provide the early console and observation paths. Other staged services are loaded on first use from the signed bundle in the initial NVMe object-store image and cached in kernel memory. Service binaries therefore no longer generally need to be linked into the kernel image.

This is a concrete step toward the minimal node, but the current `-deploy-test` serial command loop is a kernel-side test and administration shim, not a production management plane.

17.5.1 Discovery and membership

The current two-node DNS test starts a statically described pair of voters. A separate boot test demonstrates discovery-bridged admission to a generic Raft group through replicated `JOIN`, joint consensus, and `FINALIZE`. Integrating that admission path with the durable DNS/shared-name-service identity remains future work.

A production node joining shared cluster state must complete three distinct operations:

1. **Discover the cluster.** The node uses broadcast discovery or a configured seed as discussed for Raft peers in Chapter 14. Discovery exists as a service, supplies MAC routes and cluster posture, and locates the anchor in the generic admission test. Discovery alone does not grant voting authority.
2. **Receive cluster identity.** The replicated catalog can distribute the set-once public signing key. Today that key must equal the build-time bootstrap anchor. Establishing trust from a genuinely blank start and rotating keys are not implemented.
3. **Join shared state.** The node becomes a member of the replicated state machine and begins receiving assignments. Test agents consume explicit assignments now, but no automatic placement controller produces them.

The target may allow a node to discover peers and participate in a restricted bootstrap before cluster key material is established, but it cannot validate executable artifacts without an accepted public key. An authorised administrative operation would establish or rotate that trust through replicated state. The current prototype instead uses a build-time anchor and a set-once, idempotent key ceremony.

17.6 Software ingress and deployment

17.6.1 The target pipeline

CharlotteOS does not remove the software supply chain; it narrows and makes explicit the object that crosses into the cluster. The intended pipeline is:

1. build a self-contained workload from reviewed source and dependencies;
2. evaluate policy and retain provenance, SBOM, test, and attestation evidence;
3. bless and sign the immutable workload identity off-cluster;
4. make the signed artifact available to the cluster; and
5. change replicated desired state so the cluster validates, places, and runs the accepted version.

Nodes never receive the signing private key and do not fetch executable dependencies from the Internet during placement. Software distribution and machine configuration are therefore separated: the same immutable object can be placed on any eligible node without reconstructing a userspace installation there.

17.6.2 The implemented pipeline

The demonstrated deployment pipeline implements a narrower three-stage path:

1. **Blessing and signing.** Version-controlled policy admits an artifact and the host-side build and `cluster-sign` tooling sign it with the cluster key. CLS2 metadata binds the logical name, class, release, rollback counter, parallel-instance permission, and an optional SHA-256 link to retained SBOM/build-provenance evidence. Nodes receive only the public key.
2. **Validation.** Each node validates every loaded ELF against the public bootstrap anchor and, in the deployment path, the matching replicated key. The loader, object-store resolver, administration ingress, and agent enforce signature, logical-name, and digest checks. The present ceremony can distribute only the build-time anchor; it is not a general enrolment or rotation protocol.
3. **Assignment.** A Raft-committed record directs a named node to load a particular object id and immutable ELF digest. The node's agent fetches, validates, and runs that exact artifact. The bytes are currently fetched from a locally staged NVMe store, not through a cluster artifact protocol.

The replicated catalog currently records one active owner and one explicit node assignment per artifact. The policy contract can express *somewhere*, replicas, and *every eligible node*, but a controller does not yet realise those choices and the name service does not yet route among multiple active owners.

17.7 Placement: contract, observation, and movement

17.7.1 Declared policy: implemented contract

The shared `PlacementPolicy` type expresses desired replicas, maximum instances per node, minimum distinct nodes, *every eligible node*, an affinity group, and an anti-affinity group. A request for concurrent replicas is invalid unless the artifact's signed CLS2 metadata contains `PARALLEL_INSTANCES`. The type and its validation are implemented and host-tested.

This is a validation contract, not a scheduler. The policy is not yet stored in the replicated deployment manifest or consumed by a controller. The current state machine commits one explicit node assignment for each artifact.

17.7.2 Runtime observation: local substrate only

Declared relationships are guesses. Shard-level message counts, byte flow, queue depth, and task snapshots are already observable inside a node (Chapter 9), and the distributed name service can identify which components call which. The target controller should aggregate these signals across nodes and use sustained behaviour, not transient noise, to refine placement.

No cluster-wide metrics aggregation or feedback-driven placement loop is implemented today. The existing executor measurements are inputs from which that controller can later be built.

17.7.3 Demonstrated stateless handoff

The two-node test starts the signed `greet` artifact on one node, invokes it remotely, reassigns it to the other node, retires the old instance, and verifies that the logical name remains reachable. Generation fencing prevents the retiring owner from unregistering its replacement. This proves assignment handoff and routing continuity for fresh service state; it does not prove general load-driven migration.

17.7.4 Future stateful cross-node migration

Stateful migration must extend the local live-upgrade protocol from Chapter 16:

1. the cluster instructs the target node to obtain, validate, and prepare the component in a new address space;
2. the old instance exports explicitly owned mutable state and the cluster transfers that state to the prepared instance;
3. the target accepts ownership under a fencing protocol;
4. the new instance registers through the distributed name service, advancing the name generation;
5. clients resolving the name reach the new owner, while clients with stale connections observe `EndpointClosed` and re-resolve; and
6. after the new owner is committed and reachable, the old instance is retired and its resources reclaimed.

Generation tracking, endpoint invalidation, name-service re-registration, and local memory-object state transfer are verified primitives. Network transport and ownership fencing for mutable service state are not implemented. The demonstrated cluster handoff starts the same immutable artifact with fresh state.

17.8 The demonstrated two-node deployment slice

The deployment-test QEMU flow boot-tests a first end-to-end slice on two guests. A signed artifact is assigned to a named node through replicated state; the remote agent obtains and verifies it; the service is invoked across the network; and the assignment moves to the other node without losing the logical name.

Both guests complete with zero failed or pending self-tests. The slice is important evidence for the architecture, but its simulated administration, pre-staged artifacts, explicit assignments, and static DNS voter set keep it well short of the production target.

17.8.1 Replicated deployment manifest

The distributed name service's Raft catalog (Chapter 14) also holds a **deployment manifest**:

```
artifact -> {object_id, artifact_sha256, node_key, generation}.
```

The map is committed through the same log as name registrations. Deployment is therefore ordered, replicated, generation-fenced cluster state rather than per-node configuration. `OP_DEPLOY` submits a record and `OP_DEPLOY_QUERY` reads a replica's applied state.

Raft agreement on a deployment record is not caller authorisation and the record carries no independent artifact signature. The production mutation path still needs a separately delegated cluster-administrator capability.

17.8.2 Cross-node hosting and generation fencing

A service hosted on the follower registers through a **remote-register relay**, because only the leader may commit catalog entries. The leader returns the committed generation. The two-phase register/activate sequence and generation-fenced endpoint-death unregister are preserved across the relay, so a retiring host cannot clobber a replacement's registration.

Cross-node invocation uses the existing `OP_CALL` relay. A client on either node reaches the service at its current owner through the replicated name service rather than through a node-specific application address.

17.8.3 The deployment agent

Each node runs an `agent`. It polls the manifest for assignments to its node key, reads the artifact from the object store, verifies the assigned SHA-256 identity, and checks that the signature note covers both the bytes and the requested logical name. It then invokes a deployment-authority syscall.

The kernel snapshots and revalidates the exact ELF, maps it into a fresh address space, and starts it. The ELF registers its local endpoint; the agent publishes that endpoint in the distributed catalog and supervises the assignment. When the manifest assigns the artifact elsewhere, the agent retires the service and causes its address space to exit. Endpoint closure is observed by `dns`, whose generation check prevents a stale unregister.

The supervisor-designated agent authority is the one narrow kernel mechanism added for this slice: only that agent may start or retire a verified, name-bound ELF memory object. The desired placement and administration logic remains a userspace coordination concern.

17.8.4 `clusterctl` and the test console

`clusterctl` is the prototype outside administration interface. It wraps the DNS manifest operations and object store behind:

- `upload`, which stores a note-signed artifact under its derived cluster-wide id after checking that CLS2 blesses the requested name;
- `deploy`, which hashes the stored bytes and submits the pinned assignment; and
- `status`, which queries the committed record.

An interactive kernel serial console in the `-deploy-test` flow lets an operator issue these commands against the live cluster. The console is a kernel test thread, not an out-of-band production management plane, and the raw mutation endpoint does not yet enforce the intended separately delegated administrator capability.

17.8.5 Artifact naming

Artifact names are currently bare cluster-global names such as `greet`. The object-store id is derived from the name using a `0xfffe...` tag over FNV-1a, so each staged node stores an artifact under the same id and the node dimension occurs only in the deployment record.

The host image producer detects and rejects collisions in the complete bundle. Production should replace the truncated 48-bit name hash with a collision-resolving or fully content-addressed namespace.

17.8.6 Cryptographic boundary

Artifact validation uses Ed25519 signatures embedded in the ELF. The private cluster key remains off-cluster in the `tools/cluster-sign` host tool. The signature is stored in a standard ELF `SHT_NOTE` section named `.note.charlotte-sig`, with note type `"CLS2"`. Its descriptor contains the logical name, artifact class and flags, release, rollback counter, signing-key id, optional provenance digest, and the 64-byte signature.

The signature covers the SHA-256 digest of the complete file with only the signature field treated as zero, making the signed representation deterministic. The matching public key enters the prototype in two ways: it is embedded in the kernel at build time as the bootstrap trust anchor and passed to services in the launch manifest; and the key ceremony commits it to replicated state through `OP_KEYCEREMONY` and `OP_SET_KEY`. This demonstrates a distribution path that a future dynamically joining node can use. It does not demonstrate blank-start trust: the replicated value must equal the build-time anchor.

The EL0 loader verifies the note before mapping a domain. Signing is mandatory: unsigned or invalidly signed images are rejected. The live-upgrade syscall reports failure gracefully; other current load paths panic. By default the build signs staged services using the version-controlled development keypair at `tools/cluster-sign/dev-key.hex`, whose public half is the embedded `CLUSTER_PUBLIC_KEY`. A different key can be supplied through `CLUSTER_SIGN_PRIVATE_KEY`, with a kernel rebuilt to embed its public half. A production key would be held outside the repository under organisational key-management procedures.

Store resolution and deployment additionally check the signed logical name, and the manifest pins the full ELF SHA-256. The demonstrated `greet` service is therefore a real, signed binary fetched from the local object store, mapped, executed, served across the network, and reassigned. Raft orders the assignment, but consensus is not an authorisation proof for the administrator who requested it.

17.8.7 What the slice proves

Within its stated limits, the slice proves that:

- an explicit node assignment can be represented as replicated, generation-fenced cluster state;
- a signed, name-bound ELF can be obtained, independently revalidated, and launched by a node agent;
- a service can be hosted away from the Raft leader and remain reachable through the distributed name service;
- generation fencing protects a stateless reassignment from a stale owner's unregister; and
- deployment and administration have protocol shapes rather than requiring direct machine configuration.

It does not prove automatic placement, multi-owner routing, stateful cross-node migration, shared artifact availability, dynamic reconfiguration of the durable DNS group, production key establishment or rotation, or production administrative authorisation. The narrower generic-Raft admission path is separately demonstrated.

17.9 Remaining implementation boundary

The gap between the demonstrated slice and the target cluster operating system is not a small amount of polish. It contains the following substantial coordination and trust work:

- **Dynamic membership hardening and integration.** Generic discovery-bridged admission performs replicated `JOIN`, joint consensus, and `FINALIZE`. It must be integrated with the durable DNS/shared-state identity. Partition behaviour, restart within a live quorum, and three-voter operation still need validation.
- **Trust establishment and key rotation.** The present key ceremony is set-once, idempotent, and limited to the build-time anchor. Authenticated blank-start establishment, overlapping rotation, revocation, and recovery remain.
- **Shared artifact availability.** Manifest records are replicated, but artifact bytes are pre-staged independently on each NVMe volume. The implemented S3 adapter is not wired into deployment. Network fetch, integrity-checked repair, retention, garbage collection, credential handling, and replication policy remain.
- **Cluster-wide observation.** Per-executor metrics must be collected and interpreted across nodes without turning transient telemetry into unstable placement decisions.
- **The placement controller.** No component yet converts desired replica policy, node capacity, trust boundaries, affinity, anti-affinity, failure domains, health, and observed communication into assignments or migration decisions.
- **Replica-set realisation and routing.** The runtime needs multiple active placements, failure-domain enforcement, health-aware selection among owners, and convergence when replica count changes.
- **Stateful cross-node migration.** The stateless handoff must be extended with network state transfer, explicit ownership acceptance, fencing, failure recovery, and a consistency model.
- **Production administration authority.** The userspace `clusterctl` protocol exists, but the raw mutation endpoint and kernel serial shim do not enforce the intended separately delegated cluster-administrator capability. Authentication, authorisation, audit, and safe concurrent administration remain.

- **Operational cluster services.** Production health management, upgrades, resource accounting, persistent storage policy, networking policy, secrets, logging, metrics, tracing, backup, and disaster recovery remain necessary even where their mechanisms differ from Kubernetes/OpenShift.

Most of this belongs in userspace coordination over the existing capability, messaging, storage, and consensus primitives. Future work must retain the discipline of the current mechanisms: explicit ownership, bounded communication, immutable artifact identity, generation fencing, and replicated decisions instead of per-machine mutation.

17.10 Related work

The vision combines ideas from distributed operating systems, capability systems, cluster schedulers, immutable software stores, and artifact-trust systems. The comparisons below identify intellectual neighbours; they are not claims of equivalent guarantees or implementation maturity.

17.10.1 Historical distributed operating systems

- **Amoeba** (Vrije Universiteit Amsterdam) is a close ancestor of the processor-pool model. Work can run on processors in a pool, dedicated machines provide file and directory services, and network-wide RPC uses capability-based access. Amoeba deliberately did not migrate processes; explicit cross-node migration is an extension in the CharlotteOS target.
- **Plan 9** (Bell Labs) separates terminals, CPU servers, and file servers, and centralises authentication and key handling in `factotum`. Its machine roles and network-transparent names challenge the assumption that every node is an independent Unix installation.
- **Inferno** continues the Plan 9 line with a portable virtual machine and code distributed as data over the network.
- **Jini and JavaSpaces** contribute a discovery pattern in which services locate a lookup service, register, and provide clients with a proxy. JavaSpaces' tuple-space model resembles work or software being placed in shared space for an eligible participant to take.
- **MOSIX** explores a single-system-image cluster with resource discovery and transparent process migration. Its work on communication-aware placement, including Keren and Barak, *Opportunity Cost Algorithms for Reduction of I/O and Interprocess Communication Overhead in a Computing Cluster*, IEEE TPDS 2003, is a precedent for using interoperation cost in placement.

17.10.2 Cluster control planes and placement

- **Borg, Omega, and Kubernetes** established the modern cluster-management shape: desired software is submitted to a pool of machines, scheduling is a cluster decision made from constraints and packing, and controllers reconcile observation with desired state. Burns, Grant, Oppenheimer, Brewer, and Wilkes describe this lineage in *Borg, Omega, and Kubernetes: Lessons from Three Decades of Platform-as-a-Service*, ACM Queue 2016.
- **Kubernetes and OpenShift** are also the central counterpoint of this chapter. CharlotteOS is not attempting to wish away their distributed control responsibilities; it is testing whether a native execution and authority model can substantially shorten the machinery underneath them.
- **HashiCorp Nomad and Consul** pair cluster scheduling with service discovery, membership, and affinity or anti-affinity constraints.

- **VMware DRS and vMotion** demonstrate the full operational arc of declared placement rules, load-observed rebalancing, live movement, and network-level switchover, though for virtual machines rather than Charlotte execution objects.
- **Erlang/OTP distribution** provides node discovery, named processes, supervision, hot code loading, and patterns resembling load, register, redirect, and retire.

17.10.3 Capabilities and constrained execution

CharlotteOS also belongs in the lineage of capability-oriented and least-authority systems. The relevant distinction is between possessing an explicit reference that confers authority and inheriting a broad machine environment whose authority must be denied by policy. Capability systems, microkernels, language sandboxes, WebAssembly runtimes, and unikernels make different trade-offs along this boundary. CharlotteOS combines a capability-constrained service model with signed native artifacts and a cluster-level placement objective; similarity in one dimension does not imply identical isolation or compatibility guarantees in the others.

17.10.4 Trust, signing, and bootstrap

- **The Update Framework (TUF)** defines signed, versioned metadata and key roles for secure update systems. Its separation of root trust, online roles, expiration, and rotation is relevant to the blank-start and key-lifecycle work not yet implemented.
- **Uptane** applies secure-update principles in the automotive setting and illustrates offline and provisioned-key patterns.
- **Sigstore/Cosign and Notary** provide related approaches to signing, identity, provenance, and policy for deployable artifacts.
- **Measured boot and remote attestation**, including TPM-, ARM CCA-, and confidential-VM-based mechanisms, can complement artifact validation by establishing properties of the node on which the artifact will run.

17.10.5 Membership and artifact stores

- **SWIM** (Gupta, Birman, and van Renesse, DSN 2002) and implementations such as **memberlist** provide scalable membership and failure-detection techniques relevant to discovery. Failure detection and Raft voting admission remain distinct responsibilities.
- **Nix** demonstrates immutable, hash-derived software-store paths and reproducible dependency closure, closely related to the goal that software be an identified object rather than a mutation of a node.
- **OCI registries with signed manifests** demonstrate content-addressed distribution and integrity for today's container delivery model. A CharlotteOS artifact service needs analogous availability and lifecycle properties even if it does not store OCI filesystem images.

17.11 Research thesis

The cluster vision can be summarised as the following proposition:

> Containers are partly an artifact of retrofitting isolation, reproducibility, software distribution, and workload mobility onto general-purpose operating systems whose primary administrative boundary remains the machine. If immutable artifact identity, capability-based authority, constrained execution, cluster-wide naming, and placement were native operating-system abstractions, much of the contemporary container execution stack could become unnecessary, while the essential distributed control plane would remain.

CharlotteOS is an experimental vehicle for testing that proposition. The implemented two-node slice establishes several necessary mechanisms: signed and name-bound loading, capability-separated services, replicated naming and assignment state, remote invocation, and generation-safe stateless handoff. It does not yet establish the full proposition. The decisive evidence will come from building the missing controller, artifact, membership, replica, state-transfer, administration, and operations layers and observing what complexity genuinely disappears.

The likely endpoint is neither a conventional per-node operating system nor the absence of orchestration. It is a cluster operating system whose reconciliation and scheduling ideas may resemble a reduced Kubernetes control plane, but whose execution objects are native, signed, capability-constrained workloads rather than containers that reconstruct and then confine a Unix machine. At that boundary, the cluster stops looking like a collection of computers on which software is installed and starts looking like the computer itself.

18 Generating Charlotte Applications

The server-class cluster vision (Chapter 17) asks what an operating system might look like if signed execution objects, explicit capabilities, and cluster-level placement were native mechanisms rather than layers constructed above general-purpose Linux. That direction creates an immediate application problem: a deliberately small, non-POSIX userspace does not run the existing body of server software, and asking ordinary application developers to program directly against a new kernel interface would make the platform impractical.

This chapter develops a proposed answer. A process-oriented generator such as Durga (https://github.com/FrodeRanders/durga) can act as the abstraction and compatibility layer between application intent and the Charlotte execution environment. Developers describe a process in BPMN and implement its business rules against a narrow generated contract; a Charlotte-specific backend generates the executable wrappers, capability requirements, communication structure, and cluster desired state. Familiarity then belongs primarily to the development tools and domain model, not to the operating-system ABI.

The current boundary is important. Durga reads BPMN and generates Kafka-oriented Java or Rust applications. Its generated targets share process-event and topic conventions, and its generated workers hide substantial messaging and lifecycle machinery. It does not generate Charlotte binaries, Charlotte capabilities, or Charlotte deployment records. CharlotteOS, in turn, implements the signed-ELF, name-service, deployment, observation, and stateless assignment mechanisms described in Chapter 17, but it does not implement the Durga integration proposed here. Except where the current Durga or Charlotte mechanisms are stated explicitly, the remainder of this chapter is an architectural target and a programme of experiments.

18.1 The application thesis

The conventional response to a new operating system is to reproduce a familiar application interface. POSIX compatibility, a C library, sockets, a filesystem hierarchy, dynamic linking, language runtimes, and ports of common packages provide a path for existing programs. This is effective compatibility, but it also tends to recreate the execution assumptions from which Charlotte is trying to depart: ambient access to a machine-like namespace, software assembled from mutable packages, and a process API designed around one host.

The alternative considered here is compatibility at a higher level:

```
business process + business rules
                 |
                 v
              generator
                 |
                 v
     native Charlotte application
```

This does not make compatibility free. Libraries, data formats, external-system connectors, diagnostics, testing, and developer tools still have to exist. The claim is narrower: a useful class of process-oriented business applications may not require developers to see the operating-system interface at all. The generator and its runtime support can be the only code that understands such details as message endpoints, capability handles, service registration, memory objects, lifecycle events, supervision, and deployment metadata.

The resulting thesis complements Chapter 17:

> A constrained execution environment need not imply a difficult application environment when infrastructure code is generated from a higher-level model. CharlotteOS need not expose a familiar operating-system API if familiar tools can generate Charlotte-native applications.

This is not an argument that every application should be expressed in BPMN, or that all existing software can be regenerated. It identifies a candidate application class in which the process graph is already an important source of truth and in which infrastructure is sufficiently regular to generate.

18.2 Durga's present application model

Durga is presently a BPMN-driven Kafka scaffolding and monitoring system. The BPMN model is executable input to a generator rather than documentation added after implementation. Durga derives workers, gateways, orchestration, topic setup, process lifecycle events, and monitoring structure from that model. The application developer completes or supplies the task-specific business behaviour while generated code implements much of the surrounding protocol.

Two current backends demonstrate that the process description need not be tied to one implementation language:

- The Java target generates a Quarkus application using SmallRye Reactive Messaging. Tasks, gateways, and orchestration become CDI components wired to Kafka channels, with shared runtime support.
- The Rust target generates a Cargo crate. Tasks and gateways become binaries under `src/bin`, with common code in a generated library and shared Durga runtime support.

Both targets use the same process-event wire model and topic layout. A monitor can therefore observe either target, and Java and Rust components can interoperate through the same generated protocol. This is implemented Durga behaviour. It is also evidence for an architectural separation: the semantics of a process can remain stable while code generation changes the language and runtime used to realise it.

The current generated deployment remains conventional. A generated application can include a JAR or Rust binaries, Kafka configuration and topic provisioning, a container build, a Kubernetes Deployment and Service, health probes, resource limits, environment configuration, and deployment scripts. Scaling combines application replicas with Kafka consumer groups and topic partitions. In an OpenShift environment, image policy, registry access, security context, network policy, routes, secrets, monitoring integration, and organisational admission policy normally add further deployment concerns.

These mechanisms are not accidental overhead. They supply real isolation, durability, routing, scaling, recovery, and operational control for conventional applications on general-purpose hosts. They do, however, illustrate how far the deployed representation has moved from the original business process.

18.3 Three distinct languages

The proposed architecture is easier to reason about if three layers are kept separate.

18.3.1 The business and process language

The upper layer contains BPMN and developer-owned business rules. It describes activities, gateways, events, participants, message flows, data objects, timers, failures, and process completion. Handwritten code supplies domain behaviour such as validating an invoice, calculating an entitlement, or selecting a payment route.

This layer should remain largely independent of Charlotte. A developer should understand the business domain, the supported BPMN subset, and the Durga task contract. Charlotte-specific message buffers, deployment calls, signing notes, and capability handles should not leak into ordinary rule code unless the application deliberately requests a platform-specific facility.

18.3.2 The Durga semantic execution model

The middle layer gives execution meaning to the process. Its concepts include:

- process definitions and versioned process instances;
- activity entry, completion, failure, cancellation, and retry;
- gateways, branch selection, fan-out, joining, and correlation;
- messages, signals, timers, call activities, and subprocess scopes;
- typed or schema-identified inputs and outputs;
- state transitions and lifecycle observations; and
- delivery, replay, idempotency, and failure semantics.

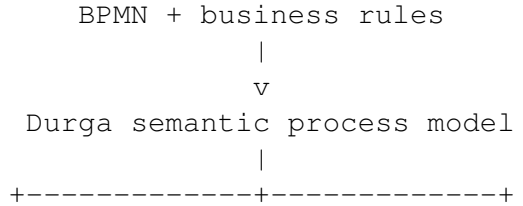
Some of these semantics are explicit in BPMN. Others are Durga runtime choices or policies that must be made explicit before an alternative backend can claim equivalence. For example, a sequence flow says that one activity follows another, but does not by itself settle whether delivery is durable, whether an activity may run more than once, where an acknowledgement occurs, or how a partially completed external side effect is recovered.

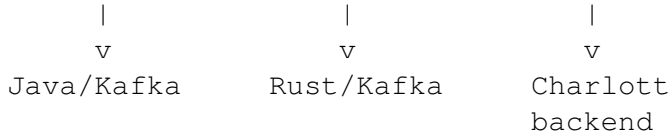
The semantic model must therefore be a real intermediate representation rather than a renaming of Kafka topics. A Charlotte backend should consume concepts such as activity, event, correlation, retry, and durable boundary. It should not have to reverse-engineer those concepts from a Kafka-specific topology.

18.3.3 The execution substrate

The lower layer realises the semantic model. Current Durga backends use Java or Rust workers and Kafka communication. A proposed backend would use Charlotte execution objects, capabilities, names, messages, supervision, and cluster desired state, while retaining an external event log where its semantics are required.

The intended relationship is therefore:





Changing the lower layer must not silently change the process contract. Where the available substrate cannot implement a required semantic, the generator must either retain a suitable service such as Kafka or reject the build. A backend is not correct merely because it can move bytes from one activity to the next.

18.4 A proposed Charlotte backend

The central proposal is a third Durga target, here called `charlotte`. It would transform the Durga semantic model and developer-owned rule code into Charlotte-native artifacts plus a deployment contract. This backend does not currently exist.

18.4.1 Generated activity boundary

For a simple activity, the developer-owned interface might be no broader than:

```
fn calculate_entitlement(
    claim: Claim
) -> Result<Decision, RuleError>
```

The exact Rust interface is a design question, not an implemented ABI. Around that function, the Charlotte backend could generate a wrapper that:

1. receives a typed activity invocation through a delegated endpoint;
2. validates the process, activity, instance, schema, and correlation identities carried by the invocation;
3. deserialises input into the developer-facing type;
4. emits an activity-entered observation;
5. invokes the business rule within its assigned protection domain;
6. converts success or failure into the Durga execution result;
7. serialises the result and sends it only through generated output capabilities;
8. emits completion, failure, and resource observations; and
9. participates in generated supervision, shutdown, and upgrade protocols.

The generated wrapper, not the business rule, would know how to receive a Charlotte message, resolve or accept a capability, allocate a shared memory object, re-register after replacement, respond to endpoint closure, and attach runtime telemetry. The developer-facing contract could remain suitable for unit tests on an ordinary development host.

This boundary must be enforced in the source layout as well as described in documentation. Generated files should be replaceable on regeneration, while developer-owned business modules remain stable. Escape hatches may be needed for specialised services, but a raw Charlotte handle passed into every rule would defeat the abstraction.

18.4.2 A deliberately narrow runtime surface

The generated runtime would target a small Charlotte service interface rather than POSIX. A candidate surface might include:

- receive from and send to delegated message endpoints;
- allocate, transfer, and release bounded memory objects;
- access a delegated clock, random source, or cryptographic service;
- invoke a named service through an explicit capability;
- emit structured observations;
- checkpoint explicitly owned state; and
- cooperate with supervision, replacement, and cancellation.

This list is proposed. The experiment should discover the minimum viable surface rather than declare it in advance. In particular, external protocols, TLS, databases, object storage, and durable logs will require trusted connectors or Charlotte-native services. Code generation cannot make those integrations disappear; it can make authority to use them explicit and can keep their mechanics outside the business rule.

18.5 From process graph to capability graph

Charlotte workloads begin with no ambient authority and receive only delegated capabilities. Durga has information from which a first capability request can be generated: which activity receives which input, which activity can send to which successor, which participant communicates with which other participant, and which declared data or external service an activity uses.

For a simple process fragment:

```
ValidateClaim -> CalculateEntitlement -> IssueDecision
                           |
                           +------------> RulesService
```

the backend could derive a graph in which:

- `ValidateClaim` may send validated claims to `CalculateEntitlement`;
- `CalculateEntitlement` may invoke `RulesService` and send decisions to `IssueDecision`;
- `IssueDecision` receives decisions but cannot invoke the rules service; and
- no activity receives a generic network, filesystem, or cluster administration capability.

The process communication graph can thus become an input to the Charlotte capability graph. BPMN message flows, pools, participants, and explicitly annotated external interactions are particularly useful because they already express architectural boundaries. A message between a claims pool and a payments pool may justify an explicit cross-domain endpoint rather than ambient reachability between their executables.

Absence of a BPMN edge is useful negative information, but it is not sufficient proof that communication must be forbidden. Gateways and runtime orchestration may introduce control paths that are implicit in the diagram; monitoring and administration require separate channels; and business-rule code may depend on an external system not represented in the model. Capability inference must therefore combine several sources:

- the supported BPMN topology and its scopes;
- typed task contracts and data declarations;
- explicit annotations for external services and durable resources;
- generator-known runtime and observation requirements;
- optional static analysis of the developer-owned code; and
- organisation policy that may narrow, approve, or reject the inferred request.

The generator should produce a reviewable request, not silently grant its own authority. A build or admission policy then decides which requested capability bindings are acceptable. This retains the distinction from Chapter 17 between an artifact's signed claims and the cluster authority that admits and delegates them.

18.5.1 Security graph and communication graph

The execution graph, communication graph, and security graph are related but not identical:

- The execution graph says which semantic transition may follow which other transition.
- The communication graph says which generated component must exchange which message or object with which other component.
- The security graph says which protection domain is authorised to perform that exchange at runtime.

One logical sequence flow need not create one network-visible endpoint, and two activities fused into the same executable may still be kept behind separate logical interfaces for observation or future decomposition. Conversely, a shared orchestrator may require communication not drawn as a direct business edge. Keeping the three graphs distinct prevents a convenient compilation choice from silently becoming the security policy.

18.6 The signed process bundle

The proposed backend would produce more than executable code. Its natural deployment unit is a signed process bundle that preserves the relationship between model, rules, generated wrappers, and cluster policy. A bundle could contain:

- the original BPMN model and a canonical digest of the model;
- the Durga semantic intermediate representation and its version;
- one or more Charlotte-native executable artifacts;
- immutable identities and signatures for every executable;
- input, output, event, and state schemas with stable identities;
- the execution, communication, and requested capability graphs;
- signer, build, generator, source, dependency, and toolchain provenance;
- an SBOM or the signed digest of retained SBOM and attestation evidence;
- resource estimates, concurrency declarations, replication policy, placement hints, and failure-domain constraints;
- durable-log and external-connector requirements;
- observation metadata mapping runtime identities back to BPMN activities; and
- the proposed cluster desired state for this process release.

Chapter 17 already implements signed, name-bound ELF artifacts with release, rollback, class, flags, and an optional provenance digest, and it pins the complete ELF digest in a deployment record. A process bundle would build on those mechanisms; it would not replace executable verification with trust in a container or archive. Each executable remains independently verifiable, while the signed bundle states that a specific set of artifact and schema identities collectively realises one version of a process.

Bundle signing is proposed future work. It also introduces supply-chain questions. The generator and build toolchain become security-sensitive because they derive authority and create wrappers. Reproducible generation, separation between requested and granted capabilities, independent policy review, retained provenance, and verification of the bundle-to-artifact links are necessary. Signing generated output does not prove that the BPMN model or generator was correct.

18.6.1 Generated desired state

The bundle could carry a declarative target such as:

```
process: claims
release: 42

component validate:
    artifact: sha256:...
    replicas: 4
    parallel: true

component calculate:
    artifact: sha256:...
    replicas: 8
    parallel: true
    requires: service:entitlement-rules

placement:
    affinity: [validate, calculate]
    minimum-failure-domains: 2

durability:
    ClaimSubmitted: external-event-log
    validate-to-calculate: native-message
```

The exact syntax is not proposed as a format. The significant point is that deployment intent is generated from the same semantic model as the executable wrappers. Cluster admission can validate the requested replica counts, parallel-safety declaration, artifact identities, capabilities, and placement constraints before committing desired state to the replicated manifest.

The existing Charlotte manifest accepts one explicit node assignment per artifact. The `PlacementPolicy` contract described in Chapter 17 can express replica and affinity properties but is not yet stored or realised by a placement controller. Consuming a Durga process bundle therefore depends on the future controller, replica routing, shared artifact availability, and production administration authority already identified there.

18.7 Process topology as placement information

A general-purpose cluster scheduler normally begins with deployable units, resource requests, and manually supplied affinity rules. It can later infer relationships from traffic and resource telemetry, but it does not ordinarily receive the application's semantic process graph.

Durga knows useful structure before execution begins:

- which activities are adjacent and expected to exchange data;
- where the graph fans out, joins, waits, or selects one branch;
- which activities may execute concurrently;
- which events enter from or leave for external systems;
- which subprocesses and participants form natural boundaries; and
- which activities are expected to block on timers, people, or durable external events.

A Charlotte backend can translate this into an initial placement hypothesis. Frequently adjacent activities may receive a common affinity group; replicas of one parallel-safe activity may receive anti-affinity or minimum-failure-domain rules; an activity using a restricted service may be constrained to the same trust domain; a high-volume data edge may favour co-location.

The BPMN topology is not a measurement. A rarely selected branch may carry very large objects, an apparently adjacent pair may spend almost all its time in an external service, and one process definition may serve workloads with very different data distributions. Resource estimates and expected frequencies may also be absent or wrong. The generated graph should therefore be the starting hypothesis, not permanent placement truth.

Charlotte's observation substrate can refine it. Runtime identities generated from the process model allow the cluster to measure message counts, bytes, latency, queue pressure, CPU time, memory pressure, failures, and retry rates per activity and edge. A future placement controller can compare declared structure with observed behaviour, then move or replicate components when sustained evidence justifies the change. Availability and trust constraints remain hard constraints; observed affinity must not collapse replicas into one failure domain.

This gives the process compiler and cluster different roles:

```
Durga model       -> initial graph and declared constraints
Charlotte runtime -> measured graph and resource evidence
placement policy  -> admissible movement and replication
controller        -> assignments in replicated cluster state
```

The feedback loop is proposed. Current Charlotte observation and assignment handoff mechanisms provide inputs and precedent, but they do not implement cluster-wide aggregation or automatic placement.

18.8 Logical activities and physical executables

BPMN describes logical units of work. It does not require one protection domain, one operating-system process, one container, or one independently scheduled artifact per activity. The current Rust generator's

activity and gateway binaries are a useful backend mapping, but a Charlotte backend need not preserve that mapping physically.

For a logical chain:

```
A -> B -> C
```

the backend might generate any of:

```
[A] -> [B] -> [C]
[A B] -> [C]
[A B C]
```

Fusion can remove serialisation, endpoint, scheduling, and network overhead. It may also permit intermediate values to remain in one address space. Separation supports independent scaling, fault containment, least authority, different trust domains, independent upgrades, and placement near distinct resources.

Executable granularity should therefore be an explicit compiler and policy decision. Relevant constraints include:

- capability and participant boundaries;
- durable handoff points and replay requirements;
- independent scaling and parallel-safety declarations;
- failure and retry domains;
- state ownership and upgrade boundaries;
- expected message size and frequency;
- architecture or hardware requirements; and
- the cost of losing a fused group when one activity fails.

The logical activity identity must survive fusion. Observations, failures, schema versions, and business audit events should still be attributable to the corresponding BPMN activity. The signed bundle should state the mapping from logical activities to physical artifacts, so that the cluster can explain its placement and so that a later release can split or fuse components without renaming the business process.

Automatic re-fusion from runtime evidence is a possible later research topic, not part of the initial proposal. Changing executable boundaries changes security, state, and upgrade semantics and should initially require a new generated and signed release rather than an opaque runtime rewrite.

18.9 Native messages and durable event logs

Kafka presently supplies more than byte transport to Durga. It offers durable logs, replay, partition ordering, consumer groups, offset tracking, integration with external producers and consumers, and transactional options. A Charlotte message endpoint must not be treated as equivalent merely because it can carry the same payload.

CharlotteOS now has an implemented, capability-scoped Kafka compatibility service (Section 13.5.2). A provisioned instance exposes one broker, a fixed consume partition, up to 64 allow-listed produce routes, a

group, transactional identity, and rights set. Its owned Rust client supports idempotent produce, bounded read-committed consume, explicit delivery acknowledgement, and a transaction that atomically combines Kafka output with the consumed delivery's group-offset advance. This supplies a concrete external event-log boundary for a future Durga backend; it does not yet connect Durga-generated process identities, schemas, or lifecycle events to that boundary.

The generated deployment should treat the broker connector and the behavioural role as separate authorities. Only `kafka` receives the immutable, digest-checked profile containing endpoint, trust, and future authentication secrets. A producer, consumer, or transactional-step runner receives a connection capability to that connector. The activity procedure behind a step receives only the bounded invocation capability and no Kafka authority.

The useful distinction is between application events whose meaning is intrinsically durable and internal execution plumbing whose purpose is to advance a running process.

An externally meaningful event such as `ClaimSubmitted` may need a retained, replayable, independently consumable record. Kafka or another durable event-log service may remain the correct substrate. An internal instruction such as "activity A completed; make B runnable" may need only the delivery, deduplication, and recovery guarantees of the Durga runtime and could use native Charlotte messaging.

A proposed Charlotte deployment may therefore be hybrid:

```
external producers
        |
        v
 durable event log
        |
        v
 [A] -> [B] -> [C]       native Charlotte messages
        |
        v
 durable business event
```

This keeps Kafka where log semantics and ecosystem integration are required, while avoiding a broker round trip for every internal transition. It also makes the durable boundary visible in the process bundle and capability request.

The present Kafka adapter is intentionally narrower than a general client. It uses a fixed direct partition assignment and a single broker, with no consumer group rebalance, broker failover, SASL, or compression. A generated deployment would therefore need either to accept that bounded profile, extend the adapter under equally explicit authority, or retain a conventional connector outside Charlotte for requirements the native service cannot satisfy.

The distinction cannot be inferred reliably from sequence flow alone. The process author or policy must declare durability, ordering, replay, retention, and delivery requirements. The Charlotte runtime must define how it recovers an in-flight process when a node fails, when an acknowledgement is lost, or when a business rule completes twice. If native messaging does not yet satisfy those requirements, the backend must retain Kafka for that edge. Removing Kafka from internal paths is an optimisation and architectural option, not a prerequisite for the Charlotte backend.

18.10 Preserving application intent

In a conventional deployment, semantic information is progressively lowered into infrastructure objects. A BPMN activity may eventually appear to the cluster as a container image, a Deployment, CPU and memory values, labels, and network endpoints. Those objects are operationally useful, but they retain little explanation of why two components communicate or what a latency measurement means to the business process.

Durga and Charlotte together could preserve an identity chain:

```
BPMN process and activity identity
                  |
                  v
Durga execution and schema identity
                  |
                  v
capability and communication edges
                  |
                  v
physical artifact and placement identity
                  |
                  v
runtime observations and process outcomes
```

Every generated message and observation can carry or imply stable process, release, instance, activity, edge, and schema identities. The signed bundle binds those logical identities to executable digests. The cluster desired state binds artifact identities to admitted capability and placement policy. Runtime telemetry can then be mapped back to the BPMN diagram.

This allows questions at several levels to share one evidence chain:

- Which business activity accounts for the current queue pressure?
- Which capability edge carried the slow calls?
- Which artifact release implemented that activity?
- On which nodes and trust domains were its instances placed?
- Did the observed traffic support the generated affinity hint?
- Did an executable fusion decision improve latency while preserving the intended fault and security boundaries?

Durga already emits lifecycle events and uses the BPMN model as the background for process monitoring. Charlotte already exposes executor observations and generation-aware names. Joining those identities and feeding the result into cluster placement is proposed future work.

Telemetry must not become ambient surveillance authority. Generated observation channels should be explicit capabilities; payloads should avoid business data unless required and authorised; and retention and access belong to policy. Preserving intent is valuable only if the observation path preserves the same security discipline as execution.

18.11 Comparison with Kubernetes and OpenShift

The comparison is not that a Charlotte backend makes orchestration disappear. It changes which layer owns particular responsibilities and which deployment objects need to be generated.

In the current conventional path, Durga may generate or depend on:

```
BPMN + rules
    -> Java/Rust application
    -> Kafka topics and configuration
    -> JAR or native executable
    -> container filesystem image
    -> image registry and image policy
    -> Kubernetes/OpenShift manifests
    -> Pods, Services, probes and NetworkPolicy
    -> container runtime
    -> Linux isolation and resource mechanisms
```

In the proposed Charlotte path:

```
BPMN + rules
    -> Durga semantic model
    -> signed process bundle
    -> Charlotte desired state
    -> Charlotte cluster
```

The shorter diagram does not mean the omitted concerns vanished. Their essential semantics are either absorbed by Charlotte, expressed in the bundle, or retained as an external service:

- An OCI filesystem image may become a signed, self-contained Charlotte execution artifact. Artifact storage, distribution, verification, and rollback remain necessary.
- A Kubernetes Deployment may become Charlotte replicated desired state plus replica and placement policy. Reconciliation, failure recovery, rollout, and admission remain necessary.
- A Service and service account may become a Charlotte name and explicitly delegated invocation capability. Naming, routing, identity, and key management remain necessary.
- NetworkPolicy may be replaced for generated internal communication by the admitted capability graph. Gateways to IP networks and external systems still require network security policy.
- Liveness and readiness endpoints may become native supervision and generated lifecycle state. The cluster must still distinguish work that is slow, blocked, failed, or safe to restart.
- Pod affinity may begin as a generated process communication graph and later be refined by Charlotte observation. Scheduling and failure-domain policy remain necessary.
- Charlotte may not need a container runtime or container-specific Linux policy. Namespaces, cgroups, and seccomp may also be absent because Charlotte starts with separate address spaces and delegated authority. Equivalent memory, CPU, and device accounting must nevertheless exist natively.
- Kafka topics used only for internal execution plumbing may become Charlotte message endpoints. Kafka remains where durable event-log and integration semantics are required.

OpenShift also supplies mature multi-tenancy, policy, secrets management, operators, ingress, storage integration, observability, upgrades, ecosystem compatibility, and operational practice. A Charlotte backend inherits none of that maturity by removing YAML or containers. The research question is which mechanisms can become smaller or more direct when the OS already has the desired trust and execution model, not whether the operational problems cease to exist.

Code generation is important in both paths. Kubernetes manifests, container builds, and policy can also be generated from Durga's model. The proposed advantage of Charlotte is not generation by itself; it is that generated intent can target native cluster and capability objects without first packaging a machine-like Linux userspace and then restricting it.

18.12 Developer experience and the compatibility boundary

If successful, the ordinary developer would work with:

- BPMN and domain-specific annotations;
- typed task inputs, outputs, errors, and business-rule interfaces;
- a host-side test harness for business behaviour;
- generated local simulations or integration tests;
- process-level diagnostics mapped to the BPMN diagram; and
- a build command that selects a verified backend.

The generator and platform team would own:

- the Charlotte ABI and generated wrappers;
- serialisation, schema evolution, correlation, and retry protocols;
- capability-request generation and policy integration;
- signing, provenance, and process-bundle construction;
- supervision, observation, deployment, and upgrade integration; and
- tested connectors to durable logs and external systems.

This division does not eliminate the need to learn Durga. It substitutes a domain and process programming model for an operating-system programming model. That is useful only if regeneration is predictable, generated code is inspectable, errors are reported in terms of model elements, and debugging can cross the boundary between rule code and runtime without exposing every kernel detail.

Backend portability also requires restraint in the task API. A business rule that assumes a local path, arbitrary outbound socket, process environment, or Java-specific runtime facility is not automatically portable. The developer contract should make portable operations easy, platform-specific dependencies explicit, and unsupported dependencies visible at build time.

18.13 A concrete first experiment

The first experiment should be deliberately narrower than a complete process runtime. Add a third Durga backend that generates one minimal Charlotte-native activity and the metadata required to admit and invoke it. The goal is to test the abstraction boundary, not to replace Kafka or Kubernetes in one step.

18.13.1 Experiment input

Use a small BPMN process containing:

- a start event;
- one service activity with a typed request and response;
- one successor or completion event; and
- an explicit annotation for any clock, log, or external service the activity requires.

The handwritten Rust module should implement one pure or nearly pure business function. It must not call a Charlotte syscall or manually construct an IPC message.

18.13.2 Generated output

The proposed `charlotte` backend should generate:

1. a Charlotte entry point and activity wrapper;
2. request and response schema identities plus serialisation code;
3. a minimal requested capability set for input, output, observation, and any explicitly declared service;
4. a communication edge and logical-to-physical identity map;
5. a self-contained ELF carrying the existing CLS2 signature note;
6. retained provenance or an SBOM/provenance digest compatible with the existing signed metadata;
7. a small process-bundle manifest binding the BPMN digest, activity identity, schema identities, and ELF digest; and
8. a desired-state record, or a translation step, for assignment through the existing Charlotte deployment test path.

The initial experiment need not implement automatic placement, replicas, fusion, native durable recovery, or a general bundle admission protocol. A single stateless activity assigned to one of the two demonstrated nodes is sufficient. Kafka may remain at the edge or be replaced by a host-side test driver for the first slice.

18.13.3 Execution and verification

The experiment should verify the following end-to-end properties:

1. Regenerating the backend does not modify the developer-owned business rule.
2. The generated ELF is signed, name-bound, digest-pinned, loaded in a fresh Charlotte address space, and invocable through the distributed name service.
3. The activity receives only the generated capabilities. An attempted undeclared communication fails because no authority is present, not because a network deny rule happens to match.
4. Typed input reaches the rule. Its typed result reaches the generated successor or test driver.
5. Activity-entered, completed, and failed observations retain the BPMN process and activity identities.
6. The existing stateless assignment handoff can move the generated artifact between nodes without changing its logical activity identity.
7. The process-bundle metadata can be independently checked against the BPMN, schemas, generated source, executable digest, and retained provenance.

The experiment should also record the amount and kind of Charlotte-specific code that remains handwritten. A successful first result is not merely that the activity runs. It is that the application developer implements the business rule without programming Charlotte, while the generated and trusted runtime surface remains small enough to inspect and verify.

18.13.4 Questions the experiment should answer

The slice is intended to expose design facts early:

- What is the minimum Charlotte ABI needed by a generated activity?
- Which capability requirements can be inferred, and which require an explicit annotation or policy decision?
- Which parts of Durga's current semantic contract accidentally depend on Kafka?
- Which schema and correlation identities must cross the wrapper boundary?
- How should a business failure differ from a wrapper, transport, or node failure?
- Can host-side tests and Charlotte execution use the same rule code without conditional platform logic?
- Which identities are required to map runtime telemetry back to the BPMN model and signed build?

Answers to these questions determine whether a larger backend is justified. Only after this slice should the work extend to multiple activities, gateways, durable boundaries, parallel replicas, fusion, automatic placement, and stateful recovery.

18.14 Remaining work

Substantial work remains in both projects:

- A substrate-neutral Durga semantic intermediate representation must distinguish process semantics from the present Kafka realisation.
- A stable generated-task contract, schema evolution rules, and explicit delivery and failure semantics are required before backends can be compared meaningfully.
- The Charlotte userspace ABI and runtime services needed by generated code must be identified, implemented, and kept deliberately narrow.
- A Charlotte backend must generate or request reviewable S3 and Kafka profiles without embedding credentials in developer-owned business code. Secret delivery, trust-anchor rotation, and profile replacement need a production provisioning path.
- Capability inference, review, admission, and binding need a format and policy model. The generator may request authority but must not grant it.
- Process-bundle format, signatures, artifact and schema identity, provenance, and reproducible generation remain to be designed.
- The Charlotte cluster still needs shared artifact fetching, production administration authority, replica realisation and routing, cluster-wide telemetry, and an automatic placement controller (Chapter 17).
- Native Charlotte messaging needs defined delivery, back-pressure, cancellation, deduplication, and recovery semantics before it can replace Kafka on any process edge that depends on them.
- Durable state, process-instance recovery, timers, human tasks, external side effects, and rolling process-version migration remain application-runtime problems even on a cluster OS.

- Debugging, local simulation, test fixtures, incident analysis, and operational explanations must be usable without requiring application teams to understand Charlotte internals.

None of these items follows automatically from code generation. The proposed integration is valuable precisely because it provides a concrete way to test them one layer at a time while preserving an honest distinction between application semantics and substrate mechanics.

18.15 Conclusion

Chapter 17 moves the administrative unit from the server to the cluster and asks which parts of the container stack are consequences of Linux compatibility rather than distributed computation. This chapter moves the same thesis into application development. If the deployable unit is generated from a semantic process model, Charlotte can offer a deliberately unfamiliar, capability-based execution environment without requiring every business developer to become an operating-system programmer.

Durga supplies a plausible compiler boundary because it already turns BPMN into Java and Rust execution structures with common process semantics and generated runtime plumbing. A future Charlotte backend could extend that transformation to signed artifacts, typed interfaces, capability and communication graphs, placement hints, provenance, and cluster desired state. BPMN would provide an initial account of application structure; Charlotte observation would test and refine that account at runtime; stable identities would keep the resulting evidence connected to business intent.

The proposal does not eliminate schedulers, durable logs, deployment policy, software supply chains, failure recovery, or observability. It relocates some of their expression from containers and infrastructure manifests into a process compiler and a cluster-native execution contract. The proposed third-backend experiment is the next falsifiable step: generate one minimal Charlotte-native activity and determine whether its developer can remain concerned with the business rule while the generated code accounts for the new operating environment.

19 Programming Model

This chapter summarizes the kernel and userspace rules for the asynchronous, shard-per-core model.

19.1 Invariants

The design relies on these rules:

19.1.1 Only the owning LP mutates its state

Per-LP data is accessed only from the LP that owns it. Cross-LP mutation goes through message dispatch (`ShardMailbox<M>`, IPI closure, or IPI RPC), never through a shared lock acquired from a foreign LP.

19.1.2 Messages are owned values

Everything that crosses a shard boundary is an owned value — a typed `M` in a `ShardSender<M>`, an IPI closure, or a memory object. No shared references, no `Arc<Mutex<...>>` as the normal programming model. `Send + 'static` boundaries enforce this at compile time.

19.1.3 References to shard-local state must not escape

`ShardLocal<T>::with(fn)` provides `&mut T` inside a closure. The closure must not store the reference, send it to another thread, or cross an `.await` point. The borrow is verified at runtime (owner-check plus re-entrancy flag).

19.1.4 Work stays on its assigned LP

A thread spawned on LP N stays on LP N. To submit work to another LP, use `try_run_on_lp(target, closure)` or `ShardSender::try_send`.

19.1.5 Observability returns owned snapshots

Never expose borrowed references into runtime internals. Diagnostic and introspection code captures state by value. Self-tests and the demo illustrate this pattern.

19.2 Kernel-side primitives

19.2.1 `PerLp<T>`

Lock-wrapped per-LP container. Use for data accessed from interrupt handlers, IPI handlers, or multiple LPs:

```
// Declaration:
static SCHEDULER: PerLp<RoundRobin> = PerLp::new();

// Access:
SCHEDULER.with(|scheduler| {
    scheduler.submit_ready_thread(tid);
});
```

The lock masks interrupts during the critical section on architectures that require it.

19.2.2 ShardLocal<T>

Lock-free per-LP container for single-LP, thread-only data:

```
// Declaration:
static MAILBOX: ShardLocal<ShardMailbox<Command>> = ShardLocal::new();

// Access (only from owning LP, not from interrupt context):
MAILBOX.with(|mb| {
    mb.try_send(Command::Ping)?;
});
```

Panics on wrong-LP access or re-entrant access.

19.2.3 ShardMailbox<M>

A typed, bounded per-LP queue has cloneable `ShardSender<M>` producers and one `ShardReceiver<M>` consumer per LP, accessed through `ShardLocal`:

```
let sender: ShardSender<Command> = mailbox.sender();
// Returns Err(data) if full.
sender.try_send(Command::Process { data })?;
```

19.2.4 IPI closures

`try_run_on_lp(target_lp, || { ... })`. Boxes arbitrary work and delivers it to the target LP's IPI queue. Returns the closure on full queue (backpressure). Use for kernel-level cross-LP work where typed mailboxes are not appropriate (e.g., TLB shootdown, scheduler commands).

19.2.5 Completion API

The API provides `submit`, `submit_worker`, `complete`, `poll`, `wait`, `cancel`, and `close`. The exit-observer path (`submit_worker`) is the intended default for fire-and-forget async kernel work:

```
let cap = submit_worker(|| {
    // Worker thread: do async work, then exit.
    // Completion fires automatically on exit.
});
let result = wait(cap)?; // Block until worker exits.
```

19.2.6 CQ ring

`CompletionQueueRing` for zero-syscall completion draining. Kernel side: `write(cap, result)`. Consumer side: `read() -> Option<CqEntry>`.

19.3 Userspace programming model

19.3.1 The launch contract

A CharlotteOS userspace program is a `no_std` ELF binary. It uses the `catten-rt` crate (the CRT, or C runtime) and the `entry!` macro:

```
#![no_std]
#![no_main]

use catten_rt::entry;

entry!(main);

fn main(ctx: Context) -> ! {
    // ctx provides:
    // - A connection to the name service
    // - The default CQ handle
    // - Heap configuration
    // - Bootstrap capabilities (device grants, etc.)

    let name_service = ctx.name_service();
    let nic_driver = name_service.lookup("driver.nic.v1")?;

    // ... service logic ...

    loop {
        // Poll tasks, drain CQ, wait for events.
    }
}
```

The program does not define `_start`, a panic handler, or an allocator. These are provided by `catten-rt`. The `entry!` macro wires them together, validates the launch ABI version, sets up the heap, and calls `main(ctx)`. A panic invokes the domain-abort syscall: every thread in the address space is retired, so no sibling can continue after shared Rust state or a userspace lock has been left inconsistent.

19.3.2 Resource ownership in Rust

The register-level syscall ABI deliberately exposes capability handles as integers. Application code should use the linear wrappers in `catten_rt::owned`:

- `OwnedMemory` closes its capability on drop and is neither `Copy` nor `Clone`.
- `MappedMemory<ReadOnly>` and `MappedMemory<Writable>` own both the mapping and capability; only the writable state can produce `&mut [u8]`.
- `DmaTransfer` is created only from unmapped `OwnedMemory` and returns CPU ownership after synchronous DMA unmapping.
- `ReadOperation<'a>` retains its `&'a mut [u8]` until completion, including the cancel-and-wait path in its destructor.
- `Completion` cancels, waits, and closes on drop; timeout does not consume the handle unless a terminal result was observed.
- `Endpoint`, `Connection`, and `PendingCall` close exactly once. A pending call retains any memory loan until reply observation or cancellation.
- `CapabilityVector` owns moved entries and retains read/write loans across a vector call. Submission failure returns the builder intact, including every moved object.
- `MmioRegion`, `MappedMmio`, and `Interrupt` provide single-close device ownership. Register access remains unsafe because width, alignment, volatility, and ordering are device-specific.
- `ThreadHandle` stores both the recyclable TID and the spawn-time generation. Joining passes both values back to the kernel, so a replacement thread cannot satisfy a delayed join.

The raw `catten-syscall` interface remains available to runtimes and drivers. Coherent `dma_map` is explicitly unsafe because the driver must synchronize CPU references and device access itself. Mixing raw operations with an adopted ownership wrapper violates the wrapper's safety contract.

These wrappers have host-side failure injection for unmap, DMA/MMIO release, completion cancellation, IPC move rollback, vector descriptor rollback, and loan cancellation. The no-std `charlotte-lifecycle` crate supplies the pure generation classifiers used by both the kernel and runtime and exercises their bounded input spaces in host tests.

19.3.3 Bootstrap authority

The `Context` carries the domain's bootstrap capabilities:

- `name_service()` — a connection to the name service, for looking up other services.
- `default_cq()` — the CQ handle for submission and waiting.
- `device_grants()` — any `MmioRegion` or `Interrupt` capabilities granted to this domain.
- `heap_config()` — memory available for the allocator.

The `Context` is the only way a userspace program acquires capabilities at startup. There is no ambient authority, no global filesystem, no `/dev` to explore.

19.3.4 Caller identity is kernel authority

Syscalls derive the caller's address space from trusted thread and trap context. Userspace cannot select another domain's address space or capability table. The signed loader also installs a stable artifact principal, exact address-space generation, and deployment roles in a kernel-only table. IPC enqueue snapshots that authentication data; the explicit authenticated receive ABI returns it alongside the numeric sender ASID, so a delayed message cannot inherit the identity of a later occupant of the same slot. The legacy nine-register receive syscalls remain unchanged and omit this extra envelope.

Security-sensitive discovery uses the name service's `OP_LOOKUP_AUTHORIZED` protocol. The request names a service and an explicit `SEND`/`CALL` rights set, but contains no caller identity. The co-located policy engine defaults to deny, evaluates the kernel-authenticated principal and exact generation, and delegates only an attenuated connection. `OP_SET_POLICY` and `OP_REGISTER_AUTHORIZED` separately require administrator and service-manager roles. Issuance, denial, and delegation failure enter a bounded administrator-readable audit stream. The legacy public and bearer-key name-service operations remain compatibility paths, not a principal authorization boundary; policy/audit persistence and retroactive revocation of already-issued direct connections are not currently promised.

19.3.5 Backpressure is normal

Bounded queues and capability tables return `WouldBlock`. The correct response is to retry, yield, or propagate backpressure upstream. Do not assume an unbounded queue.

19.3.6 Sitas integration

When writing code that uses sitas on CharlotteOS:

- A sitas shard is an LP-affine CharlotteOS user thread.
- `CharlotteReactor::new(lp_id)` binds a reactor to the calling shard's LP.
- Async operations return kernel-owned completion handles. A completion handle is opaque: wait on it, poll it, close it, or pass it through a typed channel.
- Cross-shard work is an owned message. Prefer sitas channels and futures over shared mutable state.
- The executor parks on `CQ_WAIT` (via `ShardParker`), not on a busy-loop.

19.4 Rule-of-thumb guidance

- **Choose the right container:** `PerLp<T>` for interrupt-accessed or cross-LP data; `ShardLocal<T>` for single-LP, thread-only data.
- **Choose the right channel:** `ShardMailbox<M>` for typed cross-LP communication within the kernel. `ShardSender/Receiver` for typed cross-shard communication within userspace. Endpoint IPC for cross-domain communication.
- **Use `submit_worker` for fire-and-forget async work.** When the worker returns, the exit-observer completes the capability.
- **Register a CQ ring at address-space creation** so completions are visible to userspace without per-entry syscalls.

- **Accept backpressure.** `WouldBlock` or `Err(m)` means the receiver is slow, not that the system is broken. Retry or propagate upstream.
- **Buffers have one owner.** While an async operation owns a buffer, userspace must not mutate or free it. Cancellation still ends in a terminal completion.
- **Never accept ASID or page-table roots from userspace.** Caller identity comes from the trap frame. This is the security invariant.
- **Do not smuggle shared kernel state into userspace.** Shared pages (CQ ring, config page) are data queues with narrow layouts, not references to kernel objects.

19.5 The syscall ABI

The userspace–kernel boundary exposes these operations:

`submit(op_code, buffer) -> OperationId` Start an async operation. Returns immediately. Buffer ownership transfers to the kernel.
`wait(cq_id) -> Vec<CompletionEntry>` Block until a completion arrives on the given CQ. Returns all pending entries.
`poll(op_id) -> Option<Completed>` Non-blocking check. Returns `None` while in flight.
`cancel(op_id) -> CancelState` Request cancellation. May complete asynchronously.
`close(op_id)` Release an operation ID after its terminal completion was observed.

For IPC:

```
endpoint_create(interface, capacity) -> EndpointCap
connection_mint(endpoint, rights) -> ConnectionCap
scalar_send(connection, opcode, arg)
scalar_call(connection, opcode, arg) -> ReplyToken
memory_send(connection, opcode, attachments)
reply(token, result)
receive(endpoint) -> Message
```

The full ABI is described in `docs/architecture/async-syscall-abi.md` and implemented in `crates/catten/src/syscall/mod.rs`.

19.6 Why the programming model matters for critical software

1. **Invariants have enforcement points.** `ShardLocal<T>` rejects wrong-LP and re-entrant access; `ShardSender<M>` moves values across the typed boundary. Unsafe code and incorrect primitive use remain review obligations.
2. **The launch contract is minimal.** A userspace program receives exactly what it needs and nothing more. There is no global state, no inherited file descriptors, no ambient filesystem. The bootstrap capabilities are the program's entire universe of authority.
3. **Messages, not shared memory, are the default.** The safe, preferred path is to send an owned value through a bounded channel. Shared memory with locks is the exception and not the rule. This reduces the number of places where concurrency bugs can hide.

4. **The ABI is auditable.** Userspace wrappers and the kernel dispatcher share the typed `SyscallNumber` enum, and dispatch is exhaustive. The source enum, rather than a duplicated count here, defines the current interface.
5. **The programming model is testable.** Self-tests exercise owner checks, re-entrancy guards, selected backpressure paths, and cancellation behavior. Coverage is substantial but not exhaustive.

20 Development Workflow

This chapter gives the current build, boot, test, and code-reading workflow.

20.1 Prerequisites

- **Rust nightly**, installed via `rustup`. The exact toolchain is pinned in the repository's `rust-toolchain.toml`; do not copy a version from this manual.
- **QEMU** for AArch64 (`qemu-system-aarch64`) or x86-64 (`qemu-system-x86_64`). On macOS, TCG is the default and HVF is opt-in. Networking and network-backed services are enabled by default under TCG; the virtio-net smoke-test switch adds verification. HVF is unsuitable for the driver's direct EL0 device-MMIO path and therefore requires `-no-network`. KVM is an optional Linux accelerator, not a networking requirement. **HVF also does not honor hardware ASIDs** — see the *AArch64 Port Status* write-up — so HVF builds rely on the `hvf_compat` whole-TLB-flush-on-switch fallback. Do not assume ASID-based TLB isolation works when debugging under `-hvf`. HVF also exposes no SMMU for protected NVMe DMA, so its compatibility boot runs the 15-test non-storage suite. Use TCG for the full 18-test suite, object-store tests, persistent Raft, and live upgrade.
- **Build dependencies:** `cargo`, `just` (optional, for convenience targets in the `Justfile`), and `mtools` for constructing x86-64 FAT boot images. VMware appliance packaging also requires `qemu-img`, normally installed with QEMU.

20.2 Build targets

CharlotteOS supports three architectures via custom target JSON specs in `target_specs`:

Target spec	Status
AArch64 custom target	Original development target; all features and self-tests pass
x86-64 custom target	Four-LP ring-3 service, storage, IOMMU, discovery, distributed DNS, smoltcp TCP/IP, HTTP, deployment, migration, UTC time, and dynamic-membership paths pass under QEMU TCG; UEFI, VT-d, NVMe, and node-local services also pass under VMware Fusion
RISC-V custom target	Planned, not actively maintained

20.2.1 Kernel build

The commands below use the AArch64 target specification and the x86-64 target specification.

```
# AArch64 (headless, serial console only):
cargo build --package catten \
  --target target_specs/aarch64-unknown-none-catten.json \
  --no-default-features --features acpi

# x86-64:
cargo build --package catten \
  --target target_specs/x86_64-unknown-none-catten.json \
  --no-default-features --features acpi
```

The `display` feature requires a C library for the flanterm framebuffer terminal. Headless boot uses serial output only (AArch64: PL011 UART; x86-64: 16550 COM1).

20.2.2 Userspace service programs

Service binaries are built separately because they target `no_std` / `no_main` with custom EL0 targets that depend on build-std:

```
# AArch64 EL0 service programs:
cargo build --release \
  --manifest-path crates/catten-services/Cargo.toml \
  --target crates/catten-services/aarch64-unknown-none.json \
  -Z build-std=core,alloc

# x86-64 ring-3 service programs:
cargo build --release \
  --manifest-path crates/catten-services/Cargo.toml \
  --target crates/catten-services/x86_64-unknown-none.json \
  -Z build-std=core,alloc

# Sitas user binary (requires external sitas crates):
cargo +nightly build --package catten-user \
  --target aarch64-unknown-none.json \
  -Zbuild-std=core,alloc
```

20.3 Booting in QEMU

20.3.1 AArch64

The boot entry points are `scripts/boot-smp1.sh`, `scripts/boot-smp2.sh`, and `scripts/run-aarch64.sh`.

Both architecture runners attach a NIC by default. Charlotte acquires a DHCP lease and launches discovery, cluster, TCP/IP, HTTP, and time services without a test option. The `*-test` switches only register additional verifiers; `-no-network` is the explicit opt-out for an isolated boot.

```
# Single-LP self-test boot:
./scripts/boot-smp1.sh

# Two-LP self-test boot:
./scripts/boot-smp2.sh

# Or choose the profile, LP count, and timeout directly:
./scripts/run-aarch64.sh debug --smp 4 --timeout 180

# Apple Silicon: accelerated 15-test non-storage compatibility suite:
./scripts/run-aarch64.sh debug --hvf --no-network --smp 4 --timeout 180
```

20.3.2 x86-64

The x86-64 runner is scripts/run-x86_64.sh; scripts/boot-x86.sh is its single-instance convenience wrapper. Both use mtools to construct the disposable FAT boot image, which avoids platform-specific mounted-image discovery on macOS.

```
# Requires -cpu max for RDTSCP/FSGSBASE support:
./scripts/boot-x86.sh

# Select storage/IOMMU and run the four-LP bounded suite directly:
./scripts/run-x86_64.sh debug --smp 4 --block nvme --iommu intel \
  --fresh-storage --timeout 90

# Serve and host-probe the HTTP state report through smoltcp:
./scripts/run-x86_64.sh release --http-test --timeout 90

# In two terminals, run the complete four-LP cluster suite on both guests:
./scripts/run-x86_64.sh release --deploy-test --smp 4 \
  --instance node-a --mac 52:54:00:12:34:01 \
  --net-listen 12002 --timeout 360
./scripts/run-x86_64.sh release --deploy-test --smp 4 \
  --instance node-b --mac 52:54:00:12:34:02 \
  --net-connect 127.0.0.1:12002 --timeout 360
```

Use `-tcpip-test` instead of `-deploy-test` for the smaller two-guest smoltcp exchange. `-live-upgrade-test` runs the isolated persistent service-manager handoff. The complete deployment suite is qualified with four LPs; two LPs are not a supported test configuration for that load.

20.4 Booting in VMware

The appliance builder uses the same x86-64 build path without launching QEMU, then converts the FAT boot image and a blank data image to VMware VMDKs:

```
./scripts/build-vmware-x86_64.sh release
```

Open `os-images/vmware/CharlotteOS.vmwarevm/CharlotteOS.vmx` in VMware Fusion or Workstation. The SATA boot disk is nonpersistent; the NVMe data disk is persistent. On first boot the guest formats that disk and installs the missing signed x86-64 service artifacts. Later boots retain valid artifacts, including deployed upgrades. The builder refuses to overwrite an existing appliance unless `-replace` is supplied. The default 1 GiB data disk requires the shared 4 MiB userspace heap so the object store can hold its allocation bitmap, directory, and mirrored-directory mount buffer.

The supplied VM attaches VMware's emulated Intel 82574L adapter to NAT. The userspace E1000E driver receives delegated MMIO, MSI-X, and protected-DMA authority, waits for link without occupying an LP, and publishes the same hardware-neutral `net0` service as virtio-net. Discovery, DNS, smoltcp, HTTP, and cluster code therefore require no VMware-specific branch. The E1000E path is qualified under Fusion, while paired VMware guests and VMXNET3 remain unqualified. The complete setup and qualification record is in the VMware x86-64 appliance guide.

20.5 Expected boot output

```
CharlotteOS booting...
[init] BSP initialization complete.
[init] Secondary CPUs brought online.
[init] Starting scheduler...
[async] coordinator: submitted worker, awaiting completion...
[async] worker on LP1: sleeping 50ms
[async] worker on LP1: exiting
[async] COMPLETION RECEIVED, result Ok(42)
[async] SUCCESS: async syscall round-trip complete
Testing Complete. All Tests Passed!
```

20.6 Self-tests

The repository combines host-side library tests with boot-time kernel and EL0 self-tests. Host suites cover components such as `catten-graft`, `charlotte-protocol-msg`, and `charlotte-smoltcp`. Boot tests run from `self_test::run_self_tests()` after initialization; assertion or verifier failure fails the boot suite.

Deferred tests register an expected result bit and spawn their verifier through `results::spawn_verifier`. The result subsystem associates that single verifier thread with the test in a fixed atomic table. If the verifier panics, the panic handler uses a best-effort non-blocking scheduler lookup and marks the associated test failed before emitting the panic. Consequently the authoritative reporter names the failed test instead of leaving it pending until the runner timeout. The panic path neither allocates nor waits for a scheduler lock.

Representative boot suites include:

- `completion.rs` — buffer ownership, cancel/deferred-reclaim, backpressure.
- `cq.rs` — CQ ring write/drain/overflow.
- `cq_completion.rs` — CQ ring integrated with completion subsystem.
- `syscall.rs` — dispatch table routing via synthetic TrapFrame.
- `ipi.rs` — bounded queue, backpressure, closure dispatch.

- shard.rs — ShardLocal<T> owner-check / re-entrancy guard, ShardMailbox<M> send/recv.
- el0.rs — real userspace thread (AArch64 EL0 and x86-64 ring 3).
- el0_demo.rs — EL0 userspace smoke test.
- el0_ipc.rs — EL0 endpoint IPC between domains.
- el0_net.rs — EL0 networking test.
- el0_pingpong.rs — bidirectional EL0 IPC.
- el0_raft.rs — two-node Raft election over local IPC.
- el0_service.rs — EL0 service lifecycle (spawn, IPC, teardown).
- el0_uart.rs — userspace UART driver (delegated MMIO + IRQ, crash recovery).
- el0_sitas.rs — sitas executor integration: sharded KV, the mailbox index demo, and the thread-join probe (Section 9.8).
- scheduler_lifecycle.rs — spawn, block, wake, abort, migration-safe migration, timer-wait affinity.
- ipc.rs — IPC path tests (scalar, memory, delegation, reply tokens).
- memory/allocator.rs — heap allocator, stack allocator.
- memory/object.rs — memory object lifecycle tests.
- adversarial.rs — cancellation races, misuse paths, error-path testing.

20.7 The async-syscall demo

crates/catten/src/demo.rs is an end-to-end test of the async model. It:

1. Spawns a coordinator and worker from bsp_main.
2. The coordinator calls submit_worker, which spawns a worker thread and returns a capability that fires when the worker exits.
3. The worker sleeps 50ms (exercising the timer and block-yield-wake cycle).
4. The worker exits; the exit-observer fires; the capability completes.
5. The coordinator (blocked on wait(cap)) wakes and reads the result.

This exercises thread creation, scheduling, timer events, the block–yield–wake cycle, exit observation, and the wait/poll ABI. It is evidence for that path, not a substitute for the wider self-test suite.

20.8 CI pipeline

GitHub Actions (.github/workflows/ci.yml):

1. Checks formatting for the workspace and standalone EL0/tool crates.
2. Runs warning-denied Clippy for the workspace, AArch64 services, and signing tool.
3. Runs host-testable library suites and the signing self-test.
4. Builds AArch64 and x86-64 kernels and the AArch64 service bundle.
5. On pushes to main, boots the four-LP AArch64 QEMU suite and verifies its required success markers.

The workflow runs on pushes and pull requests for its configured branches; the QEMU boot job is restricted to pushes to `main`.

20.9 Codebase reading guide

For new contributors, the recommended reading order:

1. `crates/catten/src/main.rs` — kernel entry point, init sequence.
2. `crates/catten/src/completion/mod.rs` — the completion subsystem: capability table, submit/complete/poll/wait/cancel/close, `submit_worker`, exit-observer.
3. `crates/catten/src/completion/cq.rs` — the CQ ring buffer.
4. `crates/catten/src/syscall/mod.rs` — syscall dispatch table.
5. `crates/catten/src/demo.rs` — the end-to-end async demo.
6. `crates/catten/src/cpu/scheduler` — threads, wakers, `RoundRobin`, `block_thread`.
7. `crates/catten/src/cpu/multiprocessor` — SMP: IPI, `PerLp<T>`, `ShardLocal<T>`, shard mailboxes.
8. `crates/catten/src/cpu/isa` — ISA abstraction layer. Start with `interface/` (traits), then `aarch64/` (reference implementation).
9. `crates/catten/src/service` — supervisor, ELF loader, bootstrap.
10. `crates/catten/src/device_management` — device driver framework.
11. `crates/catten-services/src/bin` — reference EL0 services.
12. `crates/catten-rt/src/lib.rs` — userspace runtime.

20.10 Troubleshooting

Triple fault during boot Check that the panic handler is working. Early faults before heap initialization rely on `early_logln!` (direct serial writes). Ensure the serial port is configured correctly.

Blocked thread The block–yield–wake cycle requires that `block_thread` not clear `current_handle` before `yield_lp` saves the thread's context. Verify that the blocked thread is still the current thread at the point of the yield.

ASID-0 collision The first user address space must not receive ASID 0 (which equals `KERNEL_ASID`). The address-space table must be pre-populated with the kernel AS at index 0.

x86-64 LA57 mismatch Derive the address width from `CR4.LA57` (the active paging mode), not from CPUID capability. Limine boots with 4-level paging even on CPUs that support 5-level.

21 Formal Safeguards with TLA+

CharlotteOS uses executable TLA+ models to review stateful functionality. The models state each safety contract, explore action orderings in a finite instance, retain known-bad designs as counterexamples, and map abstract transitions back to Rust.

This is stronger than relying on selected test schedules, but narrower than a proof of the complete operating system. TLC proves invariants of the modeled state machine for the configured finite bounds. The repository supplements that result with an action-level conformance map, Rust regression tests, target builds, Clippy, and boot tests. Each layer answers a different question:

TLA+ model Is the protocol design safe under every modeled action ordering, crash point, retry, and stale event within the configured bounds?
Conformance map Which concrete Rust state and linearization point implement each abstract action?
Rust and boot tests Does the implementation exercise the intended transition on the host and on CharlotteOS targets?
Build and lint gates Does the integrated implementation remain type-correct and warning-free for the actual target?

The models and their configurations live in `docs/tla`. The concise model inventory is in `docs/tla/README.md`; the concrete-to-abstract mapping is in `docs/tla/CONFORMANCE.md`. Those files are maintained with the implementation and are authoritative over the overview in this chapter.

21.1 Why model functionality as a state machine?

Most failures addressed by these models are not computation errors. They are ordering errors: a stale cleanup arrives after a replacement, a thread is removed while it can still execute, a session identifier is reused after a restart, or a joiner forgets which cluster has authority over it. A unit test can exercise one such interleaving. A state-machine model asks TLC to enumerate all reachable interleavings in a deliberately small world.

A TLA+ safety specification has four essential parts:

1. **Variables** record only the state relevant to the property, such as an owner, generation, durable term, queue, or admission anchor.
2. **Init** defines every allowed initial state.
3. **Next** is a disjunction of named actions. Each action defines its enabling condition and complete next-state update.
4. **Invariants** state what must be true in every reachable state.

The common shape is:

```
VARIABLES phase, generation, owner
vars == <<phase, generation, owner>>

Init == ...
```

```
Next == Start \/ Replace \/ Stop \/ Crash \/ Restart
Spec == Init /\ [][Next]_vars

TypeOK == ...
Safety == ...
Invariants == TypeOK /\ Safety
```

The square temporal operator means that every step must be a `Next` step or a stuttering step and that the invariant must hold throughout the behavior. TLC constructs the reachable state graph rather than sampling it. Constants in a `.cfg` file bound identities, queue capacities, generations, terms, and log indices so that this graph is finite.

Small bounds are intentional. Many protocol violations need only two owners, two generations, or three Raft nodes. Exhaustive exploration of that small instance is often more revealing than a large randomized run because TLC systematically covers the orderings rather than hoping to encounter them.

21.2 The safeguarding loop

CharlotteOS applies the models as a feedback loop between design, implementation, and validation:

1. **Choose a safety boundary.** Identify the authority or resource whose invalid transition would be harmful: reply authority, memory ownership, an executing address space, a committed log entry, or a service generation.
2. **Project concrete state.** Remove bytes, addresses, and policy details that cannot affect the selected invariant. Preserve identity, ownership, durability, ordering, and lifecycle phases.
3. **Name the transitions.** Actions use implementation language. Names such as `Receive`, `Reap`, and `Restart` make review against Rust possible.
4. **State the invariant positively.** Examples are "a reaped thread is off-CPU," "a committed entry agrees at every replica," and "a stale generation cannot remove its replacement."
5. **Model crashes and stale work.** Durable and volatile state are separated where restart matters. Delayed messages, old handles, and retries are actions rather than assumptions that the environment is well behaved.
6. **Retain a negative specification.** When a faulty transition is understood, an `Unsafe...` action and configuration are kept. The check suite requires TLC to find the named invariant violation. This proves that the invariant and test harness are capable of seeing the regression rather than passing vacuously.
7. **Map back to code.** `docs/tla/CONFORMANCE.md` records the concrete function, persisted field, and linearization point for every modeled action. Disagreement is resolved by changing the model, changing CharlotteOS, or documenting behavior outside the abstraction.
8. **Validate both levels.** Run TLC, focused Rust tests, target builds and Clippy, and relevant boot or multi-node tests.

The model is an executable safety contract, not a diagram added after implementation. A change is incomplete if the model claims an atomic or durable transition that the code does not provide.

21.3 What is modeled

The current suite contains twenty-one safe specifications. They are divided by state ownership and failure boundary so that each model remains reviewable. The Raft models are deliberately layered: election, log replication, membership, pre-membership admission, and snapshots have different state and different proof obligations.

Model	Functional boundary	Principal safety properties
`CharlotteIPC`	Endpoint calls, reply tokens, memory move/copy/borrow, cancellation, endpoint close, domain teardown	Bounded queues; unique receiver-visible reply authority; exclusive write borrow; every remaining borrow backed by a live token; no dangling moved-memory capability.
`CharlotteIpc Transaction`	Multi-entry vector preparation, connection attachment, rollback, delivery, reply and non-terminal timeout	Failed preparation restores every affected registry; attached connections cannot escape their transaction; timeout cannot release a live call's loan.
`Charlotte EndpointObservers`	Readiness observers and connection-close watches	Ordinary message arrival cannot falsely signal endpoint closure; closure wakes the appropriate observers.
`CharlotteCQ`	Completion operations, CQ ring/backlog, wait/wake, cancellation, timers	Non-lossy completion tracking; bounded visible ring; no lost wakeup; timer completion remains tied to its operation; a buffered operation retains its kernel loan through cancellation until terminal completion.
`CharlotteTimedWait`	Work-generation publication, delayed wake delivery, and watchdog expiry	A timer cannot report timeout after a new work generation has already been published.
`CharlotteScheduler`	Spawn, admission, execution, blocking, migration, cross-LP abort, reaping, address-space destruction	At most one execution per LP; only the owner LP retires an executing thread; reaping occurs off-CPU; domain abort prevents late sibling publication; destroyed address spaces have no live threads.
`CharlotteThreadJoin`	Recyclable TIDs, spawn generations, delayed exit-observer registration	A captured handle observes only its exact generation; a missing or recycled slot completes immediately; exhausted generation identity fails closed before wrap.
`Charlotte AddressSpace`	Reusable numeric ASIDs and software-generation handles	A stale handle cannot close a replacement that reused the same numeric ASID.
`Charlotte HardwareAsid`	Hardware-ASID allocation, retirement, cached translations, invalidation	A hardware tag is not reused while stale translations for its former owner remain.
`Charlotte InterruptRoute`	Interrupt binding, queued wake, unbinding, deferred drain	A queued wake cannot be delivered through a later route generation to the wrong owner.
`CharlotteCapability`	Unified per-address-space capability namespace and move rollback	Handles are fresh and authoritative; move revokes the source; rollback restores only the exact transaction; a closed namespace is empty.

Model	Functional boundary	Principal safety properties
`Charlotte Authorization`	Versioned subject/service policy, generation-fenced service bindings, decisions and capability issuance	Only authorized roles mutate policy or publication; decisions are bound to a subject, policy version, service generation and rights; redemption is single-use and cannot amplify authority.
`CharlotteDMA`	CPU mapping, IPC loans, coherent/exclusive SMMU mapping, invalidation, quarantine, driver exit, pin reclamation	Unique stream/domain ownership; exclusive DMA has no overlapping CPU authority, loan, or second pin; mapped memory stays pinned; failed hardware acknowledgement retains rather than prematurely frees authority.
`Charlotte ServiceLifecycle`	Signed artifact staging, loading, start, two-phase publication, lookup, fenced unregister, exit and teardown	Untrusted images never load; only activated generations resolve; stale activation/unregister cannot affect a replacement; teardown follows domain-wide reaping after cooperative exit or domain abort.
`CharlotteRaft`	Terms, durable votes, election, higher-term step-down, crash/restart	A voter records at most one durable vote per term and at most one leader can hold a term.
`CharlotteRaftLog`	Append, conflict repair, replication, commit, crash/restart	Log matching, committed-entry agreement, commit within the log, and leader completeness.
`Charlotte RaftMembership`	Voters/learners, joint consensus, finalization, decommissioning	Both voter majorities are required while joint; all proposed peers reach the joint fence before finalization; leadership and decommissioning follow active membership.
`CharlotteRaftJoin`	Selected-anchor admission before membership, JOIN fence, catch-up, crash/restart	A joiner accepts only the chosen anchor, cannot campaign, is promoted only after catch-up, and restores its durable admission fence after restart.
`Charlotte RaftSnapshot`	Chunk reception, durable installation, matching suffix, membership recovery	Stale snapshots cannot regress progress; the installed state and retained suffix agree; restart restores membership and application state before use.
`CharlotteRemoteCall`	Call identity, target generation, duplicate suppression, timeout uncertainty, result eviction	At most one execution while tracked; stale targets never execute; uncertain outcomes are not reported as success; deduplication evidence remains until transport settlement.
`Charlotte ReliableMessage`	Wire sessions across retry abandonment and unilateral service restart	Logical epochs have unique ordered wire identities and delayed sessions cannot move receive state backwards.

This decomposition also states what is *not* silently being claimed. For example, Ethernet fragmentation and retry timing are not part of the Raft election proof; protobuf bytes and peer addresses are not part of joint consensus; cryptographic correctness is not part of the service-lifecycle trust-bit abstraction. Such omissions are recorded in the conformance map.

21.4 How model reasoning changes CharlotteOS

The most valuable result is often a mismatch discovered while constructing the model. The correct response depends on which side was wrong: sometimes the manual or model overstated functionality; sometimes the code lacked a safety fence.

21.4.1 Authorization is controlled capability issuance

The local name service now implements a co-located production authorization path alongside its older public and reusable-key compatibility protocols. The signed loader binds an artifact-derived stable principal, exact address-space generation, and roles in a kernel-only table. IPC enqueue places that authenticated snapshot in the receive envelope; request memory cannot choose the caller identity.

The authorization model makes the intended boundary explicit. The catalog publishes a service generation and the maximum rights that may be delegated. A separate logical policy role maps a stable principal and service identity to an allowed-rights set and policy version. A decision is bound to the kernel-authenticated subject, requested rights, current service generation and policy version, and can be redeemed once for an attenuated connection. A co-located name/policy process performs decision and attenuated connection delegation as one serialized transition. Policy mutation and authorized publication require distinct roles, and issuance, denial, and delegation failure enter a bounded administrator-readable audit stream. A later split service needs an unforgeable single-use grant and must revalidate every binding at redemption. Policy and audit state are not yet durable or replicated, and the compatibility lookup opcodes must not be treated as this authorization boundary.

The model also prevents an attractive but false claim. A policy update blocks new issuance and invalidates unredeemed decisions, but cannot selectively retract a direct connection already delegated by the kernel. Hard revocation requires endpoint closure, a revocable proxy, or a future kernel lease object. This prospective revocation contract is safer than documenting authority that the implementation cannot enforce. The practical design is recorded in `docs/architecture/authorization-policy.md`.

21.4.2 IPC attachments are cross-registry transactions

The base IPC model intentionally collapses vector attachments. The `CharlotteIpcTransaction` refinement instead represents two or more moved entries, a live memory loan, an attached connection, the queue and the pending-call registry. Current vector calls carry the memory entries, whereas scalar calls carry connection attachments and may copy memory. The refinement deliberately composes these paths into a stronger rollback obligation; it does not claim that one current syscall carries every form. No request is visible until preparation commits, and every pre-commit failure restores all affected registries. Its unsafe rollback keeps one moved entry and a connection in the target after failure and must violate `RollbackRestoresAll`.

A reply-wait timeout is also modeled separately from terminal call completion. It reports only that no reply was observed during the wait and therefore cannot release a buffer still reachable by the kernel or receiver. The unsafe timeout transition releases that loan while the call remains pending and must violate `TimeoutRetainsLoan`.

21.4.3 Two-phase service publication

An earlier lifecycle abstraction treated replacement publication as one atomic swap. The implemented distributed catalog instead performs three observable steps: commit an inactive prepared generation, publish the local connection, and commit activation. Lookup intentionally excludes the inactive generation, so replacement has a temporary unavailable interval.

The model was changed to describe that real contract rather than claim continuous availability. It also exposed the code-level obligation that generation allocation must fail closed. Distributed generation saturation and local unchecked increment were replaced with checked allocation that returns an error without mutating the catalog. A delayed unregister is accepted only when both owner and generation still match, which preserves a replacement.

Formal safeguarding does not promise every desired availability property. Here it prevents the documentation from claiming atomic replacement and protects the generation safety that the implementation does provide.

21.4.4 Reliable-message session identity

The reliable-message service formerly initialized a wire session from the service generation and incremented that number when abandoning a retry epoch. Generation N after one abandonment could therefore use the same identity as generation $N + 1$ before abandonment. A bounded retired-session window also allowed a sufficiently old delayed SYN to replace newer receive state after it fell out of the window.

The session model represents a session as the ordered pair $(serviceGeneration, retryEpoch)$. The repaired wire encoding packs those values into disjoint 32-bit fields, accepts a replacement session only when the packed identity is well formed and strictly newer, and disables sends rather than wrapping at exhaustion. Two unsafe configurations retain the old flat-identity collision and receive-regression behaviors as required counterexamples.

21.4.5 Restart-safe Raft admission

Dynamic admission originally kept `joining` and `joining_from` only in process memory. A node could select an existing cluster as its authority, crash before membership became durable, restart from its launch-time singleton configuration, and campaign. The gap represented real CharlotteOS functionality, not merely a missing model action.

The repaired implementation persists a `JoinAdmission` record containing the selected anchor and the snapshot index that existed before admission. Construction restores the joining posture and disables elections. When the joint membership commits, the node first writes a newer membership-bearing snapshot and only then clears the durable admission record. This ordering covers both crash edges:

- before the membership snapshot is durable, restart retains the anchor fence and the node cannot campaign;
- after the snapshot is durable but before the fence-clear write, restart recognizes the strictly newer membership snapshot as completion evidence and safely clears the stale admission record.

Operational auto-join belongs to the durable DNS-owned Raft node. Volatile Raft nodes must not auto-join: without stable state, a process restart cannot prove which cluster has authority. The safe TLA+ specification includes crash and restart; an unsafe restart that forgets admission must violate `RestartPreservesAdmission`.

21.4.6 Retained regression patterns

Other negative specifications preserve compact versions of repaired faults:

Unsafe action	Failure represented	Required violation
`UnsafeMessageWake`	Message readiness and endpoint-close observation share one observer queue.	`CloseSignalImpliesClosed`
`UnsafeRollback`	A failed abstract attachment transaction leaves one moved entry and its attached connection in target registries.	`RollbackRestoresAll`
`UnsafeTimeoutRelease`	Reply timeout releases a loan while the pending call can still access it.	`TimeoutRetainsLoan`
`UnsafeRemoteAbort`	A remote LP removes and reaps a context that may still execute.	`ReapOnlyOffCpu`
`UnsafeSpawn DuringAbort`	A sibling publishes after the domain-abort snapshot.	`AbortingThreadsDoomed`
`UnsafeCancelReleases Buffer`	Cancellation releases a mutable buffer before the operation becomes terminal.	`NonTerminalBuffer RemainsLoaned`
`UnsafeTimerFire`	A watchdog reports timeout without rechecking a concurrently published work generation.	`TimeoutObservedNoWork`
`UnsafeObserveJoin`	A delayed join attaches its old handle to the replacement occupying the same TID.	`ObserverMatches CapturedHandle`
`UnsafeSpawnWrap`	Exhausted thread generation wraps and aliases a previously allocated identity.	`GenerationNeverReused`
`UnsafeBeginExclusive Map`	Exclusive DMA begins while CPU or other transfer authority still exists.	`ExclusiveDmaHasNo CpuAuthority`
`UnsafeStaleClose`	A recycled numeric ASID is closed through an old generation handle.	`ReplacementSurvives StaleHandle`
`UnsafeAllocate`	A hardware ASID is reassigned before stale translations are invalidated.	`NoDirtyTagReuse`
`DrainUnsafe`	A deferred interrupt wake ignores its route generation.	`NoStaleWakeDelivery`
`UnsafeStaleUnregister`	Cleanup from an old service generation removes its replacement.	`ReplacementSurvives StaleUnregister`
`UnsafeReplicate ToJoiner`	A non-selected peer supplies authority to a joining node.	`JoiningAcceptsOnly SelectedAnchor`
`UnsafeRestartForgets Admission`	Restart makes an incomplete joiner election-eligible.	`RestartPreservesAdmission`
Flat session allocation	Retry and restart epochs alias one wire session.	`SessionIdentityUnique`
`AcceptDelayedSession`	An older delayed SYN regresses receive state.	`ReceiveSessionMonotonic`

Unsafe action	Failure represented	Required violation
`UnsafeSetPolicy`	A non-administrator changes a policy rule.	`PolicyMutationAuthorized`
`UnsafeRedeemOtherPrincipal`	A decision for one subject is redeemed for another.	`MintBoundToPrincipal`
`UnsafeRedeemStalePolicy`	A decision survives a policy-version change.	`MintUsesCurrentPolicy`
`UnsafeRedeemStaleBinding`	A decision for an old service generation reaches its replacement.	`MintTargetsCurrentBinding`
`UnsafeAmplifyRights`	Redemption mints rights absent from the decision.	`NoRightsAmplification`

The check suite expects all twenty-three negative configurations to fail for the named reason. If one unexpectedly satisfies its invariant, the regression harness has lost sensitivity and the suite fails.

21.5 Concrete conformance and atomicity

TLA+ actions are atomic. Rust code usually crosses several structures, locks, or persistent writes. The conformance review must therefore identify a linearization point or represent the concrete intermediate states explicitly.

The mapping uses two correspondence classes:

Direct The abstract action follows one concrete transition while omitting data irrelevant to the invariant. For example, a successful fenced unregister directly checks owner and generation.

Abstract Several concrete operations are collapsed. This is valid only when intermediate states are unobservable or separately guarded. For example, scheduler retirement spans multiple tables, so the concrete `RETIREMENTS_IN_FLIGHT` state prevents address-space teardown during that interval.

Persistent ordering deserves the same care. In Raft admission, "snapshot membership and clear join fence" is one model transition, but two durable writes in Rust. Safety depends on snapshot flush preceding fence removal. A crash between those writes must map to a modeled state that remains safe.

The conformance document is not yet a machine-checked refinement proof. It is a reviewable refinement *obligation*: it names the functions and state that must be re-examined when either side changes. Code review should reject a model update that merely hides a new concrete intermediate state.

21.6 Validation

21.6.1 Running TLC

The repository wrapper runs the complete bounded suite:

```
docs/tla/check.sh /path/to/tla2tools.jar
```

Alternatively, `TLA2TOOLS_JAR` may name the tools archive. The script:

- runs all twenty-one safe configurations;
- enables action coverage and requires selected actions to have a positive successor count;
- rejects structural TLC warnings about variables, `EXCEPT` expressions, or incomplete successor states;
- runs twenty-three negative configurations and requires each one to fail on the named invariant after exercising the named unsafe action; and
- writes checkpoints and traces into a temporary directory so generated state does not become part of the source tree.

Action coverage is important. An invariant can pass because a transition is accidentally disabled. Requiring coverage does not prove that an action is correct, but it detects this common vacuity.

At the 15 August 2026 implementation synchronization point, representative fast configurations produced the following complete state graphs:

Model	Generated	Distinct	Depth
`CharlotteIPC`	6,118,645	1,254,153	13
`CharlotteCQ`	1,586,249	302,659	12
`CharlotteTimedWait`	22	15	8
`CharlotteScheduler`	4,308,577	819,972	22
`CharlotteThreadJoin`	46	39	10
`CharlotteIpc Transaction`	71	26	11
`Charlotte ServiceLifecycle`	689,321	74,232	37
`CharlotteDMA`	19,331,379	2,092,882	26
`CharlotteAuthorization`	144,598	54,705	12
`CharlotteRaftJoin`	2,029	520	13
`Charlotte ReliableMessage`	30	21	5

State counts are evidence about one configuration, not a quality score. A small model may encode the decisive identity argument, while a large model may contain substantial independent bookkeeping. The invariant, abstraction, and coverage matter more than graph size.

21.6.2 Implementation validation

TLC is followed by validation proportional to the affected code. The August synchronization work, including the 15 August domain-abort and memory-ownership repair, included:

- host tests for `catten-graft`, including restart before durable membership and restart between membership persistence and fence clear;
- host tests for `charlotte-protocol-msg`, including disjoint retry/restart sessions and fail-closed exhaustion;
- host tests for ownership-aware runtime cleanup and the shared `charlotte-lifecycle` thread-join and timed-wait classifiers;

- AArch64 release builds and warning-denied Clippy for the changed `raft`, `relmsg`, and `ns` services;
- AArch64 and x86-64 kernel checks, plus warning-denied AArch64 Clippy, for the domain-abort publication fence and memory-ownership paths;
- a four-LP AArch64 QEMU boot whose authoritative result reported all eighteen registered self-tests passing, including owner-LP retirement, DMA/SMMU, completion cancellation, and service crash/restart; and
- formatting, shell syntax, and diff-whitespace checks.

Boot and multi-guest tests remain necessary when a change affects the concrete scheduler, device, storage, or network integration. A TLC result cannot show that QEMU delivers an interrupt, that an object-store flush reaches the emulated disk, or that a manifest starts the intended services.

21.7 What a successful check means

A successful safe configuration means that TLC found no reachable violation of the declared invariants within that specification and its finite constants. It does *not* by itself establish:

- a proof that arbitrary larger bounds preserve the result;
- a mechanically checked refinement from Rust to TLA+;
- liveness, fairness, bounded latency, or eventual leader election unless explicitly expressed as temporal properties;
- correctness of omitted bytes, parsers, cryptography, memory mappings, hardware programming, or transport framing;
- correctness under storage failure modes below the modeled durable write interface; or
- freedom from ordinary implementation bugs such as arithmetic, aliasing, or unsafe-memory errors outside the modeled projection.

Conversely, a gap between code and model is actionable information. It can mean one of three things:

1. **Functionality is unsafe.** The Raft admission restart gap was of this kind and required a CharlotteOS change.
2. **The model overclaims.** Atomic service replacement was of this kind; the model was corrected to admit temporary unavailability.
3. **The behavior is outside scope.** It must be documented rather than silently attributed to the checked invariants.

Therefore a green TLC run is not a blanket "formally verified" label for CharlotteOS.

21.8 Workflow for changes

Any change to ownership, generations, persistence, retry identity, lifecycle, or distributed membership should be checked against this chapter's method. A practical review checklist is:

1. Search `docs/tla/CONFORMANCE.md` for the changed Rust function or state field.
2. Decide whether the change adds an action, changes an enabling condition, moves a linearization point, or changes durable/volatile classification.

3. Update the safe model and bounds. If repairing a discovered fault, retain the old behavior as an unsafe action and named negative configuration.
4. Add the new action to the required-coverage list in `docs/tla/check.sh`.
5. Run the entire TLC suite, not only the edited module, because shared documentation claims and layered Raft assumptions can drift together.
6. Update the conformance map with exact Rust functions and ordering.
7. Add implementation tests at the crash, retry, or stale-generation edges identified by the model.
8. Run target validation and the relevant boot or multi-node suite.
9. Record deliberate omissions and avoid converting a safety result into an unsupported liveness or availability claim.

Used this way, TLA+ is part of the engineering control system for CharlotteOS. It preserves the reasoning behind a safety fence, makes regressions executable, and forces the model, manual, code, and tests to describe the same functionality.

A Glossary

AS (Address Space) A page-table context identified by an `AddressSpaceId` (0 = kernel, >0 = user). Contains the `TTBR0_EL1` (user) and `TTBR1_EL1` (kernel) pointers on AArch64, or `CR3` on x86-64.

Block (verb) To remove a running thread from the scheduler and register its `Waker` on an `Observable`. The thread yields; when the event fires the waker re-admits it.

Capability An unforgeable per-domain handle that names a kernel object and carries rights. Answers: "May I perform this operation?"

Capability table A per-AS sparse table mapping capability indices to kernel objects. Used for the thread table, address-space table, and per-domain capability table.

Completion The terminal result of an asynchronous operation. The kernel tracks operation state and delivers results through polling, waitable completion objects, or CQ entries.

Connection capability A client-side authority to send or call an endpoint. Minted from an endpoint, typically by the name service.

CQ ring (Completion Queue) A shared-memory ring buffer for zero-syscall completion delivery from the kernel to userspace.

EL0 Exception Level 0 on AArch64. In CharlotteOS source and test names, also used as the architecture-neutral label for userspace, including x86-64 ring 3.

EL1 Exception Level 1 (AArch64). Kernel execution.

Endpoint A bounded server-side IPC receive object. Has an owning domain, a queue capacity, and a lifecycle.

Exit-observer An `Observer` registered on a `Thread`; fires when the thread is dropped (exits). The ABI's intended completion mechanism: `submit_worker` registers the capability as the worker's exit-observer.

HHDM (Higher Half Direct Mapping) A linear mapping of all physical memory into the kernel's virtual address space, provided by the bootloader (Limine).

IdTable A sparse `Vec<Option<T>>` with ID recycling. Used for the thread table, address-space table, and capability table.

IOMMU / SMMU I/O Memory Management Unit. Hardware that enforces DMA access permissions from devices, analogous to the MMU for CPU access.

IPI (Inter-Processor Interrupt) A signal from one LP to another. The kernel's IPI queues are bounded per-LP queues carrying typed commands.

LP (Logical Processor) A schedulable hardware execution context, usually one core or hardware thread. It is distinct from a sitas shard.

Memory object A page-backed kernel object with explicit ownership and capability semantics. Can be mapped, moved, lent, or copied between domains.

MMU Memory Management Unit. Hardware that enforces page-table permissions for CPU access.

Name service A userspace service that maps human-readable names to connection capabilities. The kernel does not understand service names.

Notification "State may have changed; inspect the corresponding object or queue." May be coalesced. Is not itself a result.

Observable / Observer The event-notification pattern. An `Observable` holds `Weak<dyn Observer>` references; when it fires, it calls `Observer::notify()`.

PerLp<T> A boxed slice of per-LP cells with lock wrapping. Interrupt-safe (masks interrupts on acquire). Use for data accessed from interrupt handlers or multiple LPs.

Protection domain An address space and authority boundary. Owns a capability table, memory mappings, threads, and failure state. Typically implemented as a process.

Quantum timer The per-LP periodic timer (10 ms default) that drives preemptive context switching in the `RoundRobin` scheduler.

Reaper The deferred-thread-cleanup mechanism: dead threads are moved to `DEAD_THREADS` and dropped from the next thread's context after the context switch.

Reply token One-shot authority to complete a blocking or deferred endpoint call. Created by the kernel, bound to the caller's pending call, consumable exactly once, invalidated by cancellation or teardown.

Shard A userspace ownership and execution domain. One executor runs its tasks; at most one task at a time mutates shard-owned state; cross-shard access goes through typed messages.

ShardLocal<T> A lock-free per-LP container using `UnsafeCell<T>` with owner-check and borrow-flag guard. sitas-derived. Use for single-LP, thread-only kernel data.

ShardMailbox<M> A typed bounded per-LP queue with `ShardSender<M>` (cloneable, any LP can send) and `ShardReceiver<M>` (single-consumer). First-class backpressure.

Supervisor Kernel-side component that creates and destroys protection domains, loads ELF images, delivers bootstrap capabilities, and handles domain teardown.

TrapFrame An architecture-specific snapshot of user register state at an exception. Syscall dispatch combines it with trusted per-thread or per-LP state to identify the caller.

Waker An `Observer` wrapping a `ThreadId`. When notified, calls `submit_ready_thread(tid)`. In userspace, a Rust `Waker` is a hint to repoll a future — it is not a kernel completion.

Yield To voluntarily give up the LP by calling `yield_lp()`. Saves the thread's context and lets the scheduler run the next ready thread.

B Implementation Status

This appendix is a snapshot of built and verified behavior. Source, tests, and repository history remain authoritative.

B.1 Implementation phases

Phase	Description	Status
1	Capability table, rights masks, object teardown	Done
2	Endpoint IPC v1: scalar send/call/receive/reply, connection delegation, reply tokens	Done
3	Userspace name/service manager, bootstrap delivery, ELF loader, supervisor, generation tracking	Done
4	First-class memory objects: allocate, map, unmap, close, ownership accounting	Done
5	Memory IPC: Copy, Move, BorrowRead, BorrowWrite, reply-bound revocation, cancellation, server-death recovery	Done
6	CQ subsystem normalization: operation IDs, detached submission, CQ_WAIT/CQ_WAKE, per-shard CQ partitioning, backlog batching, 32-byte completion records	Done
7	Sitas endpoint/CQ backend: endpoint readiness binding, unified shard wait, ShardExecutor (budgeted polling), ShardParker (spin free), per-shard CQ rings	Done
8	Userspace UART driver: delegated MMIO + IRQ, EL0 MMIO writes, interrupt-driven deferred reads, crash recovery	Done
9	Virtio-net driver: PCI discovery, MSI-X and IOMMU/SMMU delegation, feature negotiation, MAC/link read, virtqueue setup, and EL0 transmit submission. Smoltcp 0.13 adapter + TCP/IP service binary compile, and the frame demultiplexer routes IPv4/ARP to the tcpip service.	Two-guest TCG suite
10	Lock-free device interrupt delivery (deferred wake), idempotent scheduler wakes, sustained-window load rebalancing	Done
11	Userspace startup ABI: ELF `_start`, `entry!` macro, typed `Context`, fixed-width versioned launch header, explicit startup input	Done

12	x86-64 multi-LP ring-3 platform: shared service ABI, synchronous TLB shootdown, VT-d/AMD-Vi, NVMe/AHCI/virtio-blk, virtio-net/E1000E, discovery, and distributed DNS, smoltcp TCP/IP, HTTP reporting, signed deployment, migration, and dynamic membership	Four-LP and two-guest TCG suites
13	UTC time service: internal endpoint API, ISO 8601 formatting, connected UDP/NTP client, drift and uncertainty estimation, monotonic holdover, and object-store calibration persistence	Default boot-validated on both targets
14	Capability-scoped external data services: S3 Signature V4 over verified TLS with streaming object operations; Kafka idempotent producer, read-committed fixed-partition consumer, and transactional produce plus offset commit	Opt-in AArch64 Docker-fixture tests

B.2 Success criteria

1. Two EL0 domains communicate through a bounded endpoint — **Verified.**
2. Deferred async call while a shard continues other tasks — **Verified.**
3. Moved memory object — sender locked out until return — **Verified.**
4. Lending enforced by mappings, including death cleanup — **Verified.**
5. Single `CQ_WAIT` for endpoint + completions + device interrupts — **Verified.**
6. Terminal CQ results survive ring overflow (non-lossy backlog) — **Verified.**
7. Coalesced remote shard wakes (idempotent scheduler fix) — **Verified.**
8. Userspace UART driver with only delegated MMIO + IRQ — **Verified.**
9. Restart invalidates stale connections + device reset + op reconciliation — **Verified.**
10. Virtio-net data path with batched packet buffers — **Runtime-validated on macOS QEMU TCG.**
11. Fixed-width stable ABI representations (32-byte CQ entries) — **Implemented; not formally audited.**
12. Capability misuse and cancellation race adversarial tests — **Partial; six IPC error-path tests.**

B.3 Verified scope

- **AArch64:** boots and the synchronous self-tests pass. The EL0 service suite starts one shared node name service; Raft peers, echo, the service manager, NVMe/object store, UART, and their clients receive delegated connections to that registry. The deferred boot tests exercise registration, waitable lookup, restart, and live upgrade through `crt0/cmain`; the harness may print its synchronous success banner before those deferred tests finish. The sitas `basic_kv` binary runs with 2 shards on per-shard CQ rings.
- **x86-64:** boots with `-cpu max` on four LPs, enters ring 3, and runs native Rust service ELFs through the shared syscall and launch ABI. The tested service path includes naming, object storage, Raft persistence, service management, live upgrade, virtio-net, discovery, two-guest distributed DNS and smoltcp TCP exchange, the HTTP state report, signed deployment, cross-node invocation, migration,

retirement, `clusterctl`, and dynamic membership. NVMe, AHCI, and virtio-blk run behind Intel VT-d or AMD-Vi. The full cluster suite is qualified with four LPs under QEMU TCG.
A two-disk VMware Fusion appliance additionally qualifies UEFI boot, guest-visible VT-d, the virtual NVMe controller, blank-store formatting, idempotent installation of signed service artifacts, persistent restart, and the userspace E1000E path through link-up and hardware transmit completion. Paired VMware guests and VMXNET3 remain unqualified.

- **Completion subsystem:** submit, complete, poll, wait, cancel, close, `submit_worker` (exit-observer), CQ ring, CQ ring integrated with AS, non-lossy CQ overflow backlog, per-shard CQ partitioning, 32-byte entries.
- **Endpoint IPC:** bounded server queues, scalar messages, connection delegation with rights attenuation, memory-object attachments (move/lend/copy), deferred replies, reply-time connection delegation.
- **Device capabilities:** MmioRegion and Interrupt object types, user-accessible device mappings, GICv3 SPI and x86 MSI/MSI-X routing, and IRQ-to-CQ coalesced delivery (lock-free deferred wake).
- **Userspace drivers:** reference UART, NVMe, AHCI, virtio-blk, virtio-net, and E1000E drivers use delegated MMIO, interrupts, and DMA domains. The UART path additionally validates crash-test restart with device reset and operation reconciliation.
- **Syscall dispatch:** AArch64 SVC and x86-64 SYSCALL entry use the shared typed `SyscallNumber` enum. Caller identity comes from trusted thread and trap context, never a userspace parameter. Explicit authenticated IPC receive operations add the sender's exact address-space generation, signed-artifact principal, and roles without changing the legacy nine-register receive ABI.
- **Authorization:** the local name service hosts a default-deny policy engine. Separate roles protect policy changes and service publication. Connection issuance is attenuated and generation-fenced; an administrator can read its bounded audit stream. Policy and audit persistence, distributed replication, and hard revocation remain future work.
- **Scheduler:** RoundRobin, quantum timer, block/yield/wake, generation-qualified wakes and retirement, deferred reaper, per-LP thread pinning, certified runtime migration, and remote owner-LP abort.
- **Sitas integration:** sitas-core compiles for aarch64-none. sitas-charlotte links against the kernel's syscall ABI. Per-shard CQ rings mapped by the loader contract. ShardParker retires busy-waiting.
- **Protocol crates:** `charlotte-protocol-net`, `charlotte-protocol-msg`.
- **Networking:** smoltcp 0.13 adapter and TCP/IP service runtime-validated over the frame demultiplexer; reliable-message fragmentation to 64 KiB. Network capability, DHCP, discovery, cluster formation, HTTP, and time synchronization launch by default when a supported NIC is discovered; test features only add verifiers. S3 and Kafka service instances are separately provisioned because their endpoint, trust, credential, and data-scope profiles are not ambient machine configuration. Opt-in AArch64 tests validate RustFS and Apache Kafka over verified TLS.
- **Raft:** two-node election over local endpoint IPC, persistent single-voter restart, discovery-led join through joint consensus, and a two-guest replicated DNS catalog. Restart-safe admission has focused host tests and a persistent-storage boot-test manifest.
- **CI:** GitHub Actions builds both architectures with `-D warnings`, QEMU boot gate on AArch64.

B.4 Known limitations

- **External data-service compatibility:** S3 is fixture-tested with RustFS, while Dell EMC ECS namespace support is implemented but not yet qualified against an ECS installation. Kafka is limited to a directly assigned partition on one broker and lacks multi-broker routing, group rebalance, SASL, compression, and topic administration. Its versioned, digest-checked profile is already delivered only

to the broker connector as read-only launch memory; production secret injection and trust-anchor rotation remain controller work.

- **HVF does not honor hardware ASIDs.** Apple's Hypervisor.framework strips the ASID bits from `TTBR0_EL1` on read and does not use them to isolate user translations, so the kernel's ASID-based TLB fast path is unsafe there. Under `hvf_compat`, `switch_ctx` flushes the whole TLB on every context switch (`tlbi vmalle1is`) and the SVC handler attributes syscalls from a per-LP tracked logical ASID rather than a `ttbr0_el1` readback. See *AArch64 Port Status* for the full diagnosis. HVF also exposes no guest SMMU for the protected-DMA NVMe path. Its compatibility boot therefore runs 15 non-storage tests; the full 18-test suite and persistent-storage workflows require TCG.
- **x86-64 architectural limitations:** PCID/INVPCID and SMAP are not enabled; TLB shootdowns conservatively reload CR3. SMEP is enabled. VT-d multi-hop bridge scopes are rejected, and AMD-Vi fault delivery is polled until post-topology MSI setup is implemented.
- **Virtio-net runtime:** initialization and transmit submission are runtime-validated on macOS under QEMU TCG. HVF remains unsuitable for direct EL0 device MMIO. The receive/transmit data path, frame demultiplexing, reliable messages, distributed name service, and the TCP/IP service are validated; TX reclamation and batching remain incomplete.
- **General process loader:** the ELF loader is fixed-address for reference service images, not a general loader with argv/env/auxv-style setup.
- **Resource accounting:** per-domain quotas are not yet implemented; capability-table capacity is fixed.
- **Cross-machine Raft boundary:** two TCG guests share a private QEMU LAN (stream backend on AArch64, socket backend on x86-64); the reliable-message frame transport and remote Raft peer routing are connected during ordinary networked boots; `-dns-test` adds the cross-node validation workload and pass/fail result.
- **Observability HTTP:** the httpd keyhole aggregates the `observe` snapshot and per-service status into a JSON report served over the TCP/IP service during ordinary boots. `-http-test` adds QEMU host forwarding and validates the response; it does not enable the service.
- **IOMMU/SMMU integration:** coherent Arm SMMUv3 discovery via ACPI IORT, Intel VT-d via DMAR, and AMD-Vi via IVRS provide per-requester DMA-domain capabilities, memory pinning, IOVA map/unmap, synchronous invalidation, rollback, fault handling, and teardown. Storage and network drivers are QEMU-tested through these backends. Non-coherent SMMUv3, ATS/PRI, multiple-IOMMU topologies, VT-d multi-hop bridge scopes, and AMD-Vi MSI fault delivery remain incomplete.
- **Upgrade:** the EL0 service manager performs handoff and invokes the capability-checked replacement spawn path. It can fetch a replacement ELF from a well-known NVMe object, pass it as a memory capability, and fall back to the embedded recovery image. Structural ELF validation, immutable kernel snapshotting, and Ed25519 CLS2 signature and logical-name checks are implemented. Rollback and key rotation policy, multiple state objects in the reference manager, and manager-owned teardown remain future work.
- **Resource bounds:** endpoint queues and rings are bounded, but the non-lossy CQ overflow backlog has no hard memory bound.

C Lessons Learned

Booting the asynchronous kernel under QEMU exposed integration faults that unit tests and static checks had missed. This appendix records the most instructive ones.

C.1 Bugs found on hardware

C.1.1 Panic handler recursed through heap-dependent logging

Symptom: any fault during early boot silently triple-faulted instead of printing a panic message. **Root cause:** the panic handler used the normal logging path, which allocated from the kernel heap. A fault before the heap was ready caused the panic handler to fault, re-entering itself, recursing until stack overflow and triple fault. **Fix:** the panic handler now uses `early_logln!` (direct serial writes, no heap dependency). This unmasked every subsequent bug.

C.1.2 Blocked threads lost their execution context

Symptom: a thread that blocked in `wait()` restarted from its entry point when woken. **Root cause:** `block_thread` cleared the scheduler's `current_handle` before `yield_lp` saved the thread's context, so the save was skipped. **Fix:** a self-blocking thread stays `current_handle` until the yield saves its context.

C.1.3 RwLock held-across-call deadlocks

Symptom: `sleep()` worked with logging but hung without it (a heisenbug). **Root cause:** an `if let` binding extended an RwLock read guard's lifetime across a call that tried to acquire the write lock. **Fix:** bind the value out of the scrutinee first, so the temporary guard drops before the re-locking call.

C.1.4 Quantum timer grew without bound

Symptom: the per-LP timer queue contained thousands of quantum events after extended operation. **Root cause:** manual yields enqueued a new quantum event each time. **Fix:** an `armed` flag keeps exactly one quantum event in flight.

C.1.5 A thread cannot free its own kernel stack

Symptom: a returning kernel thread hung during teardown. **Root cause:** `ThreadContext::Drop` deallocated the stack while the thread was still executing on it. **Fix:** a deferred reaper moves dead threads to a zombie list and drops them from the next thread's context after the switch.

C.1.6 LA57 paging-mode mismatch

Symptom: a `#GP` with a non-canonical address on x86-64. **Root cause:** the address width was derived from CPUID (57-bit capability) instead of `CR4.LA57` (the active mode, 48-bit under Limine). **Fix:** derive from the active paging mode.

C.1.7 ASID-0 collision

Symptom: the first user thread faulted at its entry point. **Root cause:** the first user address space received ASID 0, which equals `KERNEL_ASID`, so the thread was created as a kernel thread (EL1), where PXN forbade execution of user code. **Fix:** pre-populate the table with the kernel AS at index 0.

C.1.8 x86-64 trampoline halted on thread return

Symptom: the async demo's worker completed but the coordinator never resumed. **Root cause:** the x86-64 trampoline entered a `hlt` loop when a thread returned, instead of calling `abort()`. **Fix:** make the x86-64 trampoline call `abort` on return.

C.1.9 Hypervisors that strip TTBR0 ASIDs break TLB isolation

Symptom: under Apple's Hypervisor.framework, EL0 tests failed with apparently unrelated data aborts, wrong syscall results, and deadline panics; TCG remained green. **Root cause:** HVF did not preserve the ASID bits in `TTBR0_EL1`. Because context switches relied on those tags instead of a full TLB flush, low user virtual addresses could resolve through another domain's stale translation. **Fix:** `hvf_compat` performs a whole-TLB flush on each context switch and obtains syscall identity from trusted per-LP state. **Lesson:** revalidate translation-tag assumptions on every hypervisor. See *AArch64 Port Status* for the detailed diagnosis.

C.2 Engineering lessons

1. **Test transitions, not only interfaces.** The difficult failures occurred where block, yield, wake, context save, and reclamation met. Tests must cross those boundaries.
2. **Keep failure paths independent.** Panic reporting must work before the heap and scheduler; reclamation must run from a live stack.
3. **Validate platform assumptions at runtime.** CPU capability does not imply active paging mode, and a hypervisor may not preserve hardware translation tags.
4. **Treat lock lifetime as part of control flow.** Bind values out of guards before calling code that may acquire the same lock.

D Research Lineages and Their Afterlives

The systems cited throughout this manual did not share one fate. Some remain active, some became products or standards, some were absorbed into a larger platform, and some ended after demonstrating one useful mechanism. This appendix separates those outcomes from CharlotteOS's own inheritance.

A research operating system need not replace Unix or Windows to matter. Its usual afterlife is selective: a fast IPC path, an authority model, a queue discipline, or a deployment abstraction survives at a different layer while the original system and its stronger unifying thesis disappear. Conversely, the adoption of one mechanism does not validate every claim made by the original architecture.

D.1 Capability kernels and confined Unix

KeyKOS, EROS, and CapROS. KeyKOS was a commercial capability system; EROS carried its object-capability and persistence ideas into research, and CapROS continues EROS as a small open-source project. They did not displace conventional operating systems, but their account of authority—possession, delegation, attenuation, confinement, and persistence as properties of references rather than global names—remains foundational. The CapROS project explicitly identifies itself as the continuation of EROS (https://www.capros.org/).

L4 and seL4. This lineage achieved both assimilation and independent survival. Earlier L4 variants demonstrated that microkernel IPC could be fast and shipped commercially in mobile devices. seL4 added machine-checked functional correctness and security properties, became open source in 2014, and acquired a foundation and commercial high-assurance ecosystem in 2020. The project's history records research prototypes, the HACMS deployment, proof milestones, and the later ecosystem (https://sel4.systems/About/history.html).

Capsicum and CHERI. Capability mechanisms also entered conventional systems in narrower forms. Capsicum has shipped in FreeBSD since version 9.0 and treats file descriptors as attenuable capabilities inside Unix (https://man.freebsd.org/cgi/man.cgi?query=capsicum&sektion=4). CHERI moved bounds and permissions into tagged pointers and processor support. Arm's Morello board and CheriBSD established a substantial experimental stack, while CHERI-RISC-V work continues toward standardisation (https://www.cl.cam.ac.uk/research/security/ctsrd/cheri/). These are complementary outcomes: Capsicum confines named Unix resources and CHERI constrains memory references; neither is by itself a complete object-capability operating system.

D.1.1 CharlotteOS inheritance

CharlotteOS adopts the strong local authority rule: an opaque, typed object capability is authority, may carry attenuated rights, and is not a name, notification, completion, or unit of work. Reply tokens are linear authority; memory, device, DMA, observer, and bootstrap access are explicitly delegated. The present kernel does *not* retain a general derivation tree, prove confinement, or provide CHERI-style protection for arbitrary pointers. This is an EROS/seL4-inspired object model, not an seL4 refinement or a CHERI substitute.

D.2 IPC, typed channels, and ownership

Mach. Pure Mach did not become the dominant Unix organisation, but it was absorbed into Apple XNU. XNU retains Mach tasks, virtual memory, ports, and IPC while co-locating BSD, networking, filesystems, and I/O components in one kernel for performance. Apple's description is candid about this hybrid outcome (Apple's XNU architecture note). The lesson is double-edged: message-mediated authority survived, while the pure user-server decomposition did not.

Singularity. Microsoft ended the standalone research OS and evolved its design into the internal Midori project. Midori never became a commercial OS, although Microsoft reports that it ran part of the company's natural language search service (https://www.microsoft.com/en-us/research/project/singularity/). Typed channel contracts, software-isolated processes, manifest-checked components, and ownership transfer through an exchange heap outlived the platform as research ideas. Managed-language enforcement did not become a mainstream OS boundary, but ownership types and message-oriented runtimes became normal engineering tools.

Xous. Xous remains a working Rust microkernel for embedded devices, not merely an historical analogy. Its userspace servers and distinct scalar, send, lend, and mutable-lend messages make it CharlotteOS's closest direct IPC ancestor (https://xous.dev/kernel/).

D.2.1 CharlotteOS inheritance

CharlotteOS takes Xous's isolated-server model and its separation of scalar from memory-bearing IPC. It takes Singularity's ownership-moving discipline but enforces it through capabilities, page mappings, tear-down state machines, and DMA rules rather than a managed-language verifier. Copy, move, read-only borrow, and writable borrow have explicit failure, cancellation, restoration, and rollback behavior. Typed protocol crates provide channel contracts above an ABI that still validates untyped words at the kernel boundary.

D.3 Multicore systems, runtimes, and fast I/O

Barrelfish and Arrakis. Barrelfish argued that a multicore machine should be treated as a distributed system with explicit messages and topology-aware state placement. The project now identifies itself as inactive (https://barrelfish.org/download.html). Arrakis gave applications virtualised device queues while leaving allocation and protection policy in the kernel; it merged back into Barrelfish in 2015. The code line ended, but per-core runtimes, hardware queue virtualisation, and control/data-plane separation became widespread design patterns.

IX, DPDK, and Seastar. IX was a research dataplane OS. Its run-to-completion, per-core, batched approach continued in datacenter runtimes. DPDK succeeded by becoming a Linux Foundation userspace packet-processing project with production device support (https://www.dpdk.org/about/). Seastar remains an active shared-nothing, shard-per-core, futures runtime used by ScyllaDB (https://seastar.io/). In this family, the ideas prospered as libraries and queues on Linux more than as replacement operating systems.

Windows completion ports, BSD `kqueue`, Linux `epoll`, and Linux `io_uring` show the same selective adoption. Submission, retained completion, aggregation, and batching became normal interfaces without making notification, readiness, completion, IPC, and authority one thing.

D.3.1 CharlotteOS inheritance

Sitas is the direct shard execution discipline and is Seastar-like: one owner per shard, cooperative futures, bounded messages, and deliberate placement. Barrelfish supplies the analogy for explicit cross-LP coordination. IX, DPDK, and Arrakis inform userspace drivers, batched queues, IOMMU protection, and buffer ownership. CharlotteOS makes a different authority choice from Arrakis: devices are delegated to isolated driver services, not directly to ordinary applications. Completion queues retain terminal operation records and remain separate from endpoint IPC and Rust `Wakers`.

D.4 Service recovery

MINIX 3, CuriOS, and Pebble explored minimally privileged userspace services, drivers, supervision, and restart. Nooks and SafeDrive instead tried to contain faulty extensions while preserving compatibility with in-kernel drivers. MINIX remains a research and teaching system; the other projects are completed prototypes. Their broad result—isolate components and supervise them—is now conventional. Their harder problem did not disappear: hardware, DMA, persistent state, and externally visible effects prevent arbitrary restart from being transparent.

CharlotteOS adopts the strong containment boundary and the honest recovery contract. The supervisor can revoke capabilities and borrows, reset a device, reconcile operations and DMA, start a fresh generation, and require clients to resolve it again. A stateful upgrade transfers explicitly serialised state; it does not inherit a process image or promise uninterrupted execution. This is closer to MINIX's supervision goal than to transparent recovery.

D.5 From distributed operating systems to cluster managers

Amoeba. Amoeba's last official release was 5.3 in 1996. Its processor pools, capability-protected objects, location-independent RPC, and FLIP network did not become a maintained product platform (https://www.cs.vu.nl/pub/amoeba/manuals/usr.pdf). The component ideas did become ordinary: pooled compute, object references, RPC, directories, and replicated services. The part modern systems mostly rejected was the promise that a distributed computer should look like one failure-free machine.

Plan 9 and Inferno. Bell Labs development ended, but Plan 9 has community descendants. Its 9P protocol remains in Linux, including a Virtio transport (https://www.kernel.org/doc/html/latest/filesystems/9p.html). Per-process namespaces and protocol-served resources stayed influential while Plan 9 and Inferno themselves remained niche.

Jini and MOSIX. Sun's Jini became Apache River; Apache retired River in 2022 (https://attic.apache.org/projects/river.html). The openMosix project officially ended in 2008 (https://sourceforge.net/p/openmosix/news/2008/02/openmosix-project-ends/). Discovery, leases, tuple spaces, automatic placement, and migration were not lost. They reappeared in registries, brokers, serverless platforms, and cluster schedulers. Modern systems usually reschedule a container or actor and reconstruct service state instead of migrating an arbitrary process image.

Borg, Omega, and Kubernetes. Borg remains Google's internal production cluster manager; Omega explored a concurrent control plane; and Kubernetes directly inherited their lessons about declarative desired state, scheduling, services, labels, and self-healing. Google's own account describes that lineage (https://kubernetes.io/blog/2015/04/borg-predecessor-to-kubernetes/). The

cluster became the deployment target, but above commodity kernels and containers rather than inside a distributed OS kernel.

Focused mechanisms. Raft succeeded as a reusable replicated-state- machine protocol with many implementations (https://raft.github.io/). SWIM-style gossip survived in memberlist and Consul. Nix made hash-identified immutable software stores practical. TUF and Uptane became maintained update security frameworks; Uptane is now a versioned standard under Linux Foundation governance (https://uptane.org/learn-more/about). Sigstore, Cosign, and Notary brought signed identity and provenance into cloud artifact workflows.

D.5.1 CharlotteOS inheritance

CharlotteOS separates a human-readable name from the connection capability returned by policy-controlled lookup. Its `dns` catalog is a Raft state machine across two guests. Remote calls use request identity, generation fencing, duplicate handling, deadlines, and an explicit uncertain outcome. The cluster slice verifies Ed25519-signed ELF notes, commits placement records, and reassigns a stateless service. It is Amoeba-like in interface and Kubernetes-like in deployment intent, but remains bounded: automatic placement, a shared content-addressed artifact store, production key rotation, general stateful migration, and distributed capability revocation are open.

D.6 The synthesis and the rejected inheritances

Relationship	Source ideas	CharlotteOS status
Direct source	Xous IPC and memory messages; Sitas shard ownership and async execution	Implemented locally, combined with CharlotteOS capabilities, memory objects, completion queues, and teardown semantics
Adopted and changed	EROS/seL4 authority; Singularity ownership transfer; Barrelfish messages; MINIX recovery; completion-ring I/O	Implemented in bounded local forms; no compatibility or refinement claim
Focused protocol	Raft replication; Ed25519/SHA-256 artifact verification	Implemented and tested, including cross-guest catalog replication and signed deployment
Architectural influence	Amoeba invocation; IX/DPDK/Arrakis fast paths; Borg/Kubernetes placement; Nix/TUF/Uptane artifacts and trust	Partial cluster slice or future work
Rejected or narrowed	Failure-transparent remote calls; arbitrary process migration; language-only isolation; direct device access by ordinary apps	Remote uncertainty is visible; migration is protocol-specific; MMU/IOMMU and capabilities enforce isolation; drivers retain device authority

E Architectural Redirections

This appendix records decisions that redirected early experiments from a "universal completion-capability" model to the three-primitive architecture in this manual. Later chapters describe the resulting implementation.

E.1 Retained mechanisms

The following early mechanisms remain part of the architecture:

- AArch64 EL0/SVC path.
- Caller identity derived from the current trap/thread context.
- Per-address-space capability tables.
- Shared CQ ring.
- Non-lossy kernel CQ backlog.
- `CQ_WAIT` blocking through scheduler/observer machinery.
- Bounded queues and explicit `WouldBlock`.
- No-std sitas split and CharlotteOS backend.
- Per-shard executor model.
- Typed in-kernel `ShardMailbox<M>`.
- Closure-based `ShardLocal<T>` with restricted use.
- Segment-aware ELF loading.
- QEMU boot CI across AArch64 and x86-64.

E.2 What to reframe

These components are correct but their *role* in the architecture has shifted:

- **Completion capability work** is the **operation completion subsystem**, not the universal service IPC subsystem. Use completion queues for kernel and device operations; use endpoints for cross-domain service calls.
- **Observer/waker code** is internal scheduling machinery, not the public semantic identity of all async objects. A Waker is a hint to repoll a future; it is not a completion record.
- **IPI queue closures** are a temporary kernel work mechanism, not the long-term cross-domain messaging model. Use typed mailboxes or closed kernel enums for subsystem-specific work.
- **The earlier mailbox syscall ABI** is a smoke-test interface only. The stable ABI is endpoint IPC.

E.3 Replacements adopted

E.3.1 Stop expanding raw mailbox syscalls

A mailbox tied directly to LP identity is not a complete userspace service abstraction: it exposes placement where authority should be exposed, lacks protocol identity, conflates shard transport with domain IPC, and lacks move/lend semantics. Endpoint and connection capabilities are therefore independent of LP identity. Clients address the service connection, not a target LP.

E.3.2 Do not allocate one completion capability per service request

For userspace service calls, the natural object is a reply token. For high-rate kernel operations, per-operation capabilities create unnecessary table pressure. The design uses reply tokens for endpoint calls; operation IDs plus CQ entries for ordinary kernel operations; first-class operation capabilities only where independent authority is required.

E.3.3 Separate readiness, notification, and completion

Their contracts differ: readiness means progress may be possible; notification means inspect state; completion means an operation changed state and has result data. Keep one aggregated wait mechanism, but define distinct object semantics.

E.3.4 Replace arbitrary cross-LP closures

Boxed `FnOnce()` work transported through the IPI layer is hard to account, difficult to inspect, and brittle under backpressure. The preferred direction is closed kernel enums for mandatory kernel RPCs and typed mailbox instances for subsystem-specific work.

E.3.5 Do not equate shard wake with IPI delivery

Remote wake mapping directly to `send_ipi(target_lp)` can generate excessive interrupts. The implemented rule is to queue first, mark pending, and send a coalesced reschedule IPI only when the target cannot otherwise observe the transition.

E.3.6 Introduce memory objects before rich IPC payloads

Move/lend semantics require durable object identity rather than transient pointers. CharlotteOS introduced memory-object capabilities, mapping, ownership transfer, temporary lending, lifecycle revocation, and DMA integration before promoting rich IPC payloads.

E.3.7 Treat service discovery as userspace policy

String-based lookup remains outside the kernel. The userspace name service receives bootstrap authority and delegates connection capabilities.